Striver's 180 DSA Patterns

Java solution notes — expanded edition

Explanation and function code only, grouped into the 17 modules on the pattern sheet: arrays, hashing, binary search, sliding window and two pointers, recursion and backtracking, linked lists, stacks and queues, greedy, heaps, binary trees, BSTs, graphs, dynamic programming, tries, strings, bit manipulation, and mathematics.

229 entries, including closely related variants for extra practice. Each note presents brute-force, better, and optimal approaches where they add learning value, with explanation, complexity, and function code. Shared node types used later:

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
}

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}
```

These are original study notes aligned to that sheet. The live site did not return the in-app notes or the full problem table.

Arrays

In-place scans, prefix markers, and divide-and-conquer on arrays. The open group on the sheet (majority, Kadane, majority II, maximum product) is the linear-candidate pattern.

1. Set Matrix Zeroes

Brute force

Copy the matrix, scan the copy for zeros, and for each zero in the copy write entire row and column to zero in the original. Safe because reads use the copy, but O(mn) extra space.

Complexity: O(mn) time, O(mn) space.

```
void setZeroesBrute(int[][] a) {
    int n = a.length, m = a[0].length;
    int[][] cp = new int[n][m];
    for (int i = 0; i < n; i++)
        System.arraycopy(a[i], 0, cp[i], 0, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (cp[i][j] == 0) {
                for (int r = 0; r < n; r++) a[r][j] = 0;
                for (int c = 0; c < m; c++) a[i][c] = 0;
            }
}
```

Better

Use boolean arrays rowMark[n] and colMark[m]. First pass records which rows and columns must be zeroed, second pass applies them. Linear time with O(m + n) auxiliary bits.

Complexity: O(mn) time, O(m + n) space.

```
void setZeroesMarks(int[][] a) {
    int n = a.length, m = a[0].length;
    boolean[] row = new boolean[n], col = new boolean[m];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] == 0) { row[i] = true; col[j] = true; }
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (row[i] || col[j]) a[i][j] = 0;
}
```

Optimal

Use the first row and first column as markers. If matrix[i][j] is 0, row i and column j must become 0. Writing those zeros immediately destroys later information, so record the row and column in matrix[i][0] and matrix[0][j]. The first cell is shared by row 0 and column 0, so keep two booleans for whether that row and column originally contained a zero. After the markers are set, clear the inner cells, then clear row 0 and column 0 if needed.

Complexity: O(mn) time, O(1) extra space.

```
void setZeroes(int[][] a) {
    int n = a.length, m = a[0].length;
    boolean row0 = false, col0 = false;
    for (int j = 0; j < m; j++) if (a[0][j] == 0) row0 = true;
    for (int i = 0; i < n; i++) if (a[i][0] == 0) col0 = true;
    for (int i = 1; i < n; i++)
        for (int j = 1; j < m; j++)
            if (a[i][j] == 0) { a[i][0] = 0; a[0][j] = 0; }
    for (int i = 1; i < n; i++)
        for (int j = 1; j < m; j++)
            if (a[i][0] == 0 || a[0][j] == 0) a[i][j] = 0;
    if (row0) for (int j = 0; j < m; j++) a[0][j] = 0;
    if (col0) for (int i = 0; i < n; i++) a[i][0] = 0;
}
```

2. Pascal's Triangle

Brute force

For each cell (r, c) compute C(r, c) with the multiplicative formula using long arithmetic. Correct but recomputes many overlapping combinations.

Complexity: O(numRows^3) time, O(numRows^2) space for output.

```
List<List<Integer>> generateBrute(int numRows) {
    List<List<Integer>> tri = new ArrayList<>();
    for (int r = 0; r < numRows; r++) {
        List<Integer> row = new ArrayList<>();
        for (int c = 0; c <= r; c++) row.add((int) nCr(r, c));
        tri.add(row);
    }
    return tri;
}
long nCr(int n, int r) {
    if (r > n - r) r = n - r;
    long ans = 1;
    for (int i = 1; i <= r; i++) ans = ans * (n - r + i) / i;
    return ans;
}
```

Optimal

Each value is the sum of the two values above it. Row r has r + 1 entries. The ends are 1. Every inner entry is the sum of the two neighbors in the previous row, which is the same recurrence as C(r, c) = C(r - 1, c - 1) + C(r - 1, c). Building row by row avoids factorial overflow and produces every row the problem asks for.

Complexity: O(numRows^2) time and space.

```
List<List<Integer>> generate(int numRows) {
    List<List<Integer>> tri = new ArrayList<>();
    for (int r = 0; r < numRows; r++) {
        List<Integer> row = new ArrayList<>();
        for (int c = 0; c <= r; c++) {
            if (c == 0 || c == r) row.add(1);
            else row.add(tri.get(r - 1).get(c - 1) + tri.get(r - 1).get(c));
        }
        tri.add(row);
    }
    return tri;
}
```

3. Next Permutation

Brute force

Generate permutations in lexicographic order (next_permutation on index array) until the sequence matches the current array, then take the following permutation. Factorial time.

Complexity: O(n! * n) time, O(n) space.

```
void nextPermutationBrute(int[] a) {
    int n = a.length;
    Integer[] idx = new Integer[n];
    for (int i = 0; i < n; i++) idx[i] = i;
    int[] b = a.clone();
    do {
        for (int i = 0; i < n; i++) a[i] = b[idx[i]];
        if (Arrays.equals(a, b)) break;
    } while (nextPerm(idx));
    do {
        for (int i = 0; i < n; i++) a[i] = b[idx[i]];
    } while (nextPerm(idx));
}
boolean nextPerm(Integer[] p) {
    int i = p.length - 2;
```

```
    while (i >= 0 && p[i] >= p[i + 1]) i--;
    if (i < 0) return false;
    int j = p.length - 1;
    while (p[j] <= p[i]) j--;
    int t = p[i]; p[i] = p[j]; p[j] = t;
    for (int l = i + 1, r = p.length - 1; l < r; l++, r--) {
        t = p[l]; p[l] = p[r]; p[r] = t;
    }
    return true;
}
```

Optimal

Find the longest non-increasing suffix, then the next pivot. The next permutation is the smallest rearrangement that is still larger than the current one. Scan from the right for the first descent: index i where a[i] < a[i + 1]. The suffix after i is descending, so it is the last permutation of those values. Swap a[i] with the smallest value to its right that is still larger than a[i], then reverse the suffix so it becomes ascending. If no descent exists, the array is the last permutation; reverse the whole array.

Complexity: O(n) time, O(1) space.

```
void nextPermutation(int[] a) {
    int i = a.length - 2;
    while (i >= 0 && a[i] >= a[i + 1]) i--;
    if (i >= 0) {
        int j = a.length - 1;
        while (a[j] <= a[i]) j--;
        int t = a[i]; a[i] = a[j]; a[j] = t;
    }
    for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {
        int t = a[l]; a[l] = a[r]; a[r] = t;
    }
}
```

4. Kadane's Algorithm

Brute force

For every subarray [i, j], compute the sum and track the maximum. All O(n^2) segments are examined.

Complexity: O(n^2) time, O(1) extra space.

```
int maxSubArrayBrute(int[] a) {
    int best = Integer.MIN_VALUE;
    for (int i = 0; i < a.length; i++) {
        int sum = 0;
        for (int j = i; j < a.length; j++) {
            sum += a[j];
            best = Math.max(best, sum);
        }
    }
    return best;
}
```

Optimal

Best subarray ending at i extends the previous ending or starts fresh. Any maximum subarray ends at some index. The best one ending at i is a[i] plus the best ending at i - 1, unless that previous sum is negative, in which case it only hurts and we start at a[i]. Tracking that running choice and the best seen so far covers every ending. If every value is negative, the answer is the largest single element, because the running sum restarts at each element.

Complexity: O(n) time, O(1) space.

```
int maxSubArray(int[] a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);
        best = Math.max(best, cur);
    }
    return best;
```

```
}
```

5. Sort an Array of 0s, 1s, and 2s

Brute force

Use nested loops to bubble 2s to the end and 0s to the front (or run a generic O(n^2) sort). Correct but quadratic.

Complexity: O(n^2) time, O(1) space.

```
void sortColorsBrute(int[] a) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] > a[j]) {
                int t = a[i]; a[i] = a[j]; a[j] = t;
            }
}
```

Better

Count how many 0s, 1s, and 2s appear, then overwrite the array with three blocks. Two passes, constant extra space besides three counters.

Complexity: O(n) time, O(1) extra space.

```
void sortColorsCount(int[] a) {
    int c0 = 0, c1 = 0, c2 = 0;
    for (int x : a) {
        if (x == 0) c0++;
        else if (x == 1) c1++;
        else c2++;
    }
    int i = 0;
    while (c0-- > 0) a[i++] = 0;
    while (c1-- > 0) a[i++] = 1;
    while (c2-- > 0) a[i++] = 2;
}
```

Optimal

Dutch national flag: three regions in one pass. Keep the invariant that [0, low) is 0s, [low, mid) is 1s, and (high, n) is 2s. mid walks the unknown middle. A 0 swaps into the zero region and both low and mid advance. A 1 is already in place, so only mid advances. A 2 swaps into the two region and high shrinks; mid stays, because the swapped-in value is still unknown.

Complexity: O(n) time, O(1) space.

```
void sortColors(int[] a) {
    int low = 0, mid = 0, high = a.length - 1;
    while (mid <= high) {
        if (a[mid] == 0) {
            int t = a[low]; a[low] = a[mid]; a[mid] = t;
            low++; mid++;
        } else if (a[mid] == 1) {
            mid++;
        } else {
            int t = a[mid]; a[mid] = a[high]; a[high] = t;
            high--;
        }
    }
}
```

6. Best Time to Buy and Sell Stock

Brute force

For each buy day i and sell day j > i, maximize prices[j] - prices[i]. Every transaction pair is checked.

Complexity: O(n^2) time, O(1) space.

```
int maxProfitBrute(int[] prices) {
    int best = 0;
    for (int i = 0; i < prices.length; i++)
```

```
        for (int j = i + 1; j < prices.length; j++)
            best = Math.max(best, prices[j] - prices[i]);
    return best;
}
```

Optimal

One pass: cheapest buy so far, best sell after it. A sale on day i must use some earlier day as the buy. The best buy for that sale is the minimum price on days 0 through i - 1. Scanning left to right, keep that minimum and the best difference. One transaction means we never need a second buy after the sell.

Complexity: O(n) time, O(1) space.

```
int maxProfit(int[] prices) {
    int best = 0, buy = prices[0];
    for (int i = 1; i < prices.length; i++) {
        best = Math.max(best, prices[i] - buy);
        buy = Math.min(buy, prices[i]);
    }
    return best;
}
```

7. Rotate Image

Brute force

Allocate a new n x n matrix and fill result[j][n - 1 - i] = a[i][j]. Then copy back into a if in-place output is required.

Complexity: O(n^2) time, O(n^2) extra space.

```
void rotateBrute(int[][] a) {
    int n = a.length;
    int[][] b = new int[n][n];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            b[j][n - 1 - i] = a[i][j];
    for (int i = 0; i < n; i++)
        System.arraycopy(b[i], 0, a[i], 0, n);
}
```

Optimal

Transpose, then reverse each row. A 90-degree clockwise rotation sends (i, j) to (j, n - 1 - i). Doing that with a cycle of four cells works, and so does a two-step rewrite: transpose turns (i, j) into (j, i), then reversing each row turns (j, i) into (j, n - 1 - i). Both steps are in place.

Complexity: O(n^2) time, O(1) space.

```
void rotate(int[][] a) {
    int n = a.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            int t = a[i][j]; a[i][j] = a[j][i]; a[j][i] = t;
        }
    for (int i = 0; i < n; i++)
        for (int l = 0, r = n - 1; l < r; l++, r--) {
            int t = a[i][l]; a[i][l] = a[i][r]; a[i][r] = t;
        }
}
```

8. Merge Intervals

Brute force

Repeatedly scan for two overlapping intervals, merge them into one, and restart until no overlap remains. Simple but can degrade to O(n^2) merges.

Complexity: O(n^2) time, O(n) space for the working list.

```
int[][] mergeBrute(int[][] intervals) {
    List<int[]> list = new ArrayList<>();
    for (int[] iv : intervals) list.add(new int[]{iv[0], iv[1]});
    boolean changed = true;
```

```
    while (changed) {
        changed = false;
        for (int i = 0; i < list.size(); i++) {
            for (int j = i + 1; j < list.size(); j++) {
                int[] a = list.get(i), b = list.get(j);
                if (a[0] <= b[1] && b[0] <= a[1]) {
                    int[] m = new int[]{Math.min(a[0], b[0]), Math.max(a[1], b[1])};
                    list.remove(j);
                    list.remove(i);
                    list.add(m);
                    changed = true;
                    break;
                }
            }
            if (changed) break;
        }
    }
    return list.toArray(new int[list.size()][]);
}
```

Optimal

Sort by start, then absorb overlaps into the last interval. After sorting by start time, an interval overlaps the previous merged interval exactly when its start is no later than that interval's end. Extend the end to the farther of the two ends. If it starts after the current end, it is a new disjoint interval. Sorting makes one left-to-right pass enough.

Complexity: O(n log n) time, O(n) space for the answer.

```
int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (x, y) -> Integer.compare(x[0], y[0]));
    List<int[]> out = new ArrayList<>();
    int[] cur = intervals[0];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] <= cur[1]) cur[1] = Math.max(cur[1], intervals[i][1]);
        else { out.add(cur); cur = intervals[i]; }
    }
    out.add(cur);
    return out.toArray(new int[out.size()][]);
}
```

9. Merge Two Sorted Arrays Without Extra Space

Brute force

Copy both arrays into a third buffer, merge with two pointers into that buffer, then copy the sorted result back into a and b. Uses O(n + m) auxiliary space.

Complexity: O(n + m) time, O(n + m) extra space.

```
void mergeExtraSpace(int[] a, int[] b) {
    int n = a.length, m = b.length;
    int[] tmp = new int[n + m];
    int i = 0, j = 0, k = 0;
    while (i < n && j < m)
        tmp[k++] = a[i] <= b[j] ? a[i++] : b[j++];
    while (i < n) tmp[k++] = a[i++];
    while (j < m) tmp[k++] = b[j++];
    for (int t = 0; t < n; t++) a[t] = tmp[t];
    for (int t = 0; t < m; t++) b[t] = tmp[n + t];
}
```

Optimal

Gap method: compare elements a fixed distance apart. Treat the two arrays as one logical sequence of length n + m. Start with gap = ceil((n + m) / 2). Swap any inversion between positions i and i + gap, then shrink the gap the same way shell sort does, down to 1. When the gap is 1, every adjacent inversion has been removed, so the logical sequence is sorted and both arrays stay in place.

Complexity: O((n + m) log(n + m)) time, O(1) extra space.

```
void merge(int[] a, int[] b) {
    int n = a.length, m = b.length, gap = (n + m + 1) / 2;
    while (gap > 0) {
        int i = 0, j = gap;
        while (j < n + m) {
            if (at(a, b, n, i) > at(a, b, n, j)) swap(a, b, n, i, j);
            i++; j++;
        }
        if (gap == 1) break;
        gap = (gap + 1) / 2;
    }
}
int at(int[] a, int[] b, int n, int i) {
    return i < n ? a[i] : b[i - n];
}
void swap(int[] a, int[] b, int n, int i, int j) {
    int vi = at(a, b, n, i), vj = at(a, b, n, j);
    if (i < n) a[i] = vj; else b[i - n] = vj;
    if (j < n) a[j] = vi; else b[j - n] = vi;
}
```

10. Find the Duplicate Number

Brute force

Sort the array (or mark indices in an auxiliary boolean array of size n) and find the value that appears twice. Modifies order or uses O(n) extra bits.

Complexity: O(n log n) time if sorting, O(n) space for marks.

```
int findDuplicateBrute(int[] nums) {
    int[] cnt = new int[nums.length];
    for (int x : nums) {
        if (++cnt[x] > 1) return x;
    }
    return -1;
}
```

Optimal

Floyd cycle detection on the index-to-value graph. Values lie in 1..n and the array has n + 1 cells, so following i -> nums[i] is a functional graph with exactly one cycle. The cycle entrance is the duplicated value, because two indices point at it. The first phase meets inside the cycle. The second phase starts one pointer at nums[0]; both move one step, and they meet at the entrance. This does not modify the array.

Complexity: O(n) time, O(1) space.

```
int findDuplicate(int[] nums) {
    int slow = nums[0], fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);
    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

11. Repeat and Missing Number

Brute force

Use a boolean array or hash set expecting values 1..n. The value seen twice is the repeat; the value never seen is the missing one.

Complexity: O(n) time, O(n) space.

```
int[] findErrorNumsBrute(int[] a) {
    int n = a.length;
    boolean[] seen = new boolean[n + 1];
    int repeat = -1, missing = -1;
    for (int v : a) {
        if (seen[v]) repeat = v;
        seen[v] = true;
    }
    for (int v = 1; v <= n; v++)
        if (!seen[v]) missing = v;
    return new int[]{repeat, missing};
}
```

Optimal

Two equations from sum and sum of squares. The array should be 1..n, but one value x is repeated and one value y is missing. The difference of sums is x - y. The difference of squares is x^2 - y^2 = (x - y)(x + y). Divide to get x + y, then solve the two linear equations. Use long so n up to 10^5 does not overflow.

Complexity: O(n) time, O(1) space.

```
int[] findErrorNums(int[] a) {
    long n = a.length, sum = 0, sq = 0;
    for (int v : a) { sum += v; sq += (long) v * v; }
    long diff = sum - n * (n + 1) / 2;
    long sqDiff = sq - n * (n + 1) * (2 * n + 1) / 6;
    long sumXY = sqDiff / diff;
    int repeat = (int) ((diff + sumXY) / 2);
    int missing = (int) (sumXY - repeat);
    return new int[]{repeat, missing};
}
```

12. Count Inversions

Brute force

Count every pair i < j with a[i] > a[j] with nested loops. Direct definition, quadratic.

Complexity: O(n^2) time, O(1) extra space.

```
long inversionCountBrute(int[] a) {
    long inv = 0;
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] > a[j]) inv++;
    return inv;
}
```

Optimal

Count cross inversions while merging. An inversion is a pair i < j with a[i] > a[j]. Pairs that sit entirely in the left or right half are counted by recursion. During the merge, both halves are sorted, so if left[i] is greater than right[j], it is also greater than every remaining element of the right half. Add that whole block at once, then merge as in merge sort.

Complexity: O(n log n) time, O(n) space.

```
long inversionCount(int[] a) {
    return sort(a, 0, a.length - 1, new int[a.length]);
}
long sort(int[] a, int l, int r, int[] tmp) {
    if (l >= r) return 0;
    int m = (l + r) >>> 1;
    long inv = sort(a, l, m, tmp) + sort(a, m + 1, r, tmp);
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];
        else { tmp[k++] = a[j++]; inv += m - i + 1; }
    }
    while (i <= m) tmp[k++] = a[i++];
```

```
    while (j <= r) tmp[k++] = a[j++];
    for (int t = l; t <= r; t++) a[t] = tmp[t];
    return inv;
}
```

13. Search a 2D Matrix

Brute force

Scan every cell until target is found or the matrix is exhausted. Ignores sorted structure.

Complexity: O(n * m) time, O(1) space.

```
boolean searchMatrixBrute(int[][] a, int target) {
    for (int[] row : a)
        for (int v : row)
            if (v == target) return true;
    return false;
}
```

Better

Start at the top-right corner. If the cell is too large, move left; if too small, move down. Each step eliminates a row or column.

Complexity: O(n + m) time, O(1) space.

```
boolean searchMatrixStair(int[][] a, int target) {
    int r = 0, c = a[0].length - 1;
    while (r < a.length && c >= 0) {
        if (a[r][c] == target) return true;
        if (a[r][c] > target) c--;
        else r++;
    }
    return false;
}
```

Optimal

Binary search on the matrix viewed as one sorted array. Each row is sorted, and the first value of a row is greater than the last value of the previous row, so the whole matrix is one sorted sequence of length n * m. Index mid maps to row mid / m and column mid % m. Ordinary binary search then applies.

Complexity: O(log(nm)) time, O(1) space.

```
boolean searchMatrix(int[][] a, int target) {
    int n = a.length, m = a[0].length;
    int lo = 0, hi = n * m - 1;
    while (lo <= hi) {
        int mid = (lo + hi) >>> 1;
        int v = a[mid / m][mid % m];
        if (v == target) return true;
        if (v < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return false;
}
```

14. Pow(x, n)

Brute force

Multiply x into an accumulator |n| times (handle negative n by inverting once). Linear in the exponent.

Complexity: O(|n|) time, O(1) space.

```
double myPowBrute(double x, int n) {
    long e = n;
    if (e < 0) { x = 1 / x; e = -e; }
    double ans = 1;
    for (long i = 0; i < e; i++) ans *= x;
    return ans;
}
```

Optimal

Binary exponentiation. x^n = (x^2)^(n/2) when n is even, and x * (x^2)^(n/2) when n is odd. That halves the exponent each step. Store the exponent in a long before negating it, because n can be Integer.MIN_VALUE. A negative exponent means invert the base first and then use the positive power.

Complexity: O(log |n|) time, O(1) space.

```
double myPow(double x, int n) {
    long e = n;
    if (e < 0) { x = 1 / x; e = -e; }
    double ans = 1;
    while (e > 0) {
        if ((e & 1) == 1) ans *= x;
        x *= x;
        e >>= 1;
    }
    return ans;
}
```

15. Majority Element I

Brute force

For each distinct value (or each index), count occurrences in a second loop and return the first value with count > n/2.

Complexity: O(n^2) time, O(1) extra space.

```
int majorityElementBrute(int[] a) {
    int n = a.length;
    for (int i = 0; i < n; i++) {
        int cnt = 0;
        for (int j = 0; j < n; j++)
            if (a[j] == a[i]) cnt++;
        if (cnt > n / 2) return a[i];
    }
    return a[0];
}
```

Better

Sort the array and return the middle element. A majority must occupy the middle index after sorting.

Complexity: O(n log n) time, O(1) extra space if sorting in place.

```
int majorityElementSort(int[] a) {
    Arrays.sort(a);
    return a[a.length / 2];
}
```

Optimal

Boyer-Moore vote: one candidate and a counter. The majority element appears more than n/2 times, so it survives pairing. Walk the array with a candidate and a count. Matching the candidate increments the count; anything else decrements it. At zero, the previous candidate has been cancelled and the current value becomes the new candidate. Because a true majority cannot be fully cancelled, it is the candidate at the end. The problem promises a majority, so a second verification pass is unnecessary.

Complexity: O(n) time, O(1) space.

```
int majorityElement(int[] a) {
    int cand = a[0], count = 0;
    for (int x : a) {
        if (count == 0) cand = x;
        count += (x == cand) ? 1 : -1;
    }
    return cand;
}
```

16. Majority Element II

Brute force

Count frequency of every value with a hash map and collect keys whose count exceeds n/3. Straightforward and linear time with extra space.

Complexity: O(n) time, O(n) space.

```
List<Integer> majorityElementBrute(int[] a) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int x : a) cnt.put(x, cnt.getOrDefault(x, 0) + 1);
    List<Integer> ans = new ArrayList<>();
    for (var e : cnt.entrySet())
        if (e.getValue() > a.length / 3) ans.add(e.getKey());
    return ans;
}
```

Optimal

Boyer-Moore with two candidates. At most two values can appear more than n/3 times. Keep two candidates and two counters. A match increments that candidate. If a counter is zero, adopt the current value. Otherwise both counters fall by one, which cancels one copy of each candidate against a third value. The survivors are only candidates. A second pass counts them and keeps those whose count is greater than n/3.

Complexity: O(n) time, O(1) space.

```
List<Integer> majorityElement(int[] a) {
    int n1 = 0, n2 = 0, c1 = 0, c2 = 0;
    for (int x : a) {
        if (x == n1) c1++;
        else if (x == n2) c2++;
        else if (c1 == 0) { n1 = x; c1 = 1; }
        else if (c2 == 0) { n2 = x; c2 = 1; }
        else { c1--; c2--; }
    }
    c1 = 0; c2 = 0;
    for (int x : a) {
        if (x == n1) c1++;
        else if (x == n2) c2++;
    }
    List<Integer> ans = new ArrayList<>();
    if (c1 > a.length / 3) ans.add(n1);
    if (n2 != n1 && c2 > a.length / 3) ans.add(n2);
    return ans;
}
```

17. Maximum Product Subarray

Brute force

For each subarray, multiply all elements and track the maximum product. Handles zeros and negatives correctly but slowly.

Complexity: O(n^2) time, O(1) extra space.

```
int maxProductBrute(int[] a) {
    int best = Integer.MIN_VALUE;
    for (int i = 0; i < a.length; i++) {
        int p = 1;
        for (int j = i; j < a.length; j++) {
            p *= a[j];
            best = Math.max(best, p);
        }
    }
    return best;
}
```

Optimal

Kadane with both the best and worst product ending here. A negative number flips a large product into a small one and a small product into a large one. So the best product ending at i is the max of a[i], a[i] times the previous max, and a[i] times the previous min. Keep both ends. A zero splits the array into independent pieces, which falls out naturally because multiplying by zero resets both ends to zero and the next element starts fresh.

Complexity: O(n) time, O(1) space.

```
int maxProduct(int[] a) {
    int best = a[0], mx = a[0], mn = a[0];
    for (int i = 1; i < a.length; i++) {
        int x = a[i];
        int hi = Math.max(x, Math.max(mx * x, mn * x));
        int lo = Math.min(x, Math.min(mx * x, mn * x));
        mx = hi; mn = lo;
        best = Math.max(best, mx);
    }
    return best;
}
```

18. Reverse Pairs

Brute force

Check every pair i < j and increment when a[i] > 2L * a[j]. Direct definition.

Complexity: O(n^2) time, O(1) extra space.

```
int reversePairsBrute(int[] a) {
    int cnt = 0;
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] > 2L * a[j]) cnt++;
    return cnt;
}
```

Optimal

Count a[i] > 2 * a[j] while merging. Same split as inversion count, but the condition is stricter than a plain greater-than. Before merging a sorted left half and right half, walk i on the left and j on the right. Because the right half is sorted, once a[i] > 2 * a[j], every earlier right element also qualifies, and j only moves forward. Count those pairs, then merge normally. The factor 2 uses a long so it does not overflow.

Complexity: O(n log n) time, O(n) space.

```
int reversePairs(int[] a) {
    return (int) sort(a, 0, a.length - 1, new int[a.length]);
}
long sort(int[] a, int l, int r, int[] tmp) {
    if (l >= r) return 0;
    int m = (l + r) >>> 1;
    long cnt = sort(a, l, m, tmp) + sort(a, m + 1, r, tmp);
    int j = m + 1;
    for (int i = l; i <= m; i++) {
        while (j <= r && a[i] > 2L * a[j]) j++;
        cnt += j - (m + 1);
    }
    int i = l, p = m + 1, k = l;
    while (i <= m && p <= r)
        tmp[k++] = a[i] <= a[p] ? a[i++] : a[p++];
    while (i <= m) tmp[k++] = a[i++];
    while (p <= r) tmp[k++] = a[p++];
    for (int t = l; t <= r; t++) a[t] = tmp[t];
    return cnt;
}
```

Hashing

A hash map stores a fact about the prefix you have already seen, so the current index can answer in constant time.

19. Two Sum

Brute force

Try every ordered pair (i, j) with i < j. If a[i] + a[j] equals the target, return those indices. Correct but quadratic.

Complexity: O(n^2) time, O(1) extra space.

```
int[] twoSumBrute(int[] a, int target) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] + a[j] == target) return new int[]{i, j};
    return new int[0];
}
```

Optimal

Map value to index; look for target - x. For each value x, the partner that completes the target is target - x. If that partner was stored earlier, the pair exists and the indices are different. Store x only after the lookup so a value is not paired with itself. One pass is enough because every earlier index is already in the map.

Complexity: O(n) time, O(n) space.

```
int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        int need = target - a[i];
        if (seen.containsKey(need)) return new int[]{seen.get(need), i};
        seen.put(a[i], i);
    }
    return new int[0];
}
```

20. 4Sum

Brute force

Four nested loops over indices i < j < l < r. When the four values sum to target, record the quadruplet. Skip duplicate logic is optional; the method is dominated by O(n^4) checks.

Complexity: O(n^4) time, O(1) extra space besides the answer.

```
List<List<Integer>> fourSumBrute(int[] a, int target) {
    List<List<Integer>> ans = new ArrayList<>();
    int n = a.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            for (int l = j + 1; l < n; l++)
                for (int r = l + 1; r < n; r++) {
                    long s = (long) a[i] + a[j] + a[l] + a[r];
                    if (s == target)
                        ans.add(Arrays.asList(a[i], a[j], a[l], a[r]));
                }
    return ans;
}
```

Optimal

Sort, fix two numbers, then two pointers. Sorting lets equal values sit together so duplicates can be skipped, and it lets the remaining two numbers move inward. Fix i and j. If the four-sum is too small, move the left pointer; if too large, move the right pointer. On a hit, record the quadruplet and skip identical neighbors of both pointers. Skip identical i and j as well, otherwise the same quadruplet is emitted many times. Use a long sum because four ints can overflow.

Complexity: O(n^3) time, O(1) extra space besides the answer.

```
List<List<Integer>> fourSum(int[] a, int target) {
    Arrays.sort(a);
```

```
    List<List<Integer>> ans = new ArrayList<>();
    int n = a.length;
    for (int i = 0; i < n; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        for (int j = i + 1; j < n; j++) {
            if (j > i + 1 && a[j] == a[j - 1]) continue;
            int l = j + 1, r = n - 1;
            while (l < r) {
                long s = (long) a[i] + a[j] + a[l] + a[r];
                if (s == target) {
                    ans.add(Arrays.asList(a[i], a[j], a[l], a[r]));
                    l++; r--;
                    while (l < r && a[l] == a[l - 1]) l++;
                    while (l < r && a[r] == a[r + 1]) r--;
                } else if (s < target) l++;
                else r--;
            }
        }
    }
    return ans;
}
```

21. Longest Consecutive Sequence

Brute force

Sort the array and scan runs of equal neighbors, extending each distinct value forward one step at a time while the next integer exists in the sorted list. Sorting makes existence checks linear in run length.

Complexity: O(n log n) time, O(1) extra space if sorting in place.

```
int longestConsecutiveBrute(int[] a) {
    if (a.length == 0) return 0;
    Arrays.sort(a);
    int best = 1, cur = 1;
    for (int i = 1; i < a.length; i++) {
        if (a[i] == a[i - 1]) continue;
        if (a[i] == a[i - 1] + 1) cur++;
        else { best = Math.max(best, cur); cur = 1; }
    }
    return Math.max(best, cur);
}
```

Optimal

A number starts a run only if num - 1 is absent. Put every value in a set. A consecutive run has exactly one left endpoint: the value whose predecessor is missing. From each such start, walk num, num + 1, num + 2 while they exist and keep the longest length. Each number is visited once as part of a walk, so the total work is linear even though the inner loop looks nested.

Complexity: O(n) time, O(n) space.

```
int longestConsecutive(int[] a) {
    Set<Integer> set = new HashSet<>();
    for (int x : a) set.add(x);
    int best = 0;
    for (int x : set) {
        if (set.contains(x - 1)) continue;
        int len = 1;
        while (set.contains(x + len)) len++;
        best = Math.max(best, len);
    }
    return best;
}
```

22. Largest Subarray with Sum 0

Brute force

For every start index i, extend j and track the running sum. When the sum is zero, update the best length j - i + 1. Every subarray is considered.

Complexity: O(n^2) time, O(1) extra space.

```
int maxLenBrute(int[] a) {
    int best = 0;
    for (int i = 0; i < a.length; i++) {
        int sum = 0;
        for (int j = i; j < a.length; j++) {
            sum += a[j];
            if (sum == 0) best = Math.max(best, j - i + 1);
        }
    }
    return best;
}
```

Optimal

Equal prefix sums bound a zero-sum segment. If prefix[j] == prefix[i], the sum from i + 1 through j is 0. The first time a prefix sum appears is the leftmost start of such a segment, so store that index and, on a repeat, take j - firstIndex. A prefix of 0 itself means the array from the start sums to 0; store sum 0 at index -1 so that case falls out of the same formula.

Complexity: O(n) time, O(n) space.

```
int maxLen(int[] a) {
    Map<Integer, Integer> first = new HashMap<>();
    first.put(0, -1);
    int sum = 0, best = 0;
    for (int i = 0; i < a.length; i++) {
        sum += a[i];
        if (first.containsKey(sum)) best = Math.max(best, i - first.get(sum));
        else first.put(sum, i);
    }
    return best;
}
```

23. Count Subarrays with XOR K

Brute force

For each start i, walk j and XOR-accumulate. Whenever the subarray xor equals k, increment the answer. Correct but quadratic.

Complexity: O(n^2) time, O(1) extra space.

```
int subarraysWithXorKBrute(int[] a, int k) {
    int ans = 0;
    for (int i = 0; i < a.length; i++) {
        int x = 0;
        for (int j = i; j < a.length; j++) {
            x ^= a[j];
            if (x == k) ans++;
        }
    }
    return ans;
}
```

Optimal

prefixXor[j] ^ prefixXor[i] = k. XOR from i + 1 to j equals prefix[j] ^ prefix[i]. That equals k exactly when prefix[i] = prefix[j] ^ k. Count how many earlier prefixes have that value. Include the empty prefix 0 so subarrays that start at index 0 are counted. Each index adds the number of valid starts ending there.

Complexity: O(n) time, O(n) space.

```
int subarraysWithXorK(int[] a, int k) {
```

```
    Map<Integer, Integer> freq = new HashMap<>();
    freq.put(0, 1);
    int xor = 0, ans = 0;
    for (int x : a) {
        xor ^= x;
        ans += freq.getOrDefault(xor ^ k, 0);
        freq.put(xor, freq.getOrDefault(xor, 0) + 1);
    }
    return ans;
}
```

24. Subarray Sum Equals K

Brute force

Enumerate all subarrays by nested loops, maintain the sum, and count how many equal k. No auxiliary structures.

Complexity: O(n^2) time, O(1) extra space.

```
int subarraySumBrute(int[] a, int k) {
    int ans = 0;
    for (int i = 0; i < a.length; i++) {
        int sum = 0;
        for (int j = i; j < a.length; j++) {
            sum += a[j];
            if (sum == k) ans++;
        }
    }
    return ans;
}
```

Optimal

prefix[j] - prefix[i] = k. The same prefix idea as the zero-sum problem, but now the needed earlier prefix is currentSum - k, and we count occurrences instead of storing only the first index. The empty prefix 0 with frequency 1 covers subarrays that start at the beginning.

Complexity: O(n) time, O(n) space.

```
int subarraySum(int[] a, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    freq.put(0, 1);
    int sum = 0, ans = 0;
    for (int x : a) {
        sum += x;
        ans += freq.getOrDefault(sum - k, 0);
        freq.put(sum, freq.getOrDefault(sum, 0) + 1);
    }
    return ans;
}
```

25. Longest Subarray with Sum K

Brute force

Try every subarray, track its sum, and keep the longest length with sum k. Works for negative numbers but scans all O(n^2) segments.

Complexity: O(n^2) time, O(1) extra space.

```
int longestSubarraySumKBrute(int[] a, int k) {
    int best = 0;
    for (int i = 0; i < a.length; i++) {
        int sum = 0;
        for (int j = i; j < a.length; j++) {
            sum += a[j];
            if (sum == k) best = Math.max(best, j - i + 1);
        }
    }
    return best;
}
```

Better

When every element is non-negative, a sliding window works: expand the right end while sum < k, shrink from the left while sum > k, and record length when sum equals k. Linear time but wrong if negatives appear.

Complexity: O(n) time, O(1) extra space (non-negative arrays only).

```
int longestSubarraySumKWindow(int[] a, int k) {
    int l = 0, sum = 0, best = 0;
    for (int r = 0; r < a.length; r++) {
        sum += a[r];
        while (sum > k && l <= r) sum -= a[l++];
        if (sum == k) best = Math.max(best, r - l + 1);
    }
    return best;
}
```

Optimal

First index of prefix sum - k, positives allow two pointers instead. For an array that may contain negatives, hashing is required: the longest subarray ending at i with sum k starts just after the earliest index whose prefix is sum - k. If the statement guarantees non-negative numbers, a sliding window is enough and uses constant space, because growing the right end only increases the sum. This function is the general version.

Complexity: O(n) time, O(n) space.

```
int longestSubarraySumK(int[] a, int k) {
    Map<Integer, Integer> first = new HashMap<>();
    first.put(0, -1);
    int sum = 0, best = 0;
    for (int i = 0; i < a.length; i++) {
        sum += a[i];
        if (first.containsKey(sum - k))
            best = Math.max(best, i - first.get(sum - k));
        first.putIfAbsent(sum, i);
    }
    return best;
}
```

26. Count Distinct Elements in Every Window of Size K

Brute force

For each window starting at i, insert its k elements into a set (or mark a frequency array if values are bounded) and report the set size. Each window is rebuilt from scratch.

Complexity: O(n * k) time, O(k) space per window.

```
List<Integer> distinctInWindowsBrute(int[] a, int k) {
    List<Integer> ans = new ArrayList<>();
    for (int i = 0; i + k <= a.length; i++) {
        Set<Integer> seen = new HashSet<>();
        for (int j = i; j < i + k; j++) seen.add(a[j]);
        ans.add(seen.size());
    }
    return ans;
}
```

Optimal

Frequency map of the current window. Build the frequency map of the first k elements. Its size is the number of distinct values. Slide one step: decrement the outgoing element and remove it when its count hits zero, then increment the incoming element. The map size after each slide is the answer for that window. Each element enters and leaves once.

Complexity: O(n) time, O(k) space.

```
List<Integer> distinctInWindows(int[] a, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    List<Integer> ans = new ArrayList<>();
    for (int i = 0; i < a.length; i++) {
```

```
        freq.put(a[i], freq.getOrDefault(a[i], 0) + 1);
        if (i >= k) {
            int out = a[i - k];
            freq.put(out, freq.get(out) - 1);
            if (freq.get(out) == 0) freq.remove(out);
        }
        if (i >= k - 1) ans.add(freq.size());
    }
    return ans;
}
```

Binary Search

The search is not always on an index. Sometimes the sorted thing is the answer itself: any smaller value fails and any larger value works, or the reverse.

27. Lower Bound and Upper Bound

Brute force

Scan from left to right. Lower bound is the first index with a[i] >= target, or n if none. Upper bound is the first index with a[i] > target. Linear and always correct on sorted input.

Complexity: O(n) time, O(1) space.

```
int lowerBoundBrute(int[] a, int target) {
    for (int i = 0; i < a.length; i++)
        if (a[i] >= target) return i;
    return a.length;
}
int upperBoundBrute(int[] a, int target) {
    for (int i = 0; i < a.length; i++)
        if (a[i] > target) return i;
    return a.length;
}
```

Optimal

First index where the predicate becomes true. Lower bound is the first index with a[i] >= target. Upper bound is the first index with a[i] > target. Keep the invariant that lo is the first possible answer and hi is just past the end. If mid already satisfies the predicate, it might be the answer, so pull hi down to mid. Otherwise the answer is strictly to the right. The loop ends with lo == hi, which is the insertion point, or n if every element fails.

Complexity: O(log n) time, O(1) space.

```
int lowerBound(int[] a, int target) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] >= target) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
int upperBound(int[] a, int target) {
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] > target) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

28. First and Last Position of an Element

Brute force

Walk the sorted array once. Remember the first index where a[i] equals target, keep updating the last index while values still match, and return [-1, -1] if never seen.

Complexity: O(n) time, O(1) space.

```
int[] searchRangeBrute(int[] a, int target) {
    int first = -1, last = -1;
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) {
            if (first == -1) first = i;
            last = i;
        }
    }
```

```
    return new int[]{first, last};
}
```

Optimal

Lower bound and upper bound of the same target. The first occurrence is the lower bound. The last occurrence is the index just before the upper bound. If the lower bound is out of range or does not hold the target, the element is absent. Two binary searches replace a linear scan of the equal run.

Complexity: O(log n) time, O(1) space.

```
int[] searchRange(int[] a, int target) {
    int l = lowerBound(a, target);
    if (l == a.length || a[l] != target) return new int[]{-1, -1};
    return new int[]{l, upperBound(a, target) - 1};
}
```

29. Search in Rotated Sorted Array

Brute force

Find the pivot where the rotation happened with a linear scan, then run ordinary binary search on the half that contains the target. Two O(log n) searches after an O(n) pivot find, or simply scan all indices.

Complexity: O(n) time for pivot scan, O(1) extra space.

```
int searchBrute(int[] a, int target) {
    for (int i = 0; i < a.length; i++)
        if (a[i] == target) return i;
    return -1;
}
```

Better

Locate the minimum index (pivot) with binary search on the rotated array, then binary search separately in the left sorted segment and the right sorted segment. Two binary searches after one pivot search.

Complexity: O(log n) time, O(1) space.

```
int searchPivotBs(int[] a, int target) {
    int n = a.length, lo = 0, hi = n - 1;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] > a[hi]) lo = mid + 1;
        else hi = mid;
    }
    int pivot = lo;
    if (target >= a[pivot] && target <= a[n - 1])
        return bs(a, pivot, n - 1, target);
    return bs(a, 0, pivot - 1, target);
}
int bs(int[] a, int lo, int hi, int target) {
    while (lo <= hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Optimal

One half of the window is still sorted. Rotation keeps one of the two halves sorted. Compare a[mid] with a[lo]. If the left half is sorted and the target lies inside it, discard the right; otherwise discard the left. The mirror test handles a sorted right half. Duplicates break the comparison a[mid] == a[lo], so this function assumes distinct values, which is the usual statement.

Complexity: O(log n) time, O(1) space.

```
int search(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
```

```
    while (lo <= hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) {
            if (a[lo] <= target && target < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {
            if (a[mid] < target && target <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

30. Single Element in a Sorted Array

Brute force

XOR every element. Values that appear twice cancel; the lone value remains. Linear time and simple, but does not exploit the sorted structure.

Complexity: O(n) time, O(1) space.

```
int singleNonDuplicateXor(int[] a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x;
}
```

Optimal

Pairs occupy even-odd indexes until the single breaks them. Before the single element, values sit in pairs at (0, 1), (2, 3), and so on. After it, pairs sit at odd-even indexes. If mid is even and a[mid] == a[mid + 1], the single is still to the right. Otherwise it is at mid or to the left. Force mid onto an even index so the partner test is always the next cell.

Complexity: O(log n) time, O(1) space.

```
int singleNonDuplicate(int[] a) {
    int lo = 0, hi = a.length - 1;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if ((mid & 1) == 1) mid--;
        if (a[mid] == a[mid + 1]) lo = mid + 2;
        else hi = mid;
    }
    return a[lo];
}
```

31. Find Peak Element

Brute force

Check each index i (with neighbors treated as -infinity at the ends). Return the first i where a[i] is at least both neighbors. A peak is guaranteed to exist.

Complexity: O(n) time, O(1) space.

```
int findPeakBrute(int[] a) {
    for (int i = 0; i < a.length; i++) {
        int left = i == 0 ? Integer.MIN_VALUE : a[i - 1];
        int right = i == a.length - 1 ? Integer.MIN_VALUE : a[i + 1];
        if (a[i] >= left && a[i] >= right) return i;
    }
    return 0;
}
```

Optimal

Walk uphill; a slope decides which side still has a peak. A peak is not smaller than its neighbors. If a[mid] < a[mid + 1], the right side is strictly uphill at mid, so some peak exists at mid + 1 or further right (the array edge counts as negative infinity). Otherwise a peak exists at mid or to the left. The loop shrinks until one index remains.

Complexity: O(log n) time, O(1) space.

```
int findPeakElement(int[] a) {
    int lo = 0, hi = a.length - 1;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] < a[mid + 1]) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

32. Nth Root of an Integer

Brute force

Try every integer x from 1 through m. The largest x with x^n <= m is the answer. Each trial multiplies n times with early exit on overflow.

Complexity: O(m * n) time, O(1) space.

```
int nthRootBrute(int n, int m) {
    int ans = -1;
    for (int x = 1; x <= m; x++) {
        if (powCmp(x, n, m) <= 0) ans = x;
        else break;
    }
    return ans;
}
```

Optimal

Binary search on the answer, multiply in a long. The nth root of m is the largest integer x with x^n <= m. The search space is 1..m. Computing x^n by a loop that aborts when the product exceeds m avoids overflow and avoids floating point. If mid^n is too big, the root is smaller; if it is too small, try a larger mid and remember it.

Complexity: O(n log m) time, O(1) space.

```
int nthRoot(int n, int m) {
    int lo = 1, hi = m, ans = -1;
    while (lo <= hi) {
        int mid = (lo + hi) >>> 1;
        int cmp = powCmp(mid, n, m);
        if (cmp == 0) return mid;
        if (cmp < 0) { ans = mid; lo = mid + 1; }
        else hi = mid - 1;
    }
    return ans;
}
int powCmp(int base, int exp, int limit) {
    long p = 1;
    for (int i = 0; i < exp; i++) {
        p *= base;
        if (p > limit) return 1;
    }
    return p == limit ? 0 : -1;
}
```

33. Median of Two Sorted Arrays

Brute force

Merge the two sorted arrays into one buffer of length n + m (standard two-pointer merge), then read the middle element(s). Correct but linear time and extra space.

Complexity: O(n + m) time, O(n + m) space.

```
double findMedianBrute(int[] a, int[] b) {
    int n = a.length, m = b.length;
    int[] c = new int[n + m];
    int i = 0, j = 0, k = 0;
    while (i < n && j < m)
        c[k++] = a[i] <= b[j] ? a[i++] : b[j++];
    while (i < n) c[k++] = a[i++];
    while (j < m) c[k++] = b[j++];
    int mid = (n + m) / 2;
    if (((n + m) & 1) == 1) return c[mid];
    return (c[mid - 1] + c[mid]) / 2.0;
}
```

Optimal

Partition both arrays so the left half has the right count. The median is the middle of the merged array. Choose a cut i in the first array; the matching cut in the second is half - i, where half is the number of elements that belong on the left. The cut is valid when the largest on the left of each array is no bigger than the smallest on the right of the other. If a[i - 1] is too big, move the cut left; otherwise move it right. Odd and even totals differ only in whether the median is one value or the average of the two middle values.

Complexity: O(log min(n, m)) time, O(1) space.

```
double findMedianSortedArrays(int[] a, int[] b) {
    if (a.length > b.length) return findMedianSortedArrays(b, a);
    int n = a.length, m = b.length, half = (n + m + 1) / 2;
    int lo = 0, hi = n;
    while (lo <= hi) {
        int i = (lo + hi) >>> 1;
        int j = half - i;
        int aL = i == 0 ? Integer.MIN_VALUE : a[i - 1];
        int aR = i == n ? Integer.MAX_VALUE : a[i];
        int bL = j == 0 ? Integer.MIN_VALUE : b[j - 1];
        int bR = j == m ? Integer.MAX_VALUE : b[j];
        if (aL <= bR && bL <= aR) {
            if (((n + m) & 1) == 1) return Math.max(aL, bL);
            return (Math.max(aL, bL) + Math.min(aR, bR)) / 2.0;
        } else if (aL > bR) hi = i - 1;
        else lo = i + 1;
    }
    return 0;
}
```

34. Kth Element of Two Sorted Arrays

Brute force

Merge with two pointers until k elements have been written, then return the kth merged value. Same merge as the median brute force but stops early at k.

Complexity: O(n + m) time, O(1) extra space.

```
int kthElementBrute(int[] a, int[] b, int k) {
    int i = 0, j = 0, cnt = 0;
    while (i < a.length && j < b.length) {
        cnt++;
        if (a[i] <= b[j]) {
            if (cnt == k) return a[i];
            i++;
        } else {
            if (cnt == k) return b[j];
            j++;
        }
```

```
    }
    while (i < a.length) {
        cnt++;
        if (cnt == k) return a[i];
        i++;
    }
    while (j < b.length) {
        cnt++;
        if (cnt == k) return b[j];
        j++;
    }
    return -1;
}
```

Optimal

The same partition, with the left side sized to k. Instead of splitting at the median, require exactly k elements on the left of the cut. The largest of the two left edges is the kth element of the merged array. The feasible range for the cut in the shorter array is max(0, k - m) through min(k, n), because the other array must be able to supply the rest.

Complexity: O(log min(n, m)) time, O(1) space.

```
int kthElement(int[] a, int[] b, int k) {
    if (a.length > b.length) return kthElement(b, a, k);
    int n = a.length, m = b.length;
    int lo = Math.max(0, k - m), hi = Math.min(k, n);
    while (lo <= hi) {
        int i = (lo + hi) >>> 1, j = k - i;
        int aL = i == 0 ? Integer.MIN_VALUE : a[i - 1];
        int aR = i == n ? Integer.MAX_VALUE : a[i];
        int bL = j == 0 ? Integer.MIN_VALUE : b[j - 1];
        int bR = j == m ? Integer.MAX_VALUE : b[j];
        if (aL <= bR && bL <= aR) return Math.max(aL, bL);
        if (aL > bR) hi = i - 1;
        else lo = i + 1;
    }
    return -1;
}
```

35. Koko Eating Bananas

Brute force

Try every integer speed from 1 up to the largest pile. For each speed, sum ceil(pile / speed) across piles and pick the smallest speed whose total hours is at most h.

Complexity: O(n * M) time, O(1) space, M = max pile.

```
int minEatingSpeedBrute(int[] piles, int h) {
    int hi = 0;
    for (int p : piles) hi = Math.max(hi, p);
    for (int s = 1; s <= hi; s++)
        if (hours(piles, s) <= h) return s;
    return hi;
}
```

Optimal

Search on the answer: minimum speed that finishes in h hours. A slower speed always takes at least as many hours as a faster speed, so feasibility is monotonic. The speed lies between 1 and the largest pile. For a candidate speed, the hours needed are the sum of ceil(pile / speed). If that fits in h, try a slower speed; otherwise the speed is too small. The leftmost feasible speed is the answer.

Complexity: O(n log M) time, O(1) space, M = max pile.

```
int minEatingSpeed(int[] piles, int h) {
    int lo = 1, hi = 0;
    for (int p : piles) hi = Math.max(hi, p);
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
```

```
        if (hours(piles, mid) <= h) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
long hours(int[] piles, int speed) {
    long t = 0;
    for (int p : piles) t += (p + speed - 1L) / speed;
    return t;
}
```

36. Aggressive Cows

Brute force

Sort stalls, then try every candidate distance d from 0 up to the span. The largest d for which greedy placement of cows succeeds is the answer. Each check walks the stalls once.

Complexity: O(n log n + n * D) time, O(1) extra space, D = span.

```
int aggressiveCowsBrute(int[] stalls, int cows) {
    Arrays.sort(stalls);
    int best = 0;
    for (int d = stalls[stalls.length - 1] - stalls[0]; d >= 0; d--)
        if (canPlace(stalls, cows, d)) return d;
    return best;
}
```

Optimal

Search on the minimum distance between cows. Sort the stalls. If some minimum distance d is possible, every smaller distance is also possible, so binary search d. A greedy check places the next cow in the first stall that is at least d away from the previous cow. If k cows fit, try a larger d; otherwise shrink. The search range is 0 through the distance between the first and last stall.

Complexity: O(n log n + n log D) time, O(1) extra space.

```
int aggressiveCows(int[] stalls, int cows) {
    Arrays.sort(stalls);
    int lo = 0, hi = stalls[stalls.length - 1] - stalls[0];
    while (lo < hi) {
        int mid = (lo + hi + 1) >>> 1;
        if (canPlace(stalls, cows, mid)) lo = mid;
        else hi = mid - 1;
    }
    return lo;
}
boolean canPlace(int[] s, int cows, int d) {
    int used = 1, last = s[0];
    for (int i = 1; i < s.length && used < cows; i++) {
        if (s[i] - last >= d) { used++; last = s[i]; }
    }
    return used == cows;
}
```

37. Allocate Minimum Number of Pages

Brute force

The answer lies between the thickest book and the sum of all pages. Try every cap in that range; the smallest cap that can be split into at most the allowed students is optimal.

Complexity: O(n * S) time, O(1) space, S = sum of pages.

```
int allocateBrute(int[] pages, int students) {
    if (students > pages.length) return -1;
    int lo = 0, hi = 0;
    for (int p : pages) { lo = Math.max(lo, p); hi += p; }
    for (int cap = lo; cap <= hi; cap++)
        if (need(pages, cap) <= students) return cap;
```

```
    return hi;
}
```

Optimal

Search on the maximum pages given to any student. Books are in order and each student gets a contiguous segment. If a maximum load cap is feasible, every larger cap is feasible. The cap is at least the thickest book and at most the sum of all books. The check walks left to right and starts a new student whenever adding the next book would exceed the cap. If the number of students needed is within the limit, try a smaller cap.

Complexity: O(n log S) time, O(1) space, S = sum of pages.

```
int allocate(int[] pages, int students) {
    if (students > pages.length) return -1;
    int lo = 0, hi = 0;
    for (int p : pages) { lo = Math.max(lo, p); hi += p; }
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (need(pages, mid) <= students) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
int need(int[] pages, int cap) {
    int students = 1, load = 0;
    for (int p : pages) {
        if (load + p > cap) { students++; load = 0; }
        load += p;
    }
    return students;
}
```

38. Median in a Row-Wise Sorted Matrix

Brute force

Copy every matrix entry into a list, sort, and return the element at rank (n*m+1)/2. Simple but ignores row-wise order.

Complexity: O(n * m * log(n * m)) time, O(n * m) space.

```
int matrixMedianBrute(int[][] a) {
    List<Integer> all = new ArrayList<>();
    for (int[] row : a)
        for (int v : row) all.add(v);
    Collections.sort(all);
    return all.get((all.size() - 1) / 2);
}
```

Optimal

Binary search the value, count how many entries are <= mid. The matrix median is the smallest value x such that at least (n * m + 1) / 2 entries are <= x. Values lie between the smallest first-column entry and the largest last-column entry. For a candidate, each row is sorted, so the count in that row is an upper bound. If the count is still short of the required rank, the median is larger.

Complexity: O(n log m * log V) time, O(1) space.

```
int matrixMedian(int[][] a) {
    int n = a.length, m = a[0].length;
    int lo = a[0][0], hi = a[0][m - 1];
    for (int i = 1; i < n; i++) {
        lo = Math.min(lo, a[i][0]);
        hi = Math.max(hi, a[i][m - 1]);
    }
    int need = (n * m + 1) / 2;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (countLeq(a, mid) >= need) hi = mid;
        else lo = mid + 1;
```

```
    }
    return lo;
}
int countLeq(int[][] a, int x) {
    int cnt = 0;
    for (int[] row : a) {
        int lo = 0, hi = row.length;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (row[mid] <= x) lo = mid + 1;
            else hi = mid;
        }
        cnt += lo;
    }
    return cnt;
}
```

39. Capacity To Ship Packages Within D Days

Brute force

Capacity must be at least the heaviest package and at most the total weight. Scan every capacity in that range and use the greedy day-packing check; return the first feasible capacity.

Complexity: O(n * S) time, O(1) space, S = total weight.

```
int shipWithinDaysBrute(int[] weights, int days) {
    int lo = 0, hi = 0;
    for (int w : weights) { lo = Math.max(lo, w); hi += w; }
    for (int cap = lo; cap <= hi; cap++) {
        int need = 1, load = 0;
        for (int w : weights) {
            if (load + w > cap) { need++; load = 0; }
            load += w;
        }
        if (need <= days) return cap;
    }
    return hi;
}
```

Optimal

Same search-on-answer shape as book allocation. Packages stay in order. A capacity is feasible when a left-to-right packing uses at most d days. Capacity is at least the heaviest package and at most the total weight. This is book allocation with days in place of students and weights in place of pages.

Complexity: O(n log S) time, O(1) space.

```
int shipWithinDays(int[] weights, int days) {
    int lo = 0, hi = 0;
    for (int w : weights) { lo = Math.max(lo, w); hi += w; }
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        int need = 1, load = 0;
        for (int w : weights) {
            if (load + w > mid) { need++; load = 0; }
            load += w;
        }
        if (need <= days) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

40. Find the Smallest Divisor Given a Threshold

Brute force

Divisors only need to be tried from 1 through the maximum array value. Pick the smallest d whose sum of ceil(a[i]/d) is at most the threshold.

Complexity: O(n * M) time, O(1) space, M = max element.

```
int smallestDivisorBrute(int[] a, int threshold) {
    int hi = 0;
    for (int x : a) hi = Math.max(hi, x);
    for (int d = 1; d <= hi; d++) {
        long sum = 0;
        for (int x : a) sum += (x + d - 1L) / d;
        if (sum <= threshold) return d;
    }
    return hi;
}
```

Optimal

Minimum divisor whose ceiling-sum is within the threshold. ceil(a[i] / d) shrinks as d grows, so the sum is monotonic. d ranges from 1 to the maximum element. The check is the same ceiling sum used for Koko. The smallest d whose sum is <= threshold is the answer.

Complexity: O(n log M) time, O(1) space.

```
int smallestDivisor(int[] a, int threshold) {
    int lo = 1, hi = 0;
    for (int x : a) hi = Math.max(hi, x);
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        long sum = 0;
        for (int x : a) sum += (x + mid - 1L) / mid;
        if (sum <= threshold) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

Sliding Window and Two Pointers

Two indexes move only forward. A window is valid when a count or a sum stays inside a bound, and the answer is the best valid window.

41. 3Sum

Brute force

Try every triple (i, j, k) with i < j < k. When the sum is zero, sort the triplet and add it to a set so duplicates collapse. Correct but cubic.

Complexity: O(n^3) time, O(n) space for deduplication.

```
List<List<Integer>> threeSumBrute(int[] a) {
    Set<List<Integer>> set = new HashSet<>();
    int n = a.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            for (int k = j + 1; k < n; k++)
                if (a[i] + a[j] + a[k] == 0)
                    set.add(Arrays.asList(a[i], a[j], a[k]));
    return new ArrayList<>(set);
}
```

Optimal

Sort, fix a[i], and two-pointer the pair in the suffix for sum -a[i]. Skip equal i and, after a hit, skip equal left and right values so each triplet is recorded once.

Complexity: O(n^2) time, O(1) extra space besides the answer.

```
List<List<Integer>> threeSum(int[] a) {
    Arrays.sort(a);
    List<List<Integer>> ans = new ArrayList<>();
    for (int i = 0; i < a.length; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        int l = i + 1, r = a.length - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s == 0) {
                ans.add(Arrays.asList(a[i], a[l], a[r]));
                l++; r--;
                while (l < r && a[l] == a[l - 1]) l++;
                while (l < r && a[r] == a[r + 1]) r--;
            } else if (s < 0) l++;
            else r--;
        }
    }
    return ans;
}
```

42. Trapping Rain Water

Brute force

For each bar, scan left for the tallest bar and right for the tallest bar. Water at i is min(leftMax, rightMax) - height[i] when positive. Each index rescans both sides.

Complexity: O(n^2) time, O(1) space.

```
int trapBrute(int[] h) {
    int water = 0;
    for (int i = 0; i < h.length; i++) {
        int left = 0, right = 0;
        for (int j = 0; j <= i; j++) left = Math.max(left, h[j]);
        for (int j = i; j < h.length; j++) right = Math.max(right, h[j]);
        water += Math.max(0, Math.min(left, right) - h[i]);
    }
    return water;
```

```
}
```

Better

Precompute leftMax[i] and rightMax[i] in two passes. Water at i uses those arrays in O(1). Same formula as brute, without rescanning every time.

Complexity: O(n) time, O(n) space.

```
int trapPrefix(int[] h) {
    int n = h.length;
    int[] left = new int[n], right = new int[n];
    left[0] = h[0];
    for (int i = 1; i < n; i++) left[i] = Math.max(left[i - 1], h[i]);
    right[n - 1] = h[n - 1];
    for (int i = n - 2; i >= 0; i--) right[i] = Math.max(right[i + 1], h[i]);
    int water = 0;
    for (int i = 0; i < n; i++)
        water += Math.max(0, Math.min(left[i], right[i]) - h[i]);
    return water;
}
```

Optimal

Two pointers maintain the tallest seen from the left and from the right. The side with the smaller max is the bottleneck: every unprocessed bar on that side is capped by that max, so add the difference and advance that pointer.

Complexity: O(n) time, O(1) space.

```
int trap(int[] h) {
    int l = 0, r = h.length - 1, left = 0, right = 0, water = 0;
    while (l < r) {
        if (h[l] <= h[r]) {
            left = Math.max(left, h[l]);
            water += left - h[l];
            l++;
        } else {
            right = Math.max(right, h[r]);
            water += right - h[r];
            r--;
        }
    }
    return water;
}
```

43. Container With Most Water

Brute force

Try every pair of lines (l, r). Area is width times the shorter height. Track the maximum.

Complexity: O(n^2) time, O(1) space.

```
int maxAreaBrute(int[] h) {
    int best = 0;
    for (int l = 0; l < h.length; l++)
        for (int r = l + 1; r < h.length; r++)
            best = Math.max(best, (r - l) * Math.min(h[l], h[r]));
    return best;
}
```

Optimal

Start at the widest pair. Moving the taller line inward cannot help: width shrinks and the shorter line is unchanged. Move the shorter line to seek a taller partner that pays for the lost width.

Complexity: O(n) time, O(1) space.

```
int maxArea(int[] h) {
    int l = 0, r = h.length - 1, best = 0;
    while (l < r) {
        best = Math.max(best, (r - l) * Math.min(h[l], h[r]));
```

```
        if (h[l] < h[r]) l++;
        else r--;
    }
    return best;
}
```

44. Remove Duplicates from Sorted Array

Brute force

For each read index, check whether the value already appears in the prefix [0..w]. If not, append it. Correct on sorted input but rescans the kept prefix every time.

Complexity: O(n^2) time, O(1) space.

```
int removeDuplicatesBrute(int[] a) {
    if (a.length == 0) return 0;
    int w = 0;
    for (int r = 1; r < a.length; r++) {
        boolean seen = false;
        for (int j = 0; j <= w; j++)
            if (a[j] == a[r]) { seen = true; break; }
        if (!seen) a[++w] = a[r];
    }
    return w + 1;
}
```

Optimal

Duplicates are adjacent after sorting. write stays on the last kept value; whenever a[read] differs, copy it just after write. The answer length is write + 1.

Complexity: O(n) time, O(1) space.

```
int removeDuplicates(int[] a) {
    if (a.length == 0) return 0;
    int w = 0;
    for (int r = 1; r < a.length; r++) {
        if (a[r] != a[w]) a[++w] = a[r];
    }
    return w + 1;
}
```

45. Max Consecutive Ones

Brute force

Every subarray is a candidate. Extend r from each start l while the subarray has no zero; track the longest run of ones.

Complexity: O(n^2) time, O(1) space.

```
int findMaxConsecutiveOnesBrute(int[] a) {
    int best = 0;
    for (int l = 0; l < a.length; l++) {
        for (int r = l; r < a.length && a[r] == 1; r++)
            best = Math.max(best, r - l + 1);
    }
    return best;
}
```

Optimal

A run of ones ends at every zero. Count the current run and remember the longest; a zero resets the run to zero.

Complexity: O(n) time, O(1) space.

```
int findMaxConsecutiveOnes(int[] a) {
    int cur = 0, best = 0;
    for (int x : a) {
        cur = x == 1 ? cur + 1 : 0;
        best = Math.max(best, cur);
    }
```

```
    return best;
}
```

46. Max Consecutive Ones III

Brute force

For each subarray, count zeros and keep it if the count is at most k. Track the longest valid length.

Complexity: O(n^2) time, O(1) space.

```
int longestOnesBrute(int[] a, int k) {
    int best = 0;
    for (int l = 0; l < a.length; l++) {
        int zeros = 0;
        for (int r = l; r < a.length; r++) {
            if (a[r] == 0) zeros++;
            if (zeros <= k) best = Math.max(best, r - l + 1);
            else break;
        }
    }
    return best;
}
```

Optimal

Flipping a zero equals allowing up to k zeros inside the window. Grow r; when zeros exceed k, shrink l until legal again. Both pointers only move forward.

Complexity: O(n) time, O(1) space.

```
int longestOnes(int[] a, int k) {
    int l = 0, zeros = 0, best = 0;
    for (int r = 0; r < a.length; r++) {
        if (a[r] == 0) zeros++;
        while (zeros > k) {
            if (a[l] == 0) zeros--;
            l++;
        }
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

47. Longest Substring Without Repeating Characters

Brute force

Try every substring [l, r]. Use a set to detect a repeated character; stop extending r when a duplicate appears.

Complexity: O(n^2) time, O(alphabet) space.

```
int lengthOfLongestSubstringBrute(String s) {
    int best = 0;
    for (int l = 0; l < s.length(); l++) {
        Set<Character> seen = new HashSet<>();
        for (int r = l; r < s.length(); r++) {
            if (!seen.add(s.charAt(r))) break;
            best = Math.max(best, r - l + 1);
        }
    }
    return best;
}
```

Better

Sliding window with a frequency map. Grow r; while a character count exceeds one, shrink l and decrement counts until the window is unique again.

Complexity: O(n) time, O(alphabet) space.

```
int lengthOfLongestSubstringSet(String s) {
    Map<Character, Integer> freq = new HashMap<>();
```

```
    int l = 0, best = 0;
    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        freq.put(c, freq.getOrDefault(c, 0) + 1);
        while (freq.get(c) > 1) {
            char left = s.charAt(l++);
            freq.put(left, freq.get(left) - 1);
        }
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

Optimal

Store the last index of each character. When a[r] was seen inside the current window, start the window after that index. Record r as the new last index.

Complexity: O(n) time, O(1) space for a fixed alphabet, else O(alphabet).

```
int lengthOfLongestSubstring(String s) {
    int[] last = new int[256];
    Arrays.fill(last, -1);
    int l = 0, best = 0;
    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        if (last[c] >= l) l = last[c] + 1;
        last[c] = r;
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

48. Minimum Window Substring

Brute force

Try every substring of s. A substring is valid when it contains every character of t with required multiplicity. Keep the shortest valid one.

Complexity: O(n^2 * m) time, O(1) space for a fixed alphabet.

```
String minWindowBrute(String s, String t) {
    int[] need = new int[128];
    for (int i = 0; i < t.length(); i++) need[t.charAt(i)]++;
    int best = Integer.MAX_VALUE, start = 0;
    for (int l = 0; l < s.length(); l++) {
        int[] have = new int[128];
        int formed = 0, needDistinct = 0;
        for (int c = 0; c < 128; c++) if (need[c] > 0) needDistinct++;
        for (int r = l; r < s.length(); r++) {
            char ch = s.charAt(r);
            if (need[ch] == 0) continue;
            if (++have[ch] == need[ch]) formed++;
            if (formed == needDistinct && r - l + 1 < best) {
                best = r - l + 1;
                start = l;
            }
        }
    }
    return best == Integer.MAX_VALUE ? "" : s.substring(start, start + best);
}
```

Optimal

need counts how many characters t still owes the window. Grow r until every requirement is satisfied, then shrink l while updating the best window until a required count goes positive again.

Complexity: O(n + m) time, O(1) space for a fixed alphabet.

```
String minWindow(String s, String t) {
```

```
    int[] need = new int[128];
    for (int i = 0; i < t.length(); i++) need[t.charAt(i)]++;
    int missing = t.length(), best = Integer.MAX_VALUE, start = 0;
    for (int l = 0, r = 0; r < s.length(); r++) {
        if (need[s.charAt(r)]-- > 0) missing--;
        while (missing == 0) {
            if (r - l + 1 < best) { best = r - l + 1; start = l; }
            if (++need[s.charAt(l)] > 0) missing++;
            l++;
        }
    }
    return best == Integer.MAX_VALUE ? "" : s.substring(start, start + best);
}
```

49. Fruit Into Baskets

Brute force

Two baskets mean at most two distinct fruit types. Check every subarray and count distinct values with a map; keep the longest valid window.

Complexity: O(n^2) time, O(1) space (at most three map entries).

```
int totalFruitBrute(int[] a) {
    int best = 0;
    for (int l = 0; l < a.length; l++) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int r = l; r < a.length; r++) {
            freq.put(a[r], freq.getOrDefault(a[r], 0) + 1);
            if (freq.size() <= 2) best = Math.max(best, r - l + 1);
            else break;
        }
    }
    return best;
}
```

Optimal

Frequency map tracks the window. When a third type appears, shrink from the left until one type disappears. The longest legal window is the answer.

Complexity: O(n) time, O(1) space.

```
int totalFruit(int[] a) {
    Map<Integer, Integer> freq = new HashMap<>();
    int l = 0, best = 0;
    for (int r = 0; r < a.length; r++) {
        freq.put(a[r], freq.getOrDefault(a[r], 0) + 1);
        while (freq.size() > 2) {
            freq.put(a[l], freq.get(a[l]) - 1);
            if (freq.get(a[l]) == 0) freq.remove(a[l]);
            l++;
        }
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

50. Number of Substrings Containing All Three Characters

Brute force

Count every substring. A substring is valid when it contains at least one a, one b, and one c.

Complexity: O(n^2) time, O(1) space.

```
int numberOfSubstringsBrute(String s) {
    int ans = 0;
    for (int l = 0; l < s.length(); l++) {
        boolean[] seen = new boolean[3];
        int have = 0;
```

```
        for (int r = l; r < s.length(); r++) {
            int idx = s.charAt(r) - 'a';
            if (!seen[idx]) { seen[idx] = true; have++; }
            if (have == 3) ans += s.length() - r;
            if (have == 3) break;
        }
    }
    return ans;
}
```

Optimal

Track the latest index of a, b, and c. Once all three have been seen up to r, every start from 0 through the earliest of those three indexes forms a valid substring ending at r. Add left + 1.

Complexity: O(n) time, O(1) space.

```
int numberOfSubstrings(String s) {
    int[] last = {-1, -1, -1};
    int ans = 0;
    for (int i = 0; i < s.length(); i++) {
        last[s.charAt(i) - 'a'] = i;
        int left = Math.min(last[0], Math.min(last[1], last[2]));
        if (left >= 0) ans += left + 1;
    }
    return ans;
}
```

51. Longest Repeating Character Replacement

Brute force

Try every substring. Replacements needed equal window length minus the count of the most frequent character inside the window.

Complexity: O(n^2) time, O(1) space.

```
int characterReplacementBrute(String s, int k) {
    int best = 0;
    for (int l = 0; l < s.length(); l++) {
        int[] freq = new int[26];
        int most = 0;
        for (int r = l; r < s.length(); r++) {
            most = Math.max(most, ++freq[s.charAt(r) - 'A']);
            if (r - l + 1 - most <= k) best = Math.max(best, r - l + 1);
        }
    }
    return best;
}
```

Optimal

Inside a window, keep the most frequent character and replace the rest. If replacements exceed k, shrink from the left. An overestimated max frequency never accepts a worse longer window because the best only grows.

Complexity: O(n) time, O(1) space.

```
int characterReplacement(String s, int k) {
    int[] freq = new int[26];
    int l = 0, most = 0, best = 0;
    for (int r = 0; r < s.length(); r++) {
        most = Math.max(most, ++freq[s.charAt(r) - 'A']);
        while (r - l + 1 - most > k) freq[s.charAt(l++) - 'A']--;
        best = Math.max(best, r - l + 1);
    }
    return best;
}
```

52. Binary Subarrays With Sum

Brute force

Count ones in every subarray. Increment the answer when the sum equals goal.

Complexity: O(n^2) time, O(1) space.

```
int numSubarraysWithSumBrute(int[] a, int goal) {
    int ans = 0;
    for (int l = 0; l < a.length; l++) {
        int sum = 0;
        for (int r = l; r < a.length; r++) {
            sum += a[r];
            if (sum == goal) ans++;
            if (sum > goal) break;
        }
    }
    return ans;
}
```

Better

Prefix sums: subarray sum equals pref[r + 1] - pref[l]. For each right end, add how many earlier prefixes gave the needed difference. Works for general arrays, not only binary.

Complexity: O(n) time, O(n) space.

```
int numSubarraysWithSumPrefix(int[] a, int goal) {
    int ans = 0, sum = 0;
    Map<Integer, Integer> cnt = new HashMap<>();
    cnt.put(0, 1);
    for (int x : a) {
        sum += x;
        ans += cnt.getOrDefault(sum - goal, 0);
        cnt.put(sum, cnt.getOrDefault(sum, 0) + 1);
    }
    return ans;
}
```

Optimal

On a binary array, count subarrays with sum at most goal by shrinking while sum exceeds goal; each right end contributes r - l + 1 starts. Exactly goal is atMost(goal) - atMost(goal - 1).

Complexity: O(n) time, O(1) space.

```
int numSubarraysWithSum(int[] a, int goal) {
    return atMost(a, goal) - atMost(a, goal - 1);
}
int atMost(int[] a, int goal) {
    if (goal < 0) return 0;
    int l = 0, sum = 0, ans = 0;
    for (int r = 0; r < a.length; r++) {
        sum += a[r];
        while (sum > goal) sum -= a[l++];
        ans += r - l + 1;
    }
    return ans;
}
```

Recursion and Backtracking

Choose, recurse, then undo the choice. The undo is what makes the same array or board reusable for the next branch.

53. Subset Sums

Brute force

Every subset is a bitmask from 0 to 2^n - 1. Sum the elements whose bit is set and collect all sums. Sort at the end if the statement wants ordered output.

Complexity: O(2^n * n) time, O(2^n) space for answers.

```
List<Integer> subsetSumsBrute(int[] a) {
    List<Integer> ans = new ArrayList<>();
    int n = a.length;
    for (int mask = 0; mask < (1 << n); mask++) {
        int sum = 0;
        for (int i = 0; i < n; i++)
            if ((mask & (1 << i)) != 0) sum += a[i];
        ans.add(sum);
    }
    Collections.sort(ans);
    return ans;
}
```

Optimal

Recurse on the next index twice: once adding a[i], once skipping it. Each finished path records one subset sum.

Complexity: O(2^n) time, O(n) recursion stack.

```
List<Integer> subsetSums(int[] a) {
    List<Integer> ans = new ArrayList<>();
    collect(a, 0, 0, ans);
    Collections.sort(ans);
    return ans;
}
void collect(int[] a, int i, int sum, List<Integer> ans) {
    if (i == a.length) { ans.add(sum); return; }
    collect(a, i + 1, sum + a[i], ans);
    collect(a, i + 1, sum, ans);
}
```

54. Subsets II

Brute force

Generate every subset via bitmask, convert to lists, sort each list and the outer list, then drop duplicates with a set or linear scan. Correct but wasteful on duplicate-heavy input.

Complexity: O(2^n * n log n) time, O(2^n * n) space.

```
List<List<Integer>> subsetsWithDupBrute(int[] a) {
    Set<String> keys = new HashSet<>();
    List<List<Integer>> ans = new ArrayList<>();
    int n = a.length;
    for (int mask = 0; mask < (1 << n); mask++) {
        List<Integer> cur = new ArrayList<>();
        for (int i = 0; i < n; i++)
            if ((mask & (1 << i)) != 0) cur.add(a[i]);
        Collections.sort(cur);
        String key = cur.toString();
        if (keys.add(key)) ans.add(cur);
    }
    return ans;
}
```

Optimal

Sort so equal values sit together. At each level skip duplicate siblings: if a[i] equals a[i - 1] and i > start, continue. That drops duplicate subsets while still allowing several copies when chosen in order.

Complexity: O(2^n * n) time, O(n) stack.

```
List<List<Integer>> subsetsWithDup(int[] a) {
    Arrays.sort(a);
    List<List<Integer>> ans = new ArrayList<>();
    build(a, 0, new ArrayList<>(), ans);
    return ans;
}
void build(int[] a, int start, List<Integer> cur, List<List<Integer>> ans) {
    ans.add(new ArrayList<>(cur));
    for (int i = start; i < a.length; i++) {
        if (i > start && a[i] == a[i - 1]) continue;
        cur.add(a[i]);
        build(a, i + 1, cur, ans);
        cur.remove(cur.size() - 1);
    }
}
```

55. Combination Sum

Brute force

For each candidate, try using it 0, 1, 2, ... times up to target / a[i], then move to the next index. Every count vector is explored; no take-or-skip shortcut at the same index.

Complexity: Exponential in target, O(target / min) stack.

```
List<List<Integer>> combinationSumBrute(int[] a, int target) {
    List<List<Integer>> ans = new ArrayList<>();
    countPick(a, 0, target, new ArrayList<>(), ans);
    return ans;
}
void countPick(int[] a, int i, int left, List<Integer> cur, List<List<Integer>> ans) {
    if (left == 0) { ans.add(new ArrayList<>(cur)); return; }
    if (i == a.length || left < 0) return;
    for (int take = 0; take * a[i] <= left; take++) {
        for (int t = 0; t < take; t++) cur.add(a[i]);
        countPick(a, i + 1, left - take * a[i], cur, ans);
        for (int t = 0; t < take; t++) cur.remove(cur.size() - 1);
    }
}
```

Optimal

Candidates may be reused, but order does not matter, so each recursive call only looks at indexes >= the one just chosen. Subtract the chosen value from the remaining target; undo before trying skip.

Complexity: Exponential in the target, O(target / min) stack.

```
List<List<Integer>> combinationSum(int[] a, int target) {
    List<List<Integer>> ans = new ArrayList<>();
    pick(a, 0, target, new ArrayList<>(), ans);
    return ans;
}
void pick(int[] a, int i, int left, List<Integer> cur, List<List<Integer>> ans) {
    if (left == 0) { ans.add(new ArrayList<>(cur)); return; }
    if (i == a.length || left < 0) return;
    cur.add(a[i]);
    pick(a, i, left - a[i], cur, ans);
    cur.remove(cur.size() - 1);
    pick(a, i + 1, left, cur, ans);
}
```

56. Combination Sum II

Brute force

Sort candidates and enumerate subsets that use each index at most once via bitmask. Keep subsets whose sum equals target; deduplicate by encoding sorted lists as strings.

Complexity: O(2^n * n) time, O(2^n) space.

```
List<List<Integer>> combinationSum2Brute(int[] a, int target) {
    Arrays.sort(a);
    Set<String> keys = new HashSet<>();
    List<List<Integer>> ans = new ArrayList<>();
    int n = a.length;
    for (int mask = 1; mask < (1 << n); mask++) {
        List<Integer> cur = new ArrayList<>();
        int sum = 0;
        for (int i = 0; i < n; i++)
            if ((mask & (1 << i)) != 0) { cur.add(a[i]); sum += a[i]; }
        if (sum != target) continue;
        String key = cur.toString();
        if (keys.add(key)) ans.add(cur);
    }
    return ans;
}
```

Optimal

Each element is used at most once, so the next call starts at i + 1. Skip duplicate values at the same recursion depth after sorting.

Complexity: O(2^n) time, O(n) stack.

```
List<List<Integer>> combinationSum2(int[] a, int target) {
    Arrays.sort(a);
    List<List<Integer>> ans = new ArrayList<>();
    pick(a, 0, target, new ArrayList<>(), ans);
    return ans;
}
void pick(int[] a, int start, int left, List<Integer> cur, List<List<Integer>> ans) {
    if (left == 0) { ans.add(new ArrayList<>(cur)); return; }
    for (int i = start; i < a.length && a[i] <= left; i++) {
        if (i > start && a[i] == a[i - 1]) continue;
        cur.add(a[i]);
        pick(a, i + 1, left - a[i], cur, ans);
        cur.remove(cur.size() - 1);
    }
}
```

57. Palindrome Partitioning

Brute force

There are 2^(n-1) ways to place cuts between characters. Encode cuts as a bitmask, split the string, and keep partitions where every piece is a palindrome.

Complexity: O(2^n * n) time, O(n) stack.

```
List<List<String>> partitionBrute(String s) {
    List<List<String>> ans = new ArrayList<>();
    int n = s.length();
    for (int mask = 0; mask < (1 << (n - 1)); mask++) {
        List<String> parts = new ArrayList<>();
        int start = 0;
        boolean ok = true;
        for (int i = 0; i < n - 1; i++) {
            if ((mask & (1 << i)) != 0) {
                String piece = s.substring(start, i + 1);
                if (!isPal(piece)) { ok = false; break; }
                parts.add(piece);
                start = i + 1;
```

```
            }
        }
        if (!ok) continue;
        String last = s.substring(start);
        if (isPal(last)) {
            parts.add(last);
            ans.add(parts);
        }
    }
    return ans;
}
boolean isPal(String t) {
    int l = 0, r = t.length() - 1;
    while (l < r) if (t.charAt(l++) != t.charAt(r--)) return false;
    return true;
}
```

Better

Backtrack cuts: from start, try every end where s[start..end] is a palindrome, append the piece, recurse from end + 1, then undo.

Complexity: O(n * 2^n) time, O(n) stack.

```
List<List<String>> partition(String s) {
    List<List<String>> ans = new ArrayList<>();
    cut(s, 0, new ArrayList<>(), ans);
    return ans;
}
void cut(String s, int start, List<String> cur, List<List<String>> ans) {
    if (start == s.length()) { ans.add(new ArrayList<>(cur)); return; }
    for (int end = start; end < s.length(); end++) {
        if (!isPal(s, start, end)) continue;
        cur.add(s.substring(start, end + 1));
        cut(s, end + 1, cur, ans);
        cur.remove(cur.size() - 1);
    }
}
boolean isPal(String s, int l, int r) {
    while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
    return true;
}
```

Optimal

Precompute isPal[l][r] in O(n^2). Backtracking then checks palindromes in O(1), which matters when the recursion tree is large.

Complexity: O(n^2 + n * 2^n) time, O(n^2) space for the table.

```
List<List<String>> partition(String s) {
    int n = s.length();
    boolean[][] pal = new boolean[n][n];
    for (int i = n - 1; i >= 0; i--)
        for (int j = i; j < n; j++)
            pal[i][j] = s.charAt(i) == s.charAt(j) && (j - i < 2 || pal[i + 1][j - 1]);
    List<List<String>> ans = new ArrayList<>();
    cut(s, 0, pal, new ArrayList<>(), ans);
    return ans;
}
void cut(String s, int start, boolean[][] pal, List<String> cur, List<List<String>> ans) {
    if (start == s.length()) { ans.add(new ArrayList<>(cur)); return; }
    for (int end = start; end < s.length(); end++) {
        if (!pal[start][end]) continue;
        cur.add(s.substring(start, end + 1));
        cut(s, end + 1, pal, cur, ans);
        cur.remove(cur.size() - 1);
    }
```

```
}
```

58. Permutations

Brute force

Sort a copy of the array and repeatedly apply next-permutation until the sequence wraps. Each step is one full permutation.

Complexity: O(n * n!) time, O(n) space.

```
List<List<Integer>> permuteBrute(int[] a) {
    int[] b = a.clone();
    Arrays.sort(b);
    List<List<Integer>> ans = new ArrayList<>();
    do {
        List<Integer> row = new ArrayList<>();
        for (int x : b) row.add(x);
        ans.add(row);
    } while (nextPerm(b));
    return ans;
}
boolean nextPerm(int[] a) {
    int i = a.length - 2;
    while (i >= 0 && a[i] >= a[i + 1]) i--;
    if (i < 0) return false;
    int j = a.length - 1;
    while (a[j] <= a[i]) j--;
    int t = a[i]; a[i] = a[j]; a[j] = t;
    for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {
        t = a[l]; a[l] = a[r]; a[r] = t;
    }
    return true;
}
```

Better

Fill position i by trying every unused value. A boolean used[] marks occupancy; undo the mark after the recursive call.

Complexity: O(n * n!) time, O(n) stack.

```
List<List<Integer>> permuteUsed(int[] a) {
    List<List<Integer>> ans = new ArrayList<>();
    boolean[] used = new boolean[a.length];
    build(a, used, new ArrayList<>(), ans);
    return ans;
}
void build(int[] a, boolean[] used, List<Integer> cur, List<List<Integer>> ans) {
    if (cur.size() == a.length) { ans.add(new ArrayList<>(cur)); return; }
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        cur.add(a[i]);
        build(a, used, cur, ans);
        cur.remove(cur.size() - 1);
        used[i] = false;
    }
}
```

Optimal

Swap a[i] with each later choice to place the next element, recurse to i + 1, then swap back so the array is restored for the next branch.

Complexity: O(n * n!) time, O(n) stack.

```
List<List<Integer>> permute(int[] a) {
    List<List<Integer>> ans = new ArrayList<>();
    perm(a, 0, ans);
    return ans;
```

```
}
void perm(int[] a, int i, List<List<Integer>> ans) {
    if (i == a.length) {
        List<Integer> row = new ArrayList<>();
        for (int x : a) row.add(x);
        ans.add(row);
        return;
    }
    for (int j = i; j < a.length; j++) {
        int t = a[i]; a[i] = a[j]; a[j] = t;
        perm(a, i + 1, ans);
        t = a[i]; a[i] = a[j]; a[j] = t;
    }
}
```

59. Kth Permutation Sequence

Brute force

Generate permutations in lex order with next-permutation and return the kth (1-based). Stops as soon as the counter reaches k.

Complexity: O(n * k) time in typical cases, O(n * n!) worst case, O(n) space.

```
String getPermutationBrute(int n, int k) {
    int[] a = new int[n];
    for (int i = 0; i < n; i++) a[i] = i + 1;
    for (int step = 1; step < k; step++)
        if (!nextPerm(a)) break;
    StringBuilder sb = new StringBuilder();
    for (int x : a) sb.append(x);
    return sb.toString();
}
boolean nextPerm(int[] a) {
    int i = a.length - 2;
    while (i >= 0 && a[i] >= a[i + 1]) i--;
    if (i < 0) return false;
    int j = a.length - 1;
    while (a[j] <= a[i]) j--;
    int t = a[i]; a[i] = a[j]; a[j] = t;
    for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {
        t = a[l]; a[l] = a[r]; a[r] = t;
    }
    return true;
}
```

Optimal

There are (n - 1)! permutations for each first digit. Pick index k / (n - 1)! from the remaining list, set k to k % (n - 1)!, and repeat. Direct construction without enumerating all n! orders.

Complexity: O(n^2) time, O(n) space.

```
String getPermutation(int n, int k) {
    List<Integer> nums = new ArrayList<>();
    int fact = 1;
    for (int i = 1; i <= n; i++) { nums.add(i); fact *= i; }
    k--;
    StringBuilder sb = new StringBuilder();
    for (int left = n; left >= 1; left--) {
        fact /= left;
        int idx = k / fact;
        sb.append(nums.remove(idx));
        k %= fact;
    }
    return sb.toString();
}
```

60. N-Queens

Brute force

Place a queen in every cell of row r and recurse. Reject a placement when any two queens share a row, column, or diagonal. No column/diagonal bitmask pruning beyond checking the board.

Complexity: O(n^n) time worst case, O(n^2) space.

```
List<List<String>> solveNQueensBrute(int n) {
    List<List<String>> ans = new ArrayList<>();
    char[][] b = new char[n][n];
    for (char[] row : b) Arrays.fill(row, '.');
    placeBrute(0, n, b, ans);
    return ans;
}
void placeBrute(int r, int n, char[][] b, List<List<String>> ans) {
    if (r == n) {
        List<String> one = new ArrayList<>();
        for (char[] row : b) one.add(new String(row));
        ans.add(one);
        return;
    }
    for (int c = 0; c < n; c++) {
        if (!safe(b, r, c, n)) continue;
        b[r][c] = 'Q';
        placeBrute(r + 1, n, b, ans);
        b[r][c] = '.';
    }
}
boolean safe(char[][] b, int r, int c, int n) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < n; j++)
            if (b[i][j] == 'Q' && (j == c || i - j == r - c || i + j == r + c))
                return false;
    return true;
}
```

Optimal

One queen per row. Track used columns and both diagonal families with boolean arrays; place, mark, recurse, unmark.

Complexity: O(n!) time, O(n) extra space.

```
List<List<String>> solveNQueens(int n) {
    List<List<String>> ans = new ArrayList<>();
    char[][] b = new char[n][n];
    for (char[] row : b) Arrays.fill(row, '.');
    place(0, n, b, new boolean[n], new boolean[2 * n], new boolean[2 * n], ans);
    return ans;
}
void place(int r, int n, char[][] b, boolean[] col, boolean[] d1, boolean[] d2,
           List<List<String>> ans) {
    if (r == n) {
        List<String> one = new ArrayList<>();
        for (char[] row : b) one.add(new String(row));
        ans.add(one);
        return;
    }
    for (int c = 0; c < n; c++) {
        int x = r - c + n - 1, y = r + c;
        if (col[c] || d1[x] || d2[y]) continue;
        col[c] = d1[x] = d2[y] = true;
        b[r][c] = 'Q';
        place(r + 1, n, b, col, d1, d2, ans);
        b[r][c] = '.';
        col[c] = d1[x] = d2[y] = false;
```

```
    }
}
```

61. Sudoku Solver

Brute force

Collect every empty cell. Try digits 1 through 9 at each empty in order, recurse to the next empty, and backtrack on failure. Row/column/box checks run from scratch each time.

Complexity: Exponential, O(1) extra space besides the stack.

```
boolean solveSudokuBrute(char[][] b) {
    List<int[]> empty = new ArrayList<>();
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++)
            if (b[r][c] == '.') empty.add(new int[]{r, c});
    return fill(b, empty, 0);
}
boolean fill(char[][] b, List<int[]> empty, int idx) {
    if (idx == empty.size()) return true;
    int r = empty.get(idx)[0], c = empty.get(idx)[1];
    for (char d = '1'; d <= '9'; d++) {
        if (!ok(b, r, c, d)) continue;
        b[r][c] = d;
        if (fill(b, empty, idx + 1)) return true;
        b[r][c] = '.';
    }
    return false;
}
boolean ok(char[][] b, int r, int c, char d) {
    for (int i = 0; i < 9; i++) {
        if (b[r][i] == d || b[i][c] == d) return false;
        int br = 3 * (r / 3) + i / 3, bc = 3 * (c / 3) + i % 3;
        if (b[br][bc] == d) return false;
    }
    return true;
}
```

Optimal

Scan for the next '.' and try legal digits with the same ok test. First complete assignment wins; typical boards prune heavily so this in-place scan is standard.

Complexity: Exponential, pruning makes typical boards small. O(1) extra space besides the stack.

```
boolean solveSudoku(char[][] b) {
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (b[r][c] != '.') continue;
            for (char d = '1'; d <= '9'; d++) {
                if (!ok(b, r, c, d)) continue;
                b[r][c] = d;
                if (solveSudoku(b)) return true;
                b[r][c] = '.';
            }
            return false;
        }
    }
    return true;
}
boolean ok(char[][] b, int r, int c, char d) {
    for (int i = 0; i < 9; i++) {
        if (b[r][i] == d || b[i][c] == d) return false;
        int br = 3 * (r / 3) + i / 3, bc = 3 * (c / 3) + i % 3;
        if (b[br][bc] == d) return false;
    }
    return true;
}
```

62. M-Coloring

Brute force

Assign colors 1..m to vertices in order without checking neighbors mid-search. When all vertices are colored, validate every edge. Backtrack only after a full assignment fails.

Complexity: O(m^n * n) worst case, O(n) stack.

```
boolean graphColoringBrute(boolean[][] graph, int m) {
    int n = graph.length;
    int[] color = new int[n];
    return assignAll(graph, color, 0, m);
}
boolean assignAll(boolean[][] g, int[] color, int v, int m) {
    if (v == g.length) return coloringValid(g, color);
    for (int c = 1; c <= m; c++) {
        color[v] = c;
        if (assignAll(g, color, v + 1, m)) return true;
    }
    color[v] = 0;
    return false;
}
boolean coloringValid(boolean[][] g, int[] color) {
    for (int v = 0; v < g.length; v++)
        for (int u = v + 1; u < g.length; u++)
            if (g[v][u] && color[v] == color[u]) return false;
    return true;
}
```

Optimal

Color vertices in order. Before recursing, only try colors that no neighbor already uses. Undo on failure.

Complexity: O(m^n * n) worst case, O(n) stack.

```
boolean graphColoring(boolean[][] graph, int m) {
    return paint(graph, new int[graph.length], 0, m);
}
boolean paint(boolean[][] g, int[] color, int v, int m) {
    if (v == g.length) return true;
    for (int c = 1; c <= m; c++) {
        boolean ok = true;
        for (int u = 0; u < g.length; u++)
            if (g[v][u] && color[u] == c) ok = false;
        if (!ok) continue;
        color[v] = c;
        if (paint(g, color, v + 1, m)) return true;
        color[v] = 0;
    }
    return false;
}
```

63. Rat in a Maze

Brute force

DFS from (0, 0) with a separate visited matrix copied at each step so every path exploration is independent. Collect path strings when (n - 1, n - 1) is reached.

Complexity: O(4^(n^2)) worst case, O(n^2) stack depth.

```
List<String> findPathBrute(int[][] maze) {
    List<String> ans = new ArrayList<>();
    int n = maze.length;
    if (maze[0][0] == 0 || maze[n - 1][n - 1] == 0) return ans;
    walkBrute(maze, 0, 0, new boolean[n][n], new StringBuilder(), ans);
    return ans;
}
void walkBrute(int[][] a, int r, int c, boolean[][] vis, StringBuilder path,
               List<String> ans) {
```

```
    int n = a.length;
    if (r == n - 1 && c == n - 1) { ans.add(path.toString()); return; }
    vis[r][c] = true;
    int[] dr = {1, 0, 0, -1};
    int[] dc = {0, -1, 1, 0};
    char[] dir = {'D', 'L', 'R', 'U'};
    for (int k = 0; k < 4; k++) {
        int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nc < 0 || nr >= n || nc >= n || a[nr][nc] == 0 || vis[nr][nc])
            continue;
        path.append(dir[k]);
        walkBrute(a, nr, nc, vis, path, ans);
        path.deleteCharAt(path.length() - 1);
    }
    vis[r][c] = false;
}
```

Optimal

Mark the current cell blocked on the grid, try D/L/R/U in order, append the direction letter, then restore the cell and remove the letter on backtrack.

Complexity: O(4^(n^2)) worst case, O(n^2) stack.

```
List<String> findPath(int[][] maze) {
    List<String> ans = new ArrayList<>();
    int n = maze.length;
    if (maze[0][0] == 0 || maze[n - 1][n - 1] == 0) return ans;
    walk(maze, 0, 0, new StringBuilder(), ans);
    return ans;
}
void walk(int[][] a, int r, int c, StringBuilder path, List<String> ans) {
    int n = a.length;
    if (r == n - 1 && c == n - 1) { ans.add(path.toString()); return; }
    a[r][c] = 0;
    int[] dr = {1, 0, 0, -1};
    int[] dc = {0, -1, 1, 0};
    char[] dir = {'D', 'L', 'R', 'U'};
    for (int k = 0; k < 4; k++) {
        int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nc < 0 || nr >= n || nc >= n || a[nr][nc] == 0) continue;
        path.append(dir[k]);
        walk(a, nr, nc, path, ans);
        path.deleteCharAt(path.length() - 1);
    }
    a[r][c] = 1;
}
```

64. Word Search

Brute force

From every cell matching the first letter, DFS with a fresh visited[][] so paths cannot reuse a cell. Try four directions for each next character.

Complexity: O(n * m * 4^L) time, O(L) stack, L = word length.

```
boolean existBrute(char[][] board, String word) {
    int rows = board.length, cols = board[0].length;
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            if (board[r][c] == word.charAt(0)
                    && dfsBrute(board, word, 0, r, c, new boolean[rows][cols]))
                return true;
    return false;
}
boolean dfsBrute(char[][] b, String w, int i, int r, int c, boolean[][] vis) {
    if (i == w.length()) return true;
    if (r < 0 || c < 0 || r >= b.length || c >= b[0].length) return false;
```

```
    if (vis[r][c] || b[r][c] != w.charAt(i)) return false;
    vis[r][c] = true;
    boolean ok = dfsBrute(b, w, i + 1, r + 1, c, vis) || dfsBrute(b, w, i + 1, r - 1, c, vis)
              || dfsBrute(b, w, i + 1, r, c + 1, vis) || dfsBrute(b, w, i + 1, r, c - 1, vis);
    vis[r][c] = false;
    return ok;
}
```

Optimal

Same DFS from each matching start, but mark the cell with a sentinel on the board and restore it after the call. That avoids allocating a visited matrix per path.

Complexity: O(n * m * 4^L) time, O(L) stack, L = word length.

```
boolean exist(char[][] board, String word) {
    for (int r = 0; r < board.length; r++)
        for (int c = 0; c < board[0].length; c++)
            if (dfs(board, word, 0, r, c)) return true;
    return false;
}
boolean dfs(char[][] b, String w, int i, int r, int c) {
    if (i == w.length()) return true;
    if (r < 0 || c < 0 || r >= b.length || c >= b[0].length) return false;
    if (b[r][c] != w.charAt(i)) return false;
    char saved = b[r][c];
    b[r][c] = '#';
    boolean ok = dfs(b, w, i + 1, r + 1, c) || dfs(b, w, i + 1, r - 1, c)
              || dfs(b, w, i + 1, r, c + 1) || dfs(b, w, i + 1, r, c - 1);
    b[r][c] = saved;
    return ok;
}
```

Linked List

ListNode is val plus next. Most of these functions only rewire next pointers. Fast and slow pointers replace an index.

65. Reverse a Linked List

Brute force

Walk the list once and push every value onto a stack. Pop the stack into new nodes chained from a dummy head. Order is reversed without touching original links, at the cost of extra storage and a new list.

Complexity: O(n) time, O(n) space.

```
ListNode reverseListBrute(ListNode head) {
    Deque<Integer> st = new ArrayDeque<>();
    for (ListNode cur = head; cur != null; cur = cur.next) st.push(cur.val);
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!st.isEmpty()) {
        tail.next = new ListNode(st.pop());
        tail = tail.next;
    }
    return dummy.next;
}
```

Better

Recursion reverses the suffix after head, then attaches head at the tail of that suffix. The base case is null or a single node. Each call uses O(n) stack depth.

Complexity: O(n) time, O(n) recursion stack.

```
ListNode reverseListRec(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode newHead = reverseListRec(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}
```

Optimal

Three pointers: previous, current, next. Each node must point at the node that used to precede it. Save cur.next before overwriting it, point cur at prev, then step both forward. At the end prev is the new head. Empty and one-node lists need no special case.

Complexity: O(n) time, O(1) space.

```
ListNode reverseList(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur != null) {
        ListNode nxt = cur.next;
        cur.next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}
```

66. Middle of a Linked List

Brute force

Count nodes in one pass, then walk again stopping after count / 2 steps. Correct but touches the list twice.

Complexity: O(n) time, O(1) space.

```
ListNode middleNodeCount(ListNode head) {
    int n = 0;
    for (ListNode cur = head; cur != null; cur = cur.next) n++;
    ListNode cur = head;
    for (int i = 0; i < n / 2; i++) cur = cur.next;
```

```
    return cur;
}
```

Optimal

Fast moves two steps, slow moves one. When fast reaches the end, slow has covered half as many nodes. For even length, slow lands on the second middle. One pass, no length count.

Complexity: O(n) time, O(1) space.

```
ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

67. Merge Two Sorted Lists

Brute force

Copy all values from both lists into an array, sort it, then build a fresh list from the sorted values. Ignores that the inputs were already sorted.

Complexity: O((n + m) log(n + m)) time, O(n + m) space.

```
ListNode mergeTwoListsBrute(ListNode a, ListNode b) {
    List<Integer> vals = new ArrayList<>();
    for (ListNode cur = a; cur != null; cur = cur.next) vals.add(cur.val);
    for (ListNode cur = b; cur != null; cur = cur.next) vals.add(cur.val);
    Collections.sort(vals);
    ListNode dummy = new ListNode(0), tail = dummy;
    for (int v : vals) {
        tail.next = new ListNode(v);
        tail = tail.next;
    }
    return dummy.next;
}
```

Optimal

Dummy head, always attach the smaller node. While both lists remain, attach the smaller head and advance that list. Attach whichever list is left in one assignment. Return dummy.next.

Complexity: O(n + m) time, O(1) space.

```
ListNode mergeTwoLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.next = a; a = a.next; }
        else { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = a != null ? a : b;
    return dummy.next;
}
```

68. Remove Nth Node From End

Brute force

Count the length, then delete the node at index length - n from the head using a dummy to handle deleting the head.

Complexity: O(n) time, O(1) space.

```
ListNode removeNthFromEndCount(ListNode head, int n) {
    int len = 0;
    for (ListNode cur = head; cur != null; cur = cur.next) len++;
    ListNode dummy = new ListNode(0);
    dummy.next = head;
```

```
    ListNode cur = dummy;
    for (int i = 0; i < len - n; i++) cur = cur.next;
    cur.next = cur.next.next;
    return dummy.next;
}
```

Optimal

Gap of n between two pointers, plus a dummy. Advance fast n steps, then move slow and fast together until fast reaches the end. Slow stands on the node before the one to delete. Dummy handles deleting the original head.

Complexity: O(n) time, O(1) space.

```
ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next != null) { fast = fast.next; slow = slow.next; }
    slow.next = slow.next.next;
    return dummy.next;
}
```

69. Add Two Numbers

Better

Same digit-by-digit addition as the iterative version, expressed recursively on the tails after consuming one digit from each list and the carry.

Complexity: O(max(n, m)) time, O(max(n, m)) space for the answer and recursion.

```
ListNode addTwoNumbersRec(ListNode a, ListNode b, int carry) {
    if (a == null && b == null && carry == 0) return null;
    int sum = carry;
    if (a != null) { sum += a.val; a = a.next; }
    if (b != null) { sum += b.val; b = b.next; }
    ListNode node = new ListNode(sum % 10);
    node.next = addTwoNumbersRec(a, b, sum / 10);
    return node;
}
ListNode addTwoNumbers(ListNode a, ListNode b) {
    return addTwoNumbersRec(a, b, 0);
}
```

Optimal

Both lists store digits least-significant first. Add the two digits plus carry, write sum % 10 as the next node, keep sum / 10 as carry. Continue while either list or carry remains.

Complexity: O(max(n, m)) time, O(max(n, m)) space for the answer.

```
ListNode addTwoNumbers(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    int carry = 0;
    while (a != null || b != null || carry != 0) {
        int sum = carry;
        if (a != null) { sum += a.val; a = a.next; }
        if (b != null) { sum += b.val; b = b.next; }
        tail.next = new ListNode(sum % 10);
        tail = tail.next;
        carry = sum / 10;
    }
    return dummy.next;
}
```

70. Delete a Node When Only That Node Is Given

Optimal

The previous node is unknown and the node is not the tail. Copy the successor's value into this node and bypass the successor. The list values match deletion; the original node object remains.

Complexity: O(1) time, O(1) space.

```
void deleteNode(ListNode node) {
    node.val = node.next.val;
    node.next = node.next.next;
}
```

71. Intersection of Two Linked Lists

Brute force

For every node in list A, walk list B and compare references. First match is the intersection. Simple but quadratic when lists are long with no shared tail.

Complexity: O(n * m) time, O(1) space.

```
ListNode getIntersectionNodeBrute(ListNode a, ListNode b) {
    for (ListNode p = a; p != null; p = p.next)
        for (ListNode q = b; q != null; q = q.next)
            if (p == q) return p;
    return null;
}
```

Optimal

Switch heads when a pointer finishes. Both walk a + b + c steps before entering the shared tail together, or both become null if there is no intersection.

Complexity: O(n + m) time, O(1) space.

```
ListNode getIntersectionNode(ListNode a, ListNode b) {
    ListNode p = a, q = b;
    while (p != q) {
        p = p == null ? b : p.next;
        q = q == null ? a : q.next;
    }
    return p;
}
```

72. Detect a Cycle

Brute force

Track visited nodes in a hash set while walking. The first repeat means a cycle; reaching null means none.

Complexity: O(n) time, O(n) space.

```
boolean hasCycleHash(ListNode head) {
    Set<ListNode> seen = new HashSet<>();
    for (ListNode cur = head; cur != null; cur = cur.next) {
        if (!seen.add(cur)) return true;
    }
    return false;
}
```

Optimal

Floyd: slow and fast. Inside a cycle, fast gains one step on slow each iteration until they meet. If fast reaches null, there is no cycle.

Complexity: O(n) time, O(1) space.

```
boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
```

```
    }
    return false;
}
```

73. Starting Point of the Cycle

Brute force

Record each node in a hash set on first visit. The first node seen twice is the cycle entrance.

Complexity: O(n) time, O(n) space.

```
ListNode detectCycleHash(ListNode head) {
    Set<ListNode> seen = new HashSet<>();
    for (ListNode cur = head; cur != null; cur = cur.next) {
        if (!seen.add(cur)) return cur;
    }
    return null;
}
```

Optimal

After Floyd finds a meeting point, reset one pointer to the head and advance both one step at a time. They meet at the cycle entrance because the stem length equals the distance from the meeting point to the entrance along the cycle.

Complexity: O(n) time, O(1) space.

```
ListNode detectCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) { slow = slow.next; fast = fast.next; }
            return slow;
        }
    }
    return null;
}
```

74. Reverse Nodes in k-Group

Brute force

Copy values into an array, reverse each block of k indices in place, then write values back into the original nodes without changing structure beyond values.

Complexity: O(n) time, O(n) space.

```
ListNode reverseKGroupBrute(ListNode head, int k) {
    List<Integer> vals = new ArrayList<>();
    for (ListNode cur = head; cur != null; cur = cur.next) vals.add(cur.val);
    for (int i = 0; i + k <= vals.size(); i += k) {
        int l = i, r = i + k - 1;
        while (l < r) {
            int t = vals.get(l); vals.set(l, vals.get(r)); vals.set(r, t);
            l++; r--;
        }
    }
    ListNode cur = head;
    for (int v : vals) { cur.val = v; cur = cur.next; }
    return head;
}
```

Better

Iteratively count k nodes, reverse that block with three pointers, stitch the old head to the next block's result, and advance. Same logic as recursion without call stack.

Complexity: O(n) time, O(1) extra space.

```
ListNode reverseKGroupIter(ListNode head, int k) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prevGroup = dummy;
    while (true) {
        ListNode kth = prevGroup;
        for (int i = 0; i < k && kth != null; i++) kth = kth.next;
        if (kth == null) break;
        ListNode groupNext = kth.next;
        ListNode prev = groupNext, cur = prevGroup.next;
        while (cur != groupNext) {
            ListNode nxt = cur.next;
            cur.next = prev;
            prev = cur;
            cur = nxt;
        }
        ListNode tmp = prevGroup.next;
        prevGroup.next = kth;
        prevGroup = tmp;
    }
    return dummy.next;
}
```

Optimal

Count k nodes; if fewer remain, return head unchanged. Reverse the block with three pointers, then recursively reverse the suffix and attach. A dummy makes the first stitch like the rest.

Complexity: O(n) time, O(n / k) recursion stack.

```
ListNode reverseKGroup(ListNode head, int k) {
    ListNode cur = head;
    for (int i = 0; i < k; i++) {
        if (cur == null) return head;
        cur = cur.next;
    }
    ListNode prev = null, node = head;
    for (int i = 0; i < k; i++) {
        ListNode nxt = node.next;
        node.next = prev;
        prev = node;
        node = nxt;
    }
    head.next = reverseKGroup(node, k);
    return prev;
}
```

75. Palindrome Linked List

Brute force

Copy values into an array and compare from both ends with two indices. Easy but uses extra space.

Complexity: O(n) time, O(n) space.

```
boolean isPalindromeBrute(ListNode head) {
    List<Integer> a = new ArrayList<>();
    for (ListNode cur = head; cur != null; cur = cur.next) a.add(cur.val);
    int l = 0, r = a.size() - 1;
    while (l < r) {
        if (!a.get(l++).equals(a.get(r--))) return false;
    }
    return true;
}
```

Optimal

Find the middle with slow and fast, reverse the second half in place, compare head with reversed half. Optionally reverse again to restore the list.

Complexity: O(n) time, O(1) space.

```
boolean isPalindrome(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    ListNode right = reverseList(slow), left = head;
    while (right != null) {
        if (left.val != right.val) return false;
        left = left.next;
        right = right.next;
    }
    return true;
}
```

76. Flatten a Multilevel Sorted List

Brute force

Collect every value via DFS on next and bottom, sort, then rebuild a single-level list with only next pointers.

Complexity: O(N log N) time for N total nodes, O(N) space.

```
Node flattenBrute(Node head) {
    List<Integer> vals = new ArrayList<>();
    collect(head, vals);
    Collections.sort(vals);
    Node dummy = new Node(0), tail = dummy;
    for (int v : vals) {
        tail.next = new Node(v);
        tail = tail.next;
    }
    return dummy.next;
}
void collect(Node node, List<Integer> vals) {
    while (node != null) {
        vals.add(node.val);
        if (node.bottom != null) collect(node.bottom, vals);
        node = node.next;
    }
}
```

Optimal

Recursively flatten everything to the right, then merge this column with that result using sorted merge on bottom pointers, clearing next on merged nodes.

Complexity: O(total nodes) time, O(columns) stack.

```
Node flatten(Node head) {
    if (head == null || head.next == null) return head;
    head.next = flatten(head.next);
    return mergeBottom(head, head.next);
}
Node mergeBottom(Node a, Node b) {
    Node dummy = new Node(0), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.bottom = a; a = a.bottom; }
        else { tail.bottom = b; b = b.bottom; }
        tail = tail.bottom;
        tail.next = null;
    }
    tail.bottom = a != null ? a : b;
    return dummy.bottom;
```

```
}
```

77. Rotate List

Brute force

Repeat k % n times: save the tail, detach it, and make it the new head. Each rotation is one full walk to the tail.

Complexity: O(n * k) time, O(1) space.

```
ListNode rotateRightBrute(ListNode head, int k) {
    if (head == null || head.next == null) return head;
    int n = 0;
    for (ListNode cur = head; cur != null; cur = cur.next) n++;
    k %= n;
    if (k == 0) return head;
    for (int r = 0; r < k; r++) {
        ListNode cur = head, prev = null;
        while (cur.next != null) { prev = cur; cur = cur.next; }
        prev.next = null;
        cur.next = head;
        head = cur;
    }
    return head;
}
```

Optimal

Find length and tail, connect tail to head to form a circle, then break at position n - k % n. The node after the break is the new head.

Complexity: O(n) time, O(1) space.

```
ListNode rotateRight(ListNode head, int k) {
    if (head == null || head.next == null) return head;
    int n = 1;
    ListNode tail = head;
    while (tail.next != null) { tail = tail.next; n++; }
    k %= n;
    if (k == 0) return head;
    tail.next = head;
    ListNode neu = head;
    for (int i = 0; i < n - k - 1; i++) neu = neu.next;
    ListNode start = neu.next;
    neu.next = null;
    return start;
}
```

78. Copy List with Random Pointer

Brute force

Two passes with a hash map from each original node to its clone. First pass creates clones, second pass wires next and random from the map.

Complexity: O(n) time, O(n) space.

```
Node copyRandomListMap(Node head) {
    if (head == null) return null;
    Map<Node, Node> map = new HashMap<>();
    for (Node cur = head; cur != null; cur = cur.next)
        map.put(cur, new Node(cur.val));
    for (Node cur = head; cur != null; cur = cur.next) {
        map.get(cur).next = map.get(cur.next);
        map.get(cur).random = map.get(cur.random);
    }
    return map.get(head);
}
```

Optimal

Insert each clone between a node and its next so clone of x.random is x.random.next. Set random links, then detach clones in a third pass. No hash map.

Complexity: O(n) time, O(1) extra space.

```
Node copyRandomList(Node head) {
    if (head == null) return null;
    for (Node cur = head; cur != null; cur = cur.next.next) {
        Node copy = new Node(cur.val);
        copy.next = cur.next;
        cur.next = copy;
    }
    for (Node cur = head; cur != null; cur = cur.next.next)
        if (cur.random != null) cur.next.random = cur.random.next;
    Node dummy = new Node(0), tail = dummy;
    for (Node cur = head; cur != null; cur = cur.next) {
        tail.next = cur.next;
        tail = tail.next;
        cur.next = cur.next.next;
    }
    return dummy.next;
}
```

Stack and Queues

A stack gives the nearest smaller or greater element in one pass. A monotonic stack stores indexes whose values are still waiting for that neighbor.

79. Valid Parentheses

Brute force

Repeatedly find and remove adjacent matching pairs like () or [] until the string is empty or no pair remains. Valid iff nothing is left. Each scan is O(n); worst case this degrades badly on nested strings.

Complexity: O(n^2) time, O(n) space for the mutable string.

```
boolean isValidBrute(String s) {
    StringBuilder sb = new StringBuilder(s);
    boolean changed = true;
    while (changed && sb.length() > 0) {
        changed = false;
        for (int i = 0; i + 1 < sb.length(); i++) {
            char a = sb.charAt(i), b = sb.charAt(i + 1);
            if ((a == '(' && b == ')') || (a == '{' && b == '}')
                    || (a == '[' && b == ']')) {
                sb.delete(i, i + 2);
                changed = true;
                break;
            }
        }
    }
    return sb.length() == 0;
}
```

Optimal

Push openings. A closing bracket is valid only when the stack top is its partner; then pop. Invalid on empty stack or mismatch. Valid when the stack is empty at the end.

Complexity: O(n) time, O(n) space.

```
boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (c == '(' || c == '{' || c == '[') st.push(c);
        else if (st.isEmpty()) return false;
        else {
            char o = st.pop();
            if (c == ')' && o != '(') return false;
            if (c == '}' && o != '{') return false;
            if (c == ']' && o != '[') return false;
        }
    }
    return st.isEmpty();
}
```

80. Min Stack

Better

Keep two stacks: one for values and one for the minimum after each push. Pop both together. Slightly more bookkeeping than pairs but the idea is transparent.

Complexity: O(1) per operation, O(n) space.

```
class MinStackTwoStacks {
    Deque<Integer> vals = new ArrayDeque<>();
    Deque<Integer> mins = new ArrayDeque<>();
    void push(int val) {
        vals.push(val);
        mins.push(mins.isEmpty() ? val : Math.min(val, mins.peek()));
```

```
    }
    void pop() { vals.pop(); mins.pop(); }
    int top() { return vals.peek(); }
    int getMin() { return mins.peek(); }
}
```

Optimal

Each stack entry stores the value and the minimum at that moment. getMin reads the second half of the top pair; pop discards both.

Complexity: O(1) per operation, O(n) space.

```
class MinStack {
    Deque<int[]> st = new ArrayDeque<>();
    void push(int val) {
        int mn = st.isEmpty() ? val : Math.min(val, st.peek()[1]);
        st.push(new int[]{val, mn});
    }
    void pop() { st.pop(); }
    int top() { return st.peek()[0]; }
    int getMin() { return st.peek()[1]; }
}
```

81. Queue Using Two Stacks

Brute force

On every pop or peek, pour the entire inbox into the outbox so the oldest element is on top. Simple but each operation can cost O(n).

Complexity: O(n) per pop/peek, O(n) space.

```
class MyQueueBrute {
    Deque<Integer> in = new ArrayDeque<>();
    Deque<Integer> out = new ArrayDeque<>();
    void push(int x) { in.push(x); }
    int pop() {
        while (!in.isEmpty()) out.push(in.pop());
        return out.pop();
    }
    int peek() {
        while (!in.isEmpty()) out.push(in.pop());
        return out.peek();
    }
    boolean empty() { return in.isEmpty() && out.isEmpty(); }
}
```

Optimal

Pour inbox into outbox only when outbox is empty. Each element moves once, so pop and peek are amortized O(1).

Complexity: Amortized O(1) per operation, O(n) space.

```
class MyQueue {
    Deque<Integer> in = new ArrayDeque<>();
    Deque<Integer> out = new ArrayDeque<>();
    void push(int x) { in.push(x); }
    void pour() {
        if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop());
    }
    int pop() { pour(); return out.pop(); }
    int peek() { pour(); return out.peek(); }
    boolean empty() { return in.isEmpty() && out.isEmpty(); }
}
```

82. Stack Using Two Queues

Better

Maintain two queues. Push enqueues into the non-empty queue, or q1 if both empty. Pop dequeues from the only non-empty queue after moving all but one element to the other queue so the last element is removed. Push stays O(1); pop costs O(n).

Complexity: O(1) push, O(n) pop, O(n) space.

```
class MyStackTwoQueues {
    Queue<Integer> q1 = new ArrayDeque<>();
    Queue<Integer> q2 = new ArrayDeque<>();
    void push(int x) {
        if (!q1.isEmpty()) q1.offer(x);
        else q2.offer(x);
    }
    int pop() {
        Queue<Integer> big = q1.size() >= q2.size() ? q1 : q2;
        Queue<Integer> small = big == q1 ? q2 : q1;
        while (big.size() > 1) small.offer(big.poll());
        return big.poll();
    }
    int top() {
        int v = pop();
        push(v);
        return v;
    }
    boolean empty() { return q1.isEmpty() && q2.isEmpty(); }
}
```

Optimal

Single queue: after enqueueing x, rotate by dequeuing and enqueueing size - 1 elements so x sits at the front. pop and top read the front.

Complexity: O(n) push, O(1) pop, O(n) space.

```
class MyStack {
    Queue<Integer> q = new ArrayDeque<>();
    void push(int x) {
        q.offer(x);
        for (int i = q.size() - 1; i > 0; i--) q.offer(q.poll());
    }
    int pop() { return q.poll(); }
    int top() { return q.peek(); }
    boolean empty() { return q.isEmpty(); }
}
```

83. Next Greater Element

Brute force

For each index, scan right until a larger value appears or return -1.

Complexity: O(n^2) time, O(1) extra space besides output.

```
int[] nextGreaterBrute(int[] a) {
    int n = a.length, ans[] = new int[n];
    for (int i = 0; i < n; i++) {
        ans[i] = -1;
        for (int j = i + 1; j < n; j++) {
            if (a[j] > a[i]) { ans[i] = a[j]; break; }
        }
    }
    return ans;
}
```

Optimal

Monotonic decreasing stack of indexes. When a[i] beats the top value, it is that index's next greater. Pop until decreasing again, then push i.

Complexity: O(n) time, O(n) space.

```
int[] nextGreater(int[] a) {
    int n = a.length, ans[] = new int[n];
    Arrays.fill(ans, -1);
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[st.peek()] < a[i]) ans[st.pop()] = a[i];
        st.push(i);
    }
    return ans;
}
```

84. Next Smaller Element

Brute force

For each index, scan right for the first strictly smaller value.

Complexity: O(n^2) time, O(1) extra space besides output.

```
int[] nextSmallerBrute(int[] a) {
    int n = a.length, ans[] = new int[n];
    for (int i = 0; i < n; i++) {
        ans[i] = -1;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[i]) { ans[i] = a[j]; break; }
        }
    }
    return ans;
}
```

Optimal

Monotonic increasing stack of indexes. When a[i] is smaller than the top, it answers every popped index. Remaining indexes get -1.

Complexity: O(n) time, O(n) space.

```
int[] nextSmaller(int[] a) {
    int n = a.length, ans[] = new int[n];
    Arrays.fill(ans, -1);
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[st.peek()] > a[i]) ans[st.pop()] = a[i];
        st.push(i);
    }
    return ans;
}
```

85. Previous Smaller Element

Brute force

For each index, scan left for the nearest strictly smaller value.

Complexity: O(n^2) time, O(1) extra space besides output.

```
int[] previousSmallerBrute(int[] a) {
    int n = a.length, ans[] = new int[n];
    for (int i = 0; i < n; i++) {
        ans[i] = -1;
        for (int j = i - 1; j >= 0; j--) {
            if (a[j] < a[i]) { ans[i] = a[j]; break; }
        }
    }
    return ans;
}
```

Optimal

Increasing stack: before pushing i, pop values >= a[i]. The top, if any, is the previous smaller; then push i.

Complexity: O(n) time, O(n) space.

```
int[] previousSmaller(int[] a) {
    int n = a.length, ans[] = new int[n];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[st.peek()] >= a[i]) st.pop();
        ans[i] = st.isEmpty() ? -1 : a[st.peek()];
        st.push(i);
    }
    return ans;
}
```

86. Stock Span

Brute force

For each day, walk left while prices are at most today's price and count days included.

Complexity: O(n^2) time, O(1) extra space besides output.

```
int[] stockSpanBrute(int[] price) {
    int n = price.length, span[] = new int[n];
    for (int i = 0; i < n; i++) {
        span[i] = 1;
        for (int j = i - 1; j >= 0 && price[j] <= price[i]; j--) span[i]++;
    }
    return span;
}
```

Optimal

Decreasing stack of indexes. Pop days with price <= today; span is i - top or i + 1 if empty.

Complexity: O(n) time, O(n) space.

```
int[] stockSpan(int[] price) {
    int n = price.length, span[] = new int[n];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && price[st.peek()] <= price[i]) st.pop();
        span[i] = st.isEmpty() ? i + 1 : i - st.peek();
        st.push(i);
    }
    return span;
}
```

87. Largest Rectangle in Histogram

Brute force

For each bar, expand left and right while heights stay at least h[i], then area is h[i] times width. Correct but each bar may scan O(n) neighbors.

Complexity: O(n^2) time, O(1) extra space.

```
int largestRectangleAreaBrute(int[] h) {
    int n = h.length, best = 0;
    for (int i = 0; i < n; i++) {
        int left = i, right = i;
        while (left > 0 && h[left - 1] >= h[i]) left--;
        while (right + 1 < n && h[right + 1] >= h[i]) right++;
        best = Math.max(best, h[i] * (right - left + 1));
    }
    return best;
}
```

Optimal

Previous and next strictly smaller index for each bar with monotonic stacks. Width is right - left - 1; maximize h[i] * width.

Complexity: O(n) time, O(n) space.

```
int largestRectangleArea(int[] h) {
    int n = h.length;
    int[] left = new int[n], right = new int[n];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && h[st.peek()] >= h[i]) st.pop();
        left[i] = st.isEmpty() ? -1 : st.peek();
        st.push(i);
    }
    st.clear();
    for (int i = n - 1; i >= 0; i--) {
        while (!st.isEmpty() && h[st.peek()] >= h[i]) st.pop();
        right[i] = st.isEmpty() ? n : st.peek();
        st.push(i);
    }
    int best = 0;
    for (int i = 0; i < n; i++)
        best = Math.max(best, h[i] * (right[i] - left[i] - 1));
    return best;
}
```

88. Maximal Rectangle

Brute force

Enumerate every submatrix by top/bottom rows and left/right columns, check if all cells are '1', track max area. Fine for tiny grids only.

Complexity: O(rows^2 * cols^2) time, O(1) extra space.

```
int maximalRectangleBrute(char[][] matrix) {
    if (matrix.length == 0) return 0;
    int rows = matrix.length, cols = matrix[0].length, best = 0;
    for (int r1 = 0; r1 < rows; r1++) {
        for (int r2 = r1; r2 < rows; r2++) {
            for (int c1 = 0; c1 < cols; c1++) {
                for (int c2 = c1; c2 < cols; c2++) {
                    boolean ok = true;
                    for (int r = r1; r <= r2 && ok; r++)
                        for (int c = c1; c <= c2; c++)
                            if (matrix[r][c] == '0') ok = false;
                    if (ok) best = Math.max(best, (r2 - r1 + 1) * (c2 - c1 + 1));
                }
            }
        }
    }
    return best;
}
```

Optimal

Each row builds a histogram height of consecutive ones above. The largest rectangle in that histogram is the best rectangle ending on this row. Max over all rows.

Complexity: O(rows * cols) time, O(cols) space.

```
int maximalRectangle(char[][] matrix) {
    if (matrix.length == 0) return 0;
    int m = matrix[0].length, best = 0;
    int[] h = new int[m];
    for (char[] row : matrix) {
        for (int c = 0; c < m; c++)
            h[c] = row[c] == '1' ? h[c] + 1 : 0;
```

```
        best = Math.max(best, largestRectangleArea(h));
    }
    return best;
}
```

89. Sliding Window Maximum

Brute force

For each window start, scan the k elements and take the maximum.

Complexity: O(n * k) time, O(1) extra space besides output.

```
int[] maxSlidingWindowBrute(int[] a, int k) {
    int n = a.length;
    int[] ans = new int[n - k + 1];
    for (int i = 0; i <= n - k; i++) {
        int mx = a[i];
        for (int j = i; j < i + k; j++) mx = Math.max(mx, a[j]);
        ans[i] = mx;
    }
    return ans;
}
```

Better

Max-heap of (value, index) for the window. Push each new index; pop from the top while the index is outside the window. Root is the current maximum.

Complexity: O(n log k) time, O(k) space.

```
int[] maxSlidingWindowHeap(int[] a, int k) {
    int n = a.length;
    int[] ans = new int[n - k + 1];
    PriorityQueue<int[]> pq = new PriorityQueue<>((x, y) -> y[0] - x[0]);
    for (int i = 0; i < n; i++) {
        pq.offer(new int[]{a[i], i});
        while (pq.peek()[1] <= i - k) pq.poll();
        if (i >= k - 1) ans[i - k + 1] = pq.peek()[0];
    }
    return ans;
}
```

Optimal

Deque of indexes with decreasing values. Drop front when out of window; drop back while value <= a[i]. Front is the window max.

Complexity: O(n) time, O(k) space.

```
int[] maxSlidingWindow(int[] a, int k) {
    int n = a.length;
    int[] ans = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();
        dq.offerLast(i);
        if (i >= k - 1) ans[i - k + 1] = a[dq.peekFirst()];
    }
    return ans;
}
```

90. The Celebrity Problem

Brute force

Try each person as candidate celebrity: everyone else must know them and they must know nobody.

Complexity: O(n^2) time, O(1) space.

```
int celebrityBrute(int[][] mat) {
    int n = mat.length;
```

```
    for (int c = 0; c < n; c++) {
        boolean ok = true;
        for (int i = 0; i < n && ok; i++) {
            if (i == c) continue;
            if (mat[c][i] == 1 || mat[i][c] == 0) ok = false;
        }
        if (ok) return c;
    }
    return -1;
}
```

Optimal

Eliminate one person per comparison: if cand knows i, cand is wrong; else i is wrong. Verify the survivor against everyone else.

Complexity: O(n) time, O(1) space.

```
int celebrity(int[][] mat) {
    int n = mat.length, cand = 0;
    for (int i = 1; i < n; i++)
        if (mat[cand][i] == 1) cand = i;
    for (int i = 0; i < n; i++) {
        if (i == cand) continue;
        if (mat[cand][i] == 1 || mat[i][cand] == 0) return -1;
    }
    return cand;
}
```

91. LRU Cache

Brute force

Store entries in a list. get finds the key linearly, moves the entry to the front, or returns -1. put updates or inserts at front and removes the tail when over capacity.

Complexity: O(n) get and put, O(capacity) space.

```
class LRUCacheBrute {
    class E { int key, val; E(int k, int v) { key = k; val = v; } }
    List<E> list = new LinkedList<>();
    int cap;
    LRUCacheBrute(int capacity) { cap = capacity; }
    int get(int key) {
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i).key == key) {
                E e = list.remove(i);
                list.add(0, e);
                return e.val;
            }
        }
        return -1;
    }
    void put(int key, int value) {
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i).key == key) {
                list.remove(i);
                list.add(0, new E(key, value));
                return;
            }
        }
        if (list.size() == cap) list.remove(list.size() - 1);
        list.add(0, new E(key, value));
    }
}
```

Better

Java LinkedHashMap with access-order true evicts the eldest entry when size exceeds capacity. get and put are O(1) amortized with less pointer wiring.

Complexity: O(1) get and put, O(capacity) space.

```
class LRUCacheLinkedHashMap extends LinkedHashMap<Integer, Integer> {
    int cap;
    LRUCacheLinkedHashMap(int capacity) {
        super(capacity, 0.75f, true);
        cap = capacity;
    }
    protected boolean removeEldestEntry(Map.Entry<Integer, Integer> e) {
        return size() > cap;
    }
    int get(int key) {
        return super.getOrDefault(key, -1);
    }
    void put(int key, int value) { super.put(key, value); }
}
```

Optimal

Hash map from key to doubly linked list node; list ordered by recency with dummy head and tail. moveFront on access; delete tail.prev on eviction.

Complexity: O(1) get and put, O(capacity) space.

```
class LRUCache {
    class N {
        int key, val; N prev, next;
        N(int k, int v) { key = k; val = v; }
    }
    Map<Integer, N> map = new HashMap<>();
    N head = new N(0, 0), tail = new N(0, 0);
    int cap;
    LRUCache(int capacity) {
        cap = capacity;
        head.next = tail; tail.prev = head;
    }
    int get(int key) {
        if (!map.containsKey(key)) return -1;
        N node = map.get(key);
        moveFront(node);
        return node.val;
    }
    void put(int key, int value) {
        if (map.containsKey(key)) {
            N node = map.get(key);
            node.val = value;
            moveFront(node);
            return;
        }
        if (map.size() == cap) {
            N last = tail.prev;
            map.remove(last.key);
            last.prev.next = tail;
            tail.prev = last.prev;
        }
        N node = new N(key, value);
        map.put(key, node);
        node.next = head.next; node.prev = head;
        head.next.prev = node; head.next = node;
    }
    void moveFront(N node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
```

```
        node.next = head.next; node.prev = head;
        head.next.prev = node; head.next = node;
    }
}
```

92. Sum of Subarray Minimums

Brute force

Enumerate every subarray, track its minimum, add to the running sum modulo 10^9 + 7.

Complexity: O(n^2) time, O(1) extra space.

```
int sumSubarrayMinsBrute(int[] a) {
    int n = a.length, mod = 1_000_000_007;
    long ans = 0;
    for (int i = 0; i < n; i++) {
        int mn = a[i];
        for (int j = i; j < n; j++) {
            mn = Math.min(mn, a[j]);
            ans = (ans + mn) % mod;
        }
    }
    return (int) ans;
}
```

Optimal

Each index is the minimum on a unique range: previous strictly smaller on the left, next smaller-or-equal on the right. Contribution is a[i] * leftSpan * rightSpan.

Complexity: O(n) time, O(n) space.

```
int sumSubarrayMins(int[] a) {
    int n = a.length, mod = 1_000_000_007;
    int[] left = new int[n], right = new int[n];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[st.peek()] > a[i]) st.pop();
        left[i] = st.isEmpty() ? i + 1 : i - st.peek();
        st.push(i);
    }
    st.clear();
    for (int i = n - 1; i >= 0; i--) {
        while (!st.isEmpty() && a[st.peek()] >= a[i]) st.pop();
        right[i] = st.isEmpty() ? n - i : st.peek() - i;
        st.push(i);
    }
    long ans = 0;
    for (int i = 0; i < n; i++)
        ans = (ans + (long) a[i] * left[i] % mod * right[i]) % mod;
    return (int) ans;
}
```

93. Remove K Digits

Brute force

Try deleting every subset of k positions (or recurse on which digit to drop). Keeps the lexicographically smallest remaining number. Exponential in k.

Complexity: O(C(n, k) * n) time, O(n) space.

```
String removeKdigitsBrute(String num, int k) {
    String best = null;
    best = dfs(num, k, 0, new StringBuilder(), best);
    if (best == null || best.isEmpty()) return "0";
    return best;
}
String dfs(String num, int k, int i, StringBuilder cur, String best) {
    if (k == 0) {
```

```
        String rest = cur.toString() + num.substring(i);
        int p = 0;
        while (p < rest.length() && rest.charAt(p) == '0') p++;
        rest = p == rest.length() ? "" : rest.substring(p);
        if (best == null || rest.compareTo(best) < 0) return rest;
        return best;
    }
    if (i >= num.length()) return best;
    best = dfs(num, k, i + 1, cur, best);
    cur.append(num.charAt(i));
    best = dfs(num, k - 1, i + 1, cur, best);
    cur.deleteCharAt(cur.length() - 1);
    return best;
}
```

Optimal

Monotonic increasing stack of digits. While top is larger than the current digit and removals remain, pop. Trim remaining k from the end, strip leading zeros, return "0" if empty.

Complexity: O(n) time, O(n) space.

```
String removeKdigits(String num, int k) {
    StringBuilder st = new StringBuilder();
    for (int i = 0; i < num.length(); i++) {
        char c = num.charAt(i);
        while (k > 0 && st.length() > 0 && st.charAt(st.length() - 1) > c) {
            st.deleteCharAt(st.length() - 1);
            k--;
        }
        st.append(c);
    }
    st.setLength(st.length() - k);
    int i = 0;
    while (i < st.length() && st.charAt(i) == '0') i++;
    String ans = st.substring(i);
    return ans.isEmpty() ? "0" : ans;
}
```

94. Asteroid Collision

Brute force

Repeatedly scan for a right-moving positive followed by a left-moving negative, resolve that collision, and restart until no collision remains.

Complexity: O(n^2) time, O(n) space.

```
int[] asteroidCollisionBrute(int[] a) {
    List<Integer> list = new ArrayList<>();
    for (int x : a) list.add(x);
    boolean hit = true;
    while (hit) {
        hit = false;
        for (int i = 0; i + 1 < list.size(); i++) {
            int x = list.get(i), y = list.get(i + 1);
            if (x > 0 && y < 0) {
                hit = true;
                if (x < -y) list.remove(i);
                else if (x > -y) list.remove(i + 1);
                else { list.remove(i + 1); list.remove(i); }
                break;
            }
        }
    }
    int[] ans = new int[list.size()];
    for (int i = 0; i < ans.length; i++) ans[i] = list.get(i);
    return ans;
}
```

Optimal

Stack of surviving asteroids. Only a positive top collides with an incoming negative. Pop while smaller; equal sizes destroy both.

Complexity: O(n) time, O(n) space.

```
int[] asteroidCollision(int[] a) {
    Deque<Integer> st = new ArrayDeque<>();
    for (int x : a) {
        boolean alive = true;
        while (alive && x < 0 && !st.isEmpty() && st.peek() > 0) {
            if (st.peek() < -x) st.pop();
            else if (st.peek() == -x) { st.pop(); alive = false; }
            else alive = false;
        }
        if (alive) st.push(x);
    }
    int[] ans = new int[st.size()];
    for (int i = ans.length - 1; i >= 0; i--) ans[i] = st.pop();
    return ans;
}
```

Greedy

Choose the locally best move when a proof shows it never blocks a global optimum. Sorting, exchange arguments, and interval sweeps are the usual tools.

95. N Meetings in One Room (Activity Selection)

Brute force

Each meeting is either taken or skipped. Enumerate all 2^n subsets, keep only subsets whose meetings do not overlap, and return the largest size. Correct but exponential.

Complexity: O(2^n * n) time, O(n) space.

```
int maxMeetingsBrute(int[][] m) {
    int n = m.length, best = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int prevEnd = -1, cnt = 0, ok = true;
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) == 0) continue;
            if (m[i][0] < prevEnd) { ok = false; break; }
            prevEnd = m[i][1];
            cnt++;
        }
        if (ok) best = Math.max(best, cnt);
    }
    return best;
}
```

Optimal

Sort by finish time. Take the next meeting if its start is not before the last chosen finish. Finishing early leaves the most room for later meetings, so an exchange argument shows no optimal schedule uses fewer meetings.

Complexity: O(n log n) time, O(1) extra space.

```
int maxMeetings(int[][] m) {
    Arrays.sort(m, (a, b) -> Integer.compare(a[1], b[1]));
    int cnt = 0, end = -1;
    for (int[] x : m) {
        if (x[0] >= end) { cnt++; end = x[1]; }
    }
    return cnt;
}
```

96. Minimum Number of Platforms

Better

Sort arrivals and departures separately. Walk both arrays: if the next arrival is before or when the next train leaves, one more platform is needed and the arrival pointer moves; otherwise a platform frees and the departure pointer moves. Track the peak count.

Complexity: O(n log n) time, O(1) extra space besides sorting.

```
int findPlatform(int[] arr, int[] dep) {
    Arrays.sort(arr);
    Arrays.sort(dep);
    int i = 0, j = 0, cur = 0, best = 0;
    while (i < arr.length) {
        if (arr[i] <= dep[j]) { cur++; i++; best = Math.max(best, cur); }
        else { cur--; j++; }
    }
    return best;
}
```

Optimal

Build events: +1 at each arrival, -1 at each departure. Sort by time; at equal times process departures before arrivals so a train leaving at t frees a platform for one arriving at t. Scan and keep the running maximum.

Complexity: O(n log n) time, O(n) space for events.

```
int findPlatformEvents(int[] arr, int[] dep) {
    int n = arr.length;
    int[][] ev = new int[2 * n][2];
    for (int i = 0; i < n; i++) {
        ev[2 * i] = new int[]{arr[i], 1};
        ev[2 * i + 1] = new int[]{dep[i], -1};
    }
    Arrays.sort(ev, (a, b) -> a[0] != b[0] ? Integer.compare(a[0], b[0])
            : Integer.compare(a[1], b[1]));
    int cur = 0, best = 0;
    for (int[] e : ev) {
        cur += e[1];
        best = Math.max(best, cur);
    }
    return best;
}
```

97. Job Sequencing Problem

Brute force

Try every ordering of jobs. For each order, assign each job to the latest free slot on or before its deadline and sum profit of jobs that fit. Keep the best profit. Correct but factorial.

Complexity: O(n! * n) time, O(n) space.

```
int jobSeqBrute(int[] profit, int[] deadline) {
    int n = profit.length;
    Integer[] idx = new Integer[n];
    for (int i = 0; i < n; i++) idx[i] = i;
    int best = 0;
    do {
        int maxD = 0;
        for (int i : idx) maxD = Math.max(maxD, deadline[i]);
        boolean[] used = new boolean[maxD + 1];
        int sum = 0;
        for (int i : idx) {
            for (int d = deadline[i]; d >= 1; d--) {
                if (!used[d]) { used[d] = true; sum += profit[i]; break; }
            }
        }
        best = Math.max(best, sum);
    } while (nextPerm(idx));
    return best;
}

boolean nextPerm(Integer[] a) {
    int i = a.length - 2;
    while (i >= 0 && a[i] >= a[i + 1]) i--;
    if (i < 0) return false;
    int j = a.length - 1;
    while (a[j] <= a[i]) j--;
    int t = a[i]; a[i] = a[j]; a[j] = t;
    for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {
        t = a[l]; a[l] = a[r]; a[r] = t;
    }
    return true;
}
```

Optimal

Sort jobs by decreasing profit. For each job, place it in the latest free day on or before its deadline using a parent array that points to the previous free slot (disjoint-set style). A high-profit job placed late never blocks a lower-profit job that needs an earlier day.

Complexity: O(n^2) time with simple slot scan; O(n log n) with DSU, O(n) space.

```
int jobSequencing(int[] profit, int[] deadline) {
    int n = profit.length;
    Integer[] id = new Integer[n];
    for (int i = 0; i < n; i++) id[i] = i;
    Arrays.sort(id, (a, b) -> Integer.compare(profit[b], profit[a]));
    int maxD = 0;
    for (int d : deadline) maxD = Math.max(maxD, d);
    int[] parent = new int[maxD + 2];
    for (int i = 0; i <= maxD + 1; i++) parent[i] = i;
    int total = 0, jobs = 0;
    for (int j : id) {
        int d = find(parent, deadline[j]);
        if (d > 0) {
            parent[d] = find(parent, d - 1);
            total += profit[j];
            jobs++;
        }
    }
    return total;
}

int find(int[] p, int x) {
    if (p[x] != x) p[x] = find(p, p[x]);
    return p[x];
}
```

98. Fractional Knapsack

Better

0/1 knapsack: each item is fully taken or skipped. DP over capacity records the best value using the first i items. This is the right model when splitting items is forbidden, but it does not solve the fractional version.

Complexity: O(n * W) time, O(W) space.

```
int knapsack01(int[] wt, int[] val, int W) {
    int[] dp = new int[W + 1];
    for (int i = 0; i < wt.length; i++)
        for (int c = W; c >= wt[i]; c--)
            dp[c] = Math.max(dp[c], dp[c - wt[i]] + val[i]);
    return dp[W];
}
```

Optimal

When fractions are allowed, take items in decreasing value-to-weight ratio until the knapsack is full. If the next whole item does not fit, take a fraction of it. Any solution that replaces a taken fraction with a worse ratio can be improved by swapping weight, so the greedy order is optimal.

Complexity: O(n log n) time, O(1) extra space besides sorting.

```
double fractionalKnapsack(int[] wt, int[] val, int W) {
    int n = wt.length;
    Integer[] id = new Integer[n];
    for (int i = 0; i < n; i++) id[i] = i;
    Arrays.sort(id, (a, b) -> Double.compare(
            (double) val[b] / wt[b], (double) val[a] / wt[a]));
    double ans = 0;
    for (int i : id) {
        if (W >= wt[i]) { ans += val[i]; W -= wt[i]; }
        else { ans += (double) val[i] * W / wt[i]; break; }
    }
```

```
    return ans;
}
```

99. Minimum Coins (Greedy vs DP)

Brute force

BFS on the amount: from amount a, try subtracting each coin denomination and enqueue a - c when non-negative. The first time 0 is reached, the path length is the minimum number of coins. Works for any coin system.

Complexity: O(amount * k) time, O(amount) space.

```
int coinChangeBfs(int[] coins, int amount) {
    if (amount == 0) return 0;
    boolean[] seen = new boolean[amount + 1];
    Deque<Integer> q = new ArrayDeque<>();
    q.offer(amount);
    seen[amount] = true;
    for (int steps = 1; !q.isEmpty(); steps++) {
        int sz = q.size();
        for (int t = 0; t < sz; t++) {
            int a = q.poll();
            for (int c : coins) {
                int nxt = a - c;
                if (nxt == 0) return steps;
                if (nxt > 0 && !seen[nxt]) { seen[nxt] = true; q.offer(nxt); }
            }
        }
    }
    return -1;
}
```

Better

Unbounded knapsack-style DP: dp[a] is the minimum coins to make a. For each amount, try every coin. Correct for arbitrary denominations, unlike a blind greedy pass.

Complexity: O(amount * k) time, O(amount) space.

```
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int c : coins)
            if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Optimal

For canonical systems (typical US coins: each denomination is at least the sum of smaller ones), repeatedly take the largest coin that does not exceed the remaining amount. One pass, no table. Fails on counterexamples such as coins [1, 3, 4] for amount 6.

Complexity: O(k) time after sorting coins, O(1) extra space.

```
int coinChangeGreedy(int[] coins, int amount) {
    Arrays.sort(coins);
    int cnt = 0;
    for (int i = coins.length - 1; i >= 0 && amount > 0; i--) {
        while (amount >= coins[i]) { amount -= coins[i]; cnt++; }
    }
    return amount == 0 ? cnt : -1;
}
```

100. Assign Cookies

Brute force

For each child in order, scan every unused cookie and assign the first cookie that satisfies their greed. Simple but can waste large cookies on small children when a sorted matching would do better.

Complexity: O(n * m) time, O(m) space for used flags.

```
int findContentChildrenBrute(int[] g, int[] s) {
    boolean[] used = new boolean[s.length];
    int ans = 0;
    for (int need : g) {
        for (int j = 0; j < s.length; j++) {
            if (!used[j] && s[j] >= need) { used[j] = true; ans++; break; }
        }
    }
    return ans;
}
```

Optimal

Sort children by greed and cookies by size. Two pointers: if the current cookie satisfies the current child, both advance; otherwise only the cookie pointer advances to find a large enough cookie. Never skip a cookie that could satisfy the smallest unserved child.

Complexity: O(n log n + m log m) time, O(1) extra space.

```
int findContentChildren(int[] g, int[] s) {
    Arrays.sort(g);
    Arrays.sort(s);
    int i = 0, j = 0;
    while (i < g.length && j < s.length) {
        if (s[j] >= g[i]) { i++; j++; }
        else j++;
    }
    return i;
}
```

101. Lemonade Change

Brute force

At each customer, try every valid way to make change (which bills to return) and recurse on the rest of the line. Correct for tiny n but explores redundant choices.

Complexity: O(c^n) time for constant c change choices, O(n) recursion depth.

```
boolean lemonadeChangeBrute(int[] bills, int idx, int f, int t) {
    if (idx == bills.length) return true;
    int b = bills[idx];
    if (b == 5) return lemonadeChangeBrute(bills, idx + 1, f + 1, t);
    if (b == 10) {
        if (f == 0) return false;
        return lemonadeChangeBrute(bills, idx + 1, f - 1, t + 1);
    }
    if (f > 0 && t > 0) return lemonadeChangeBrute(bills, idx + 1, f - 1, t - 1);
    if (f >= 2) return lemonadeChangeBrute(bills, idx + 1, f - 2, t);
    return false;
}
```

Optimal

Customers pay 5, 10, or 20. Track counts of 5s and 10s in the drawer. A 10 needs one 5; a 20 prefers one 10 and one 5, otherwise three 5s. Greedy change use preserves 5s when possible because only 10s need them.

Complexity: O(n) time, O(1) space.

```
boolean lemonadeChange(int[] bills) {
    int f = 0, t = 0;
    for (int b : bills) {
        if (b == 5) f++;
```

```
        else if (b == 10) { if (f == 0) return false; f--; t++; }
        else {
            if (t > 0 && f > 0) { t--; f--; }
            else if (f >= 2) f -= 2;
            else return false;
        }
    }
    return true;
}
```

102. Jump Game I

Brute force

From index i, try every jump length from 1 to nums[i] and recurse. Memoize reachable positions to avoid recomputation. Still explores many paths.

Complexity: O(n^2) time, O(n) space.

```
boolean canJumpBrute(int[] nums) {
    Boolean[] memo = new Boolean[nums.length];
    return canJumpFrom(nums, 0, memo);
}

boolean canJumpFrom(int[] nums, int i, Boolean[] memo) {
    if (i >= nums.length - 1) return true;
    if (memo[i] != null) return memo[i];
    for (int step = 1; step <= nums[i]; step++)
        if (canJumpFrom(nums, i + step, memo)) return memo[i] = true;
    return memo[i] = false;
}
```

Optimal

Track the farthest index reachable from any position seen so far. Scan left to right; if i is beyond that farthest, the gap cannot be crossed. Otherwise extend farthest to i + nums[i]. Reaching the last index is enough.

Complexity: O(n) time, O(1) space.

```
boolean canJump(int[] nums) {
    int far = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > far) return false;
        far = Math.max(far, i + nums[i]);
        if (far >= nums.length - 1) return true;
    }
    return true;
}
```

103. Jump Game II

Brute force

BFS treats each index as a state and enqueues all indices reachable in one jump. The level when the last index is first dequeued is the minimum number of jumps.

Complexity: O(n^2) time in the worst case, O(n) space.

```
int jumpBfs(int[] nums) {
    int n = nums.length;
    if (n <= 1) return 0;
    Deque<Integer> q = new ArrayDeque<>();
    boolean[] seen = new boolean[n];
    q.offer(0);
    seen[0] = true;
    for (int jumps = 0; !q.isEmpty(); jumps++) {
        int sz = q.size();
        for (int t = 0; t < sz; t++) {
            int i = q.poll();
            for (int step = 1; step <= nums[i]; step++) {
                int j = i + step;
```

```
                if (j >= n - 1) return jumps + 1;
                if (!seen[j]) { seen[j] = true; q.offer(j); }
            }
        }
    }
    return -1;
}
```

Better

dp[i] is the minimum jumps to reach i. For each j < i with j + nums[j] >= i, relax dp[i] from dp[j] + 1. Straightforward dynamic programming.

Complexity: O(n^2) time, O(n) space.

```
int jumpDp(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, n);
    dp[0] = 0;
    for (int i = 1; i < n; i++)
        for (int j = 0; j < i; j++)
            if (j + nums[j] >= i) dp[i] = Math.min(dp[i], dp[j] + 1);
    return dp[n - 1];
}
```

Optimal

Greedy on jump layers. currentEnd is the last index of the current jump layer; farthest is the best reach using one more step from that layer. When i hits currentEnd, increment jumps and set currentEnd to farthest. Each jump is forced to cover as far as possible before the next is spent.

Complexity: O(n) time, O(1) space.

```
int jump(int[] nums) {
    int jumps = 0, curEnd = 0, far = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        far = Math.max(far, i + nums[i]);
        if (i == curEnd) { jumps++; curEnd = far; }
    }
    return jumps;
}
```

104. Candy

Brute force

Binary search the total candy count T. Check feasibility by scanning ratings and enforcing each child beats neighbors when ratings rise; impossible if the implied minimum exceeds T. Correct but heavy for large n.

Complexity: O(n log S) time where S is sum of upper bounds, O(n) space.

```
int candyBrute(int[] ratings) {
    int n = ratings.length;
    int[] c = new int[n];
    return assign(ratings, c, 0) ? sum(c) : -1;
}

boolean assign(int[] r, int[] c, int i) {
    if (i == r.length) return valid(r, c);
    for (int give = 1; give <= r.length; give++) {
        c[i] = give;
        if (assign(r, c, i + 1)) return true;
    }
    return false;
}

boolean valid(int[] r, int[] c) {
    for (int i = 0; i < r.length; i++) {
        if (i > 0 && r[i] > r[i - 1] && c[i] <= c[i - 1]) return false;
```

```
        if (i + 1 < r.length && r[i] > r[i + 1] && c[i] <= c[i + 1]) return false;
    }
    return true;
}

int sum(int[] c) {
    int s = 0;
    for (int x : c) s += x;
    return s;
}
```

Optimal

Two passes. Left to right: if rating rises, candy is one more than the left neighbor, else one. Right to left: if rating rises versus the right neighbor, take max with right neighbor plus one. Each pass enforces one side of the constraint; the max merges both.

Complexity: O(n) time, O(n) space.

```
int candy(int[] ratings) {
    int n = ratings.length;
    int[] c = new int[n];
    Arrays.fill(c, 1);
    for (int i = 1; i < n; i++)
        if (ratings[i] > ratings[i - 1]) c[i] = c[i - 1] + 1;
    for (int i = n - 2; i >= 0; i--)
        if (ratings[i] > ratings[i + 1]) c[i] = Math.max(c[i], c[i + 1] + 1);
    int sum = 0;
    for (int x : c) sum += x;
    return sum;
}
```

105. Insert Interval

Brute force

Append the new interval to the list, then run the standard merge-intervals routine until no overlaps remain. Simple and reuses merge logic, but touches every interval twice.

Complexity: O(n log n) time if merge sorts, O(n) space.

```
int[][] insertAndMerge(int[][] intervals, int[] ni) {
    List<int[]> list = new ArrayList<>(Arrays.asList(intervals));
    list.add(ni);
    list.sort(Comparator.comparingInt(a -> a[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] iv : list) {
        if (out.isEmpty() || out.get(out.size() - 1)[1] < iv[0]) out.add(iv);
        else out.get(out.size() - 1)[1] = Math.max(out.get(out.size() - 1)[1], iv[1]);
    }
    return out.toArray(new int[out.size()][]);
}
```

Optimal

One scan: add intervals that end before the new start; merge overlapping intervals into the new interval by expanding its end; append the rest unchanged. No full sort because existing intervals are already sorted and disjoint.

Complexity: O(n) time, O(n) space for output.

```
int[][] insert(int[][] intervals, int[] ni) {
    List<int[]> out = new ArrayList<>();
    int i = 0, n = intervals.length;
    while (i < n && intervals[i][1] < ni[0]) out.add(intervals[i++]);
    while (i < n && intervals[i][0] <= ni[1]) {
        ni[0] = Math.min(ni[0], intervals[i][0]);
        ni[1] = Math.max(ni[1], intervals[i][1]);
        i++;
    }
```

```
    out.add(ni);
    while (i < n) out.add(intervals[i++]);
    return out.toArray(new int[out.size()][]);
}
```

106. Merge Intervals

Brute force

Repeatedly scan for any pair of overlapping intervals, merge them into one, and restart until a full pass finds no overlap. Correct but each merge can shift many intervals.

Complexity: O(n^3) time worst case, O(n) space.

```
int[][] mergeBrute(int[][] intervals) {
    List<int[]> list = new ArrayList<>();
    for (int[] iv : intervals) list.add(new int[]{iv[0], iv[1]});
    boolean merged;
    do {
        merged = false;
        for (int i = 0; i < list.size(); i++) {
            for (int j = i + 1; j < list.size(); j++) {
                int[] a = list.get(i), b = list.get(j);
                if (a[0] <= b[1] && b[0] <= a[1]) {
                    int[] m = new int[]{Math.min(a[0], b[0]), Math.max(a[1], b[1])};
                    list.remove(j);
                    list.remove(i);
                    list.add(m);
                    merged = true;
                    break;
                }
            }
            if (merged) break;
        }
    } while (merged);
    return list.toArray(new int[list.size()][]);
}
```

Optimal

Sort by start time. The current merged interval extends its end when the next interval starts on or before that end; otherwise push the current interval and start a new one. After sorting, one left-to-right pass is enough.

Complexity: O(n log n) time, O(n) space for output.

```
int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (x, y) -> Integer.compare(x[0], y[0]));
    List<int[]> out = new ArrayList<>();
    int[] cur = intervals[0];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] <= cur[1]) cur[1] = Math.max(cur[1], intervals[i][1]);
        else { out.add(cur); cur = intervals[i]; }
    }
    out.add(cur);
    return out.toArray(new int[out.size()][]);
}
```

Heaps

A heap keeps the best candidate at the root for streaming top-K, merging sorted streams, and scheduling by frequency. Min-heaps track the k largest; max-heaps track the k smallest.

107. Kth Largest Element in an Array

Better

Sort descending and return the element at index k - 1. Simple and fast when k is large relative to n, but sorts the whole array.

Complexity: O(n log n) time, O(1) extra space if sorting in place.

```
int findKthLargestSort(int[] nums, int k) {
    Arrays.sort(nums);
    return nums[nums.length - k];
}
```

Optimal

Maintain a min-heap of size k. Each new value evicts the smallest among the k when it is larger. After one pass, the root is the kth largest. Memory stays O(k) instead of sorting all n.

Complexity: O(n log k) time, O(k) space.

```
int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int x : nums) {
        pq.offer(x);
        if (pq.size() > k) pq.poll();
    }
    return pq.peek();
}
```

108. Kth Smallest Element in an Array

Better

Sort ascending and read index k - 1. Same trade-off as sorting for the kth largest.

Complexity: O(n log n) time, O(1) extra space if sorting in place.

```
int findKthSmallestSort(int[] nums, int k) {
    Arrays.sort(nums);
    return nums[k - 1];
}
```

Optimal

Keep a max-heap of size k. When a new value is smaller than the heap maximum, replace the max. The root after scanning is the kth smallest.

Complexity: O(n log k) time, O(k) space.

```
int findKthSmallest(int[] nums, int k) {
    PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
    for (int x : nums) {
        pq.offer(x);
        if (pq.size() > k) pq.poll();
    }
    return pq.peek();
}
```

109. Top K Frequent Elements

Better

Count frequencies in a hash map, copy entries to a list, and sort by frequency descending. Take the first k keys.

Complexity: O(n + u log u) time where u is distinct keys, O(u) space.

```
int[] topKFrequentSort(int[] nums, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int x : nums) cnt.put(x, cnt.getOrDefault(x, 0) + 1);
```

```
    List<Integer> keys = new ArrayList<>(cnt.keySet());
    keys.sort((a, b) -> cnt.get(b) - cnt.get(a));
    int[] ans = new int[k];
    for (int i = 0; i < k; i++) ans[i] = keys.get(i);
    return ans;
}
```

Optimal

After counting, maintain a min-heap of size k ordered by frequency. Each entry beats the weakest frequent element in the heap. The heap holds the k most frequent when the scan finishes.

Complexity: O(n + u log k) time, O(u) space.

```
int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int x : nums) cnt.put(x, cnt.getOrDefault(x, 0) + 1);
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    for (var e : cnt.entrySet()) {
        pq.offer(new int[]{e.getKey(), e.getValue()});
        if (pq.size() > k) pq.poll();
    }
    int[] ans = new int[k];
    for (int i = k - 1; i >= 0; i--) ans[i] = pq.poll()[0];
    return ans;
}
```

110. Merge K Sorted Lists

Brute force

Merge lists two at a time: merge the first two, then merge that result with the third, and so on. Early merges produce long lists that are scanned again and again.

Complexity: O(kN) time where N is total nodes, O(1) extra besides output.

```
ListNode mergeKListsBrute(ListNode[] lists) {
    ListNode head = null;
    for (ListNode node : lists)
        head = mergeTwoLists(head, node);
    return head;
}

ListNode mergeTwoLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), cur = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { cur.next = a; a = a.next; }
        else { cur.next = b; b = b.next; }
        cur = cur.next;
    }
    cur.next = a != null ? a : b;
    return dummy.next;
}
```

Optimal

Push the head of every non-empty list into a min-heap keyed by node value. Pop the smallest, attach it to the answer, and push its next if any. Always pick the global minimum among list fronts.

Complexity: O(N log k) time, O(k) heap space.

```
ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode n : lists) if (n != null) pq.offer(n);
    ListNode dummy = new ListNode(0), cur = dummy;
    while (!pq.isEmpty()) {
        ListNode node = pq.poll();
        cur.next = node;
        cur = cur.next;
        if (node.next != null) pq.offer(node.next);
    }
```

```
    return dummy.next;
}
```

111. Merge K Sorted Arrays

Brute force

Concatenate all arrays and sort the combined result. Correct and easy, but ignores the fact that each array is already sorted.

Complexity: O(N log N) time for N total elements, O(N) space.

```
int[] mergeKSortedBrute(int[][] arrays) {
    int total = 0;
    for (int[] a : arrays) total += a.length;
    int[] all = new int[total];
    int p = 0;
    for (int[] a : arrays) for (int x : a) all[p++] = x;
    Arrays.sort(all);
    return all;
}
```

Optimal

Store triples (value, array index, position) in a min-heap. Pop the smallest value, append it, and push the next element from the same array if it exists. Same idea as merging k lists, on raw arrays.

Complexity: O(N log k) time, O(k) heap space.

```
int[] mergeKSorted(int[][] arrays) {
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    for (int i = 0; i < arrays.length; i++)
        if (arrays[i].length > 0) pq.offer(new int[]{arrays[i][0], i, 0});
    List<Integer> out = new ArrayList<>();
    while (!pq.isEmpty()) {
        int[] t = pq.poll();
        out.add(t[0]);
        int ai = t[1], pos = t[2] + 1;
        if (pos < arrays[ai].length)
            pq.offer(new int[]{arrays[ai][pos], ai, pos});
    }
    return out.stream().mapToInt(Integer::intValue).toArray();
}
```

112. Find Median from Data Stream

Brute force

Store every number in a list. On median query, sort the list and return the middle element or average of the two middles. addNum is cheap; findMedian is not.

Complexity: O(1) addNum, O(n log n) findMedian, O(n) space.

```
class MedianFinderBrute {
    List<Integer> a = new ArrayList<>();
    void addNum(int num) { a.add(num); }
    double findMedian() {
        Collections.sort(a);
        int n = a.size();
        if ((n & 1) == 1) return a.get(n / 2);
        return (a.get(n / 2 - 1) + a.get(n / 2)) / 2.0;
    }
}
```

Optimal

Two heaps: max-heap for the lower half, min-heap for the upper half. Keep sizes within one and every lower value at or below every upper value by rebalancing after each insert. The median is the top of the larger heap or the average of both tops.

Complexity: O(log n) addNum, O(1) findMedian, O(n) space.

```
class MedianFinder {
```

```
    PriorityQueue<Integer> lo = new PriorityQueue<>(Collections.reverseOrder());
    PriorityQueue<Integer> hi = new PriorityQueue<>();
    void addNum(int num) {
        lo.offer(num);
        hi.offer(lo.poll());
        if (lo.size() < hi.size()) lo.offer(hi.poll());
    }
    double findMedian() {
        return lo.size() > hi.size() ? lo.peek() : (lo.peek() + hi.peek()) / 2.0;
    }
}
```

113. K Closest Points to Origin

Better

Sort points by squared distance to the origin and take the first k. No square root is needed because squaring is monotonic for non-negative distances.

Complexity: O(n log n) time, O(1) extra besides output.

```
int[][] kClosestSort(int[][] points, int k) {
    Arrays.sort(points, (a, b) -> dist(a) - dist(b));
    int[][] ans = new int[k][];
    for (int i = 0; i < k; i++) ans[i] = points[i];
    return ans;
}

int dist(int[] p) { return p[0] * p[0] + p[1] * p[1]; }
```

Optimal

Maintain a max-heap of size k by squared distance. A new point replaces the heap top only if it is closer than the farthest of the current k. The heap then holds the k closest points.

Complexity: O(n log k) time, O(k) space.

```
int[][] kClosest(int[][] points, int k) {
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> dist(b) - dist(a));
    for (int[] p : points) {
        pq.offer(p);
        if (pq.size() > k) pq.poll();
    }
    int[][] ans = new int[k][];
    for (int i = k - 1; i >= 0; i--) ans[i] = pq.poll();
    return ans;
}

int dist(int[] p) { return p[0] * p[0] + p[1] * p[1]; }
```

114. Task Scheduler

Better

Count task frequencies and sort descending. Greedily fill slots of length n + 1 with the most frequent remaining tasks, separated by idle gaps, until all tasks are placed. Math on idle slots gives the same answer without a heap.

Complexity: O(m) time for m tasks, O(1) space for 26 letters.

```
int leastIntervalMath(char[] tasks, int n) {
    int[] cnt = new int[26];
    for (char c : tasks) cnt[c - 'A']++;
    Arrays.sort(cnt);
    int maxF = cnt[25], same = 1;
    for (int i = 24; i >= 0 && cnt[i] == maxF; i--) same++;
    int part = (maxF - 1) * (n + 1) + same;
    return Math.max(part, tasks.length);
}
```

Optimal

Max-heap of remaining counts. Each round of n + 1 slots pops up to n + 1 tasks, decrements them, and pushes back positive counts. Idle time appears when the heap is empty inside a round. Mirrors the schedule construction directly.

Complexity: O(m log 26) time, O(1) space.

```
int leastInterval(char[] tasks, int n) {
    int[] cnt = new int[26];
    for (char c : tasks) cnt[c - 'A']++;
    PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
    for (int x : cnt) if (x > 0) pq.offer(x);
    int time = 0;
    while (!pq.isEmpty()) {
        List<Integer> tmp = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
            if (pq.isEmpty()) break;
            int f = pq.poll() - 1;
            if (f > 0) tmp.add(f);
        }
        time += pq.isEmpty() ? tmp.size() : n + 1;
        for (int f : tmp) pq.offer(f);
    }
    return time;
}
```

115. Reorganize String

Brute force

Try to build the string position by position: at each index, try every character with positive count that differs from the previous character. Backtrack on failure. Correct for small alphabets only.

Complexity: O(k^n) time in worst case, O(n) space.

```
boolean reorganizeBrute(String s, StringBuilder path, int[] cnt, char prev) {
    if (path.length() == s.length()) return true;
    for (int c = 0; c < 26; c++) {
        if (cnt[c] == 0 || (char) ('a' + c) == prev) continue;
        cnt[c]--;
        path.append((char) ('a' + c));
        if (reorganizeBrute(s, path, cnt, (char) ('a' + c))) return true;
        path.deleteCharAt(path.length() - 1);
        cnt[c]++;
    }
    return false;
}
```

Optimal

Max-heap by count. Append the most frequent character that is not the same as the last placed; push the previous character back if its count remains. If the top repeats the last, the string is impossible. A letter with more than half the length cannot be placed without adjacency.

Complexity: O(n log 26) time, O(26) space.

```
String reorganizeString(String s) {
    int[] cnt = new int[26];
    for (char c : s.toCharArray()) cnt[c - 'a']++;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[1] - a[1]);
    for (int i = 0; i < 26; i++) if (cnt[i] > 0) pq.offer(new int[]{i, cnt[i]});
    StringBuilder sb = new StringBuilder();
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        if (sb.length() > 0 && sb.charAt(sb.length() - 1) == (char) ('a' + cur[0])) {
            if (pq.isEmpty()) return "";
            int[] nxt = pq.poll();
            sb.append((char) ('a' + nxt[0]));
            if (--nxt[1] > 0) pq.offer(nxt);
```

```
            pq.offer(cur);
        } else {
            sb.append((char) ('a' + cur[0]));
            if (--cur[1] > 0) pq.offer(cur);
        }
    }
    return sb.toString();
}
```

116. Kth Largest Element in a Stream

Better

Keep all numbers seen so far in a list. On each add, sort descending and return the element at index k - 1. Simple but resorts the full history every time.

Complexity: O(m log m) per add for m elements so far, O(m) space.

```
class KthLargestSort {
    List<Integer> a = new ArrayList<>();
    int k;
    KthLargestSort(int k, int[] nums) {
        this.k = k;
        for (int x : nums) a.add(x);
    }
    int add(int val) {
        a.add(val);
        Collections.sort(a, Collections.reverseOrder());
        return a.get(k - 1);
    }
}
```

Optimal

Min-heap of size k holds the k largest elements seen. add pushes the new value and drops the smallest if size exceeds k. The root is always the kth largest in the stream.

Complexity: O(log k) per add, O(k) space.

```
class KthLargest {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    int k;
    KthLargest(int k, int[] nums) {
        this.k = k;
        for (int x : nums) add(x);
    }
    int add(int val) {
        pq.offer(val);
        if (pq.size() > k) pq.poll();
        return pq.peek();
    }
}
```

Binary Trees

Most tree answers are a post-order or level-order story: combine what the left and right subtrees already know, or peel the tree ring by ring from the root.

117. Preorder Traversal

Optimal

Visit root, then left, then right. Recursion mirrors the definition: record the value, then trust the same function on left and right. An empty child is a no-op.

Complexity: O(n) time, O(h) recursion stack.

```
List<Integer> preorder(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    dfsPre(root, ans);
    return ans;
}
void dfsPre(TreeNode node, List<Integer> ans) {
    if (node == null) return;
    ans.add(node.val);
    dfsPre(node.left, ans);
    dfsPre(node.right, ans);
}
```

Better

Iterative preorder uses a stack that always holds the next node to visit. Pop, record, push right then left so left is popped first. No recursion stack overflow on deep skewed trees.

Complexity: O(n) time, O(h) explicit stack.

```
List<Integer> preorderIterative(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    Deque<TreeNode> st = new ArrayDeque<>();
    st.push(root);
    while (!st.isEmpty()) {
        TreeNode node = st.pop();
        ans.add(node.val);
        if (node.right != null) st.push(node.right);
        if (node.left != null) st.push(node.left);
    }
    return ans;
}
```

118. Inorder Traversal

Optimal

Inorder is left, root, right. On a BST this order is sorted. Recursion walks left until null, records, then walks right.

Complexity: O(n) time, O(h) stack.

```
List<Integer> inorder(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    dfsIn(root, ans);
    return ans;
}
void dfsIn(TreeNode node, List<Integer> ans) {
    if (node == null) return;
    dfsIn(node.left, ans);
    ans.add(node.val);
    dfsIn(node.right, ans);
}
```

Better

Morris inorder threads the tree: when there is no left child, visit and go right. When there is a left child, find its inorder predecessor. If its right link is null, link it to the current node and go left. If the link already points back, unlink, visit, go right. No stack and O(1) extra space aside from the output.

Complexity: O(n) time, O(1) extra space.

```
List<Integer> inorderMorris(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    TreeNode cur = root;
    while (cur != null) {
        if (cur.left == null) {
            ans.add(cur.val);
            cur = cur.right;
        } else {
            TreeNode pred = cur.left;
            while (pred.right != null && pred.right != cur) pred = pred.right;
            if (pred.right == null) {
                pred.right = cur;
                cur = cur.left;
            } else {
                pred.right = null;
                ans.add(cur.val);
                cur = cur.right;
            }
        }
    }
    return ans;
}
```

119. Postorder Traversal

Optimal

Postorder is left, right, root. Recursion must finish both subtrees before recording the node. That order is natural for problems that delete or aggregate bottom-up.

Complexity: O(n) time, O(h) stack.

```
List<Integer> postorder(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    dfsPost(root, ans);
    return ans;
}
void dfsPost(TreeNode node, List<Integer> ans) {
    if (node == null) return;
    dfsPost(node.left, ans);
    dfsPost(node.right, ans);
    ans.add(node.val);
}
```

Better

One stack variant pushes root, then repeatedly pops. If the popped node is not the stack's top left/right child in the right order, push it back and push its children so they are handled first. A last pointer tracks the last visited node to avoid revisiting.

Complexity: O(n) time, O(h) stack.

```
List<Integer> postorderIterative(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    Deque<TreeNode> st = new ArrayDeque<>();
    st.push(root);
    TreeNode last = null;
    while (!st.isEmpty()) {
        TreeNode peek = st.peek();
        if (peek.left != null && last != peek.left && last != peek.right)
            st.push(peek.left);
```

```
        else if (peek.right != null && last != peek.right)
            st.push(peek.right);
        else {
            last = st.pop();
            ans.add(last.val);
        }
    }
    return ans;
}
```

120. Level Order Traversal

Brute force

For each depth d from 0 to height, scan the whole tree and collect nodes at depth d. Correct but revisits nodes many times.

Complexity: O(n * h) time, O(h) output per level.

```
List<List<Integer>> levelOrderBrute(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    int h = height(root);
    for (int d = 0; d <= h; d++) {
        List<Integer> row = new ArrayList<>();
        collectDepth(root, d, row);
        if (!row.isEmpty()) ans.add(row);
    }
    return ans;
}
int height(TreeNode node) {
    if (node == null) return -1;
    return 1 + Math.max(height(node.left), height(node.right));
}
void collectDepth(TreeNode node, int d, List<Integer> row) {
    if (node == null) return;
    if (d == 0) row.add(node.val);
    else {
        collectDepth(node.left, d - 1, row);
        collectDepth(node.right, d - 1, row);
    }
}
```

Optimal

BFS with a queue holds the frontier of the current level. Record the queue size before dequeuing so each level is its own list. Enqueue children left to right for standard order.

Complexity: O(n) time, O(w) space where w is max width.

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    if (root == null) return ans;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int sz = q.size();
        List<Integer> row = new ArrayList<>();
        for (int i = 0; i < sz; i++) {
            TreeNode node = q.poll();
            row.add(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        ans.add(row);
    }
    return ans;
}
```

121. Maximum Depth of Binary Tree

Brute force

At every node recompute the height of left and right by full subtree walks. Each node is counted in many height calls.

Complexity: O(n^2) time on a skewed tree, O(h) stack.

```
int maxDepthBrute(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepthBrute(root.left), maxDepthBrute(root.right));
}
```

Optimal

Single post-order pass: depth of a node is one plus the max of child depths. One visit per node.

Complexity: O(n) time, O(h) stack.

```
int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

122. Diameter of Binary Tree

Brute force

For every node, compute left height and right height independently and take their sum as a candidate diameter. Heights are recomputed from scratch each time.

Complexity: O(n^2) time, O(h) stack.

```
int diameterBrute(TreeNode root) {
    if (root == null) return 0;
    int through = height(root.left) + height(root.right);
    return Math.max(through, Math.max(diameterBrute(root.left), diameterBrute(root.right)));
}
int height(TreeNode node) {
    if (node == null) return 0;
    return 1 + Math.max(height(node.left), height(node.right));
}
```

Optimal

While computing height bottom-up, the longest path through the current node is leftHeight + rightHeight. Track the global maximum during the same DFS that returns height to the parent.

Complexity: O(n) time, O(h) stack.

```
int diameterOfBinaryTree(TreeNode root) {
    int[] best = new int[1];
    heightDiameter(root, best);
    return best[0];
}
int heightDiameter(TreeNode node, int[] best) {
    if (node == null) return 0;
    int L = heightDiameter(node.left, best);
    int R = heightDiameter(node.right, best);
    best[0] = Math.max(best[0], L + R);
    return 1 + Math.max(L, R);
}
```

123. Balanced Binary Tree

Brute force

A tree is balanced when every node's left and right heights differ by at most one. Check that predicate at every node with fresh height computations.

Complexity: O(n^2) time, O(h) stack.

```
boolean isBalancedBrute(TreeNode root) {
    if (root == null) return true;
```

```
    int diff = Math.abs(height(root.left) - height(root.right));
    return diff <= 1 && isBalancedBrute(root.left) && isBalancedBrute(root.right);
}
int height(TreeNode node) {
    if (node == null) return 0;
    return 1 + Math.max(height(node.left), height(node.right));
}
```

Optimal

Return height from DFS but use -1 as a sentinel when a subtree is already unbalanced. If either child fails or the current skew exceeds one, bubble -1 up immediately.

Complexity: O(n) time, O(h) stack.

```
boolean isBalanced(TreeNode root) {
    return checkBalance(root) != -1;
}
int checkBalance(TreeNode node) {
    if (node == null) return 0;
    int L = checkBalance(node.left);
    if (L == -1) return -1;
    int R = checkBalance(node.right);
    if (R == -1) return -1;
    if (Math.abs(L - R) > 1) return -1;
    return 1 + Math.max(L, R);
}
```

124. Maximum Path Sum

Brute force

Try every node as the highest point of a path and explore all downward paths in its left and right subtrees to maximize sum. Exponential in path choices without pruning.

Complexity: O(2^n) time in worst case, O(h) stack.

```
int maxPathSumBrute(TreeNode root) {
    if (root == null) return Integer.MIN_VALUE / 4;
    int alone = root.val;
    int withLeft = root.val + bestDown(root.left);
    int withRight = root.val + bestDown(root.right);
    int through = root.val + bestDown(root.left) + bestDown(root.right);
    int bestHere = Math.max(alone, Math.max(withLeft, Math.max(withRight, through)));
    return Math.max(bestHere, Math.max(maxPathSumBrute(root.left), maxPathSumBrute(root.right)));
}
int bestDown(TreeNode node) {
    if (node == null) return 0;
    return Math.max(0, node.val + Math.max(bestDown(node.left), bestDown(node.right)));
}
```

Optimal

For each node, the best downward path through it to offer a parent is node.val plus the better nonnegative child gain. The best path that uses this node as a bend is node.val plus both child gains. Update a global answer; return only the single-arm gain to the parent.

Complexity: O(n) time, O(h) stack.

```
int maxPathSum(TreeNode root) {
    int[] best = new int[]{Integer.MIN_VALUE / 4};
    gain(root, best);
    return best[0];
}
int gain(TreeNode node, int[] best) {
    if (node == null) return 0;
    int L = Math.max(0, gain(node.left, best));
    int R = Math.max(0, gain(node.right, best));
    best[0] = Math.max(best[0], node.val + L + R);
    return node.val + Math.max(L, R);
```

```
}
```

125. Same Tree

Brute force

Serialize both trees to the same traversal string and compare. Any traversal works if null markers are explicit.

Complexity: O(n) time, O(n) space for strings.

```
boolean isSameTreeSerialize(TreeNode p, TreeNode q) {
    return serialize(p).equals(serialize(q));
}
String serialize(TreeNode node) {
    if (node == null) return "#,";
    return node.val + "," + serialize(node.left) + serialize(node.right);
}
```

Optimal

Walk both roots together. Mismatch on null pairing or value means false. Otherwise both left pairs and both right pairs must match.

Complexity: O(n) time, O(h) stack.

```
boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null || q == null) return p == q;
    return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}
```

126. Symmetric Tree

Optimal

Mirror equality: the outer pair (left.left, right.right) and inner pair (left.right, right.left) must match values and mirror structure. Base case is two null nodes.

Complexity: O(n) time, O(h) stack.

```
boolean isSymmetric(TreeNode root) {
    return root == null || mirror(root.left, root.right);
}
boolean mirror(TreeNode a, TreeNode b) {
    if (a == null || b == null) return a == b;
    return a.val == b.val && mirror(a.left, b.right) && mirror(a.right, b.left);
}
```

Better

Iterative BFS enqueues pairs that should mirror each other. Each level check compares front pairs before pushing children in mirror order.

Complexity: O(n) time, O(w) queue space.

```
boolean isSymmetricBfs(TreeNode root) {
    if (root == null) return true;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root.left);
    q.offer(root.right);
    while (!q.isEmpty()) {
        TreeNode a = q.poll(), b = q.poll();
        if (a == null && b == null) continue;
        if (a == null || b == null || a.val != b.val) return false;
        q.offer(a.left); q.offer(b.right);
        q.offer(a.right); q.offer(b.left);
    }
    return true;
}
```

127. Zigzag Level Order

Better

Standard level order, then reverse every other row in the answer list.

Complexity: O(n) time, O(w) space.

```
List<List<Integer>> zigzagLevelOrderReverse(TreeNode root) {
    List<List<Integer>> ans = levelOrder(root);
    for (int i = 1; i < ans.size(); i += 2) Collections.reverse(ans.get(i));
    return ans;
}
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    if (root == null) return ans;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int sz = q.size();
        List<Integer> row = new ArrayList<>();
        for (int i = 0; i < sz; i++) {
            TreeNode node = q.poll();
            row.add(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        ans.add(row);
    }
    return ans;
}
```

Optimal

Use a deque as the level container. Even levels append at the back; odd levels insert at the front so the row is built reversed without a second pass.

Complexity: O(n) time, O(w) space.

```
List<List<Integer>> zigzagLevelOrder(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    if (root == null) return ans;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    boolean leftToRight = true;
    while (!q.isEmpty()) {
        int sz = q.size();
        LinkedList<Integer> row = new LinkedList<>();
        for (int i = 0; i < sz; i++) {
            TreeNode node = q.poll();
            if (leftToRight) row.addLast(node.val);
            else row.addFirst(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        ans.add(row);
        leftToRight = !leftToRight;
    }
    return ans;
}
```

128. Right Side View

Brute force

For each depth, scan the tree and keep the last value seen at that depth during a preorder walk.

Complexity: O(n * h) time, O(h) stack.

```
List<Integer> rightSideViewBrute(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
```

```
    int h = maxDepth(root);
    for (int d = 0; d < h; d++) {
        int[] last = new int[]{0};
        lastAtDepth(root, 0, d, last);
        ans.add(last[0]);
    }
    return ans;
}
int maxDepth(TreeNode node) {
    if (node == null) return 0;
    return 1 + Math.max(maxDepth(node.left), maxDepth(node.right));
}
void lastAtDepth(TreeNode node, int cur, int target, int[] last) {
    if (node == null) return;
    if (cur == target) last[0] = node.val;
    lastAtDepth(node.left, cur + 1, target, last);
    lastAtDepth(node.right, cur + 1, target, last);
}
```

Optimal

BFS: the last node dequeued at each level is visible from the right. DFS preorder visiting right before left also works: the first node hit at each depth wins.

Complexity: O(n) time, O(h) stack or O(w) queue.

```
List<Integer> rightSideView(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    dfsRight(root, 0, ans);
    return ans;
}
void dfsRight(TreeNode node, int depth, List<Integer> ans) {
    if (node == null) return;
    if (depth == ans.size()) ans.add(node.val);
    dfsRight(node.right, depth + 1, ans);
    dfsRight(node.left, depth + 1, ans);
}
```

129. Boundary Traversal

Optimal

Anti-clockwise boundary: root, left boundary (exclude leaves), all leaves left to right, right boundary bottom-up (exclude leaves). Mark leaves during left/right boundary walks so they are not duplicated.

Complexity: O(n) time, O(h) stack.

```
List<Integer> boundaryTraversal(TreeNode root) {
    List<Integer> ans = new ArrayList<>();
    if (root == null) return ans;
    if (!isLeaf(root)) ans.add(root.val);
    addLeftBoundary(root.left, ans);
    addLeaves(root, ans);
    addRightBoundary(root.right, ans);
    return ans;
}
boolean isLeaf(TreeNode node) {
    return node != null && node.left == null && node.right == null;
}
void addLeftBoundary(TreeNode node, List<Integer> ans) {
    while (node != null) {
        if (!isLeaf(node)) ans.add(node.val);
        node = node.left != null ? node.left : node.right;
    }
}
void addLeaves(TreeNode node, List<Integer> ans) {
    if (node == null) return;
    if (isLeaf(node)) { ans.add(node.val); return; }
```

```
    addLeaves(node.left, ans);
    addLeaves(node.right, ans);
}
void addRightBoundary(TreeNode node, List<Integer> ans) {
    Deque<Integer> stack = new ArrayDeque<>();
    while (node != null) {
        if (!isLeaf(node)) stack.push(node.val);
        node = node.right != null ? node.right : node.left;
    }
    while (!stack.isEmpty()) ans.add(stack.pop());
}
```

130. Construct from Preorder and Inorder

Brute force

The first preorder value is the root. Find it in inorder; everything left of that index is the left subtree, everything right is the right subtree. Recurse on the corresponding subarrays. Each step scans inorder for the root value.

Complexity: O(n^2) time, O(n) space.

```
TreeNode buildTree(int[] pre, int[] in) {
    return build(pre, 0, pre.length - 1, in, 0, in.length - 1);
}
TreeNode build(int[] pre, int pl, int pr, int[] in, int il, int ir) {
    if (pl > pr) return null;
    int rootVal = pre[pl];
    int k = il;
    while (in[k] != rootVal) k++;
    int leftSize = k - il;
    TreeNode root = new TreeNode(rootVal);
    root.left = build(pre, pl + 1, pl + leftSize, in, il, k - 1);
    root.right = build(pre, pl + leftSize + 1, pr, in, k + 1, ir);
    return root;
}
```

Optimal

Hash inorder indices once. A preorder pointer picks the next root. Inorder bounds tell whether that root belongs in the current subtree. Recursion only chooses left or right child based on index comparisons.

Complexity: O(n) time, O(n) space.

```
TreeNode buildTreeOpt(int[] pre, int[] in) {
    Map<Integer, Integer> idx = new HashMap<>();
    for (int i = 0; i < in.length; i++) idx.put(in[i], i);
    int[] p = new int[]{0};
    return build(pre, idx, p, 0, in.length - 1);
}
TreeNode build(int[] pre, Map<Integer, Integer> idx, int[] p, int lo, int hi) {
    if (lo > hi) return null;
    int val = pre[p[0]++];
    TreeNode root = new TreeNode(val);
    int mid = idx.get(val);
    root.left = build(pre, idx, p, lo, mid - 1);
    root.right = build(pre, idx, p, mid + 1, hi);
    return root;
}
```

131. Lowest Common Ancestor

Brute force

Build parent pointers from the root with BFS. Walk both nodes to the root recording depth. Align depths, then step both upward until they meet.

Complexity: O(n) time, O(n) space.

```
TreeNode lowestCommonAncestorParent(TreeNode root, TreeNode p, TreeNode q) {
    Map<TreeNode, TreeNode> par = new HashMap<>();
    par.put(root, null);
```

```
    Queue<TreeNode> qu = new ArrayDeque<>();
    qu.offer(root);
    while (!qu.isEmpty()) {
        TreeNode node = qu.poll();
        if (node.left != null) { par.put(node.left, node); qu.offer(node.left); }
        if (node.right != null) { par.put(node.right, node); qu.offer(node.right); }
    }
    Set<TreeNode> seen = new HashSet<>();
    for (TreeNode cur = p; cur != null; cur = par.get(cur)) seen.add(cur);
    for (TreeNode cur = q; cur != null; cur = par.get(cur))
        if (seen.contains(cur)) return cur;
    return null;
}
```

Optimal

Post-order: if the current node is p or q, return it. If both subtrees return non-null, current is the LCA. If only one side returns non-null, bubble that answer up.

Complexity: O(n) time, O(h) stack.

```
TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode L = lowestCommonAncestor(root.left, p, q);
    TreeNode R = lowestCommonAncestor(root.right, p, q);
    if (L != null && R != null) return root;
    return L != null ? L : R;
}
```

Binary Search Trees

BST order is a one-way rule: everything left is smaller, everything right is larger. That lets search, insert, and delete walk one path; inorder is sorted.

132. Search in BST

Brute force

Inorder the tree into a list and binary search the target. Correct but ignores the BST shape.

Complexity: O(n) time, O(n) space.

```
boolean searchBSTBrute(TreeNode root, int target) {
    List<Integer> vals = new ArrayList<>();
    inorder(root, vals);
    int lo = 0, hi = vals.size() - 1;
    while (lo <= hi) {
        int mid = (lo + hi) >>> 1;
        if (vals.get(mid) == target) return true;
        if (vals.get(mid) < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return false;
}
void inorder(TreeNode node, List<Integer> vals) {
    if (node == null) return;
    inorder(node.left, vals);
    vals.add(node.val);
    inorder(node.right, vals);
}
```

Optimal

Compare target with the current value. Equal means found. Smaller means recurse left; larger means recurse right. Null means absent.

Complexity: O(h) time, O(h) stack.

```
TreeNode searchBST(TreeNode root, int val) {
    if (root == null || root.val == val) return root;
    return val < root.val ? searchBST(root.left, val) : searchBST(root.right, val);
}
```

133. Minimum Element in BST

Brute force

Track the minimum while visiting every node in any traversal.

Complexity: O(n) time, O(h) stack.

```
int minValueBrute(TreeNode root) {
    if (root == null) throw new IllegalArgumentException();
    int[] mn = new int[]{Integer.MAX_VALUE};
    visit(root, mn);
    return mn[0];
}
void visit(TreeNode node, int[] mn) {
    if (node == null) return;
    mn[0] = Math.min(mn[0], node.val);
    visit(node.left, mn);
    visit(node.right, mn);
}
```

Optimal

The smallest value sits at the leftmost chain. Walk left until null.

Complexity: O(h) time, O(1) space.

```
int minValue(TreeNode root) {
    while (root.left != null) root = root.left;
```

```
    return root.val;
}
```

134. Insert into BST

Brute force

Inorder to a sorted list, insert the value, rebuild a balanced BST from the array. Works but destroys the original structure.

Complexity: O(n) time, O(n) space.

```
TreeNode insertIntoBSTBrute(TreeNode root, int val) {
    List<Integer> vals = new ArrayList<>();
    inorder(root, vals);
    int i = 0;
    while (i < vals.size() && vals.get(i) < val) i++;
    vals.add(i, val);
    return sortedArrayToBST(vals, 0, vals.size() - 1);
}
void inorder(TreeNode node, List<Integer> vals) {
    if (node == null) return;
    inorder(node.left, vals);
    vals.add(node.val);
    inorder(node.right, vals);
}
TreeNode sortedArrayToBST(List<Integer> a, int lo, int hi) {
    if (lo > hi) return null;
    int mid = (lo + hi) >>> 1;
    TreeNode node = new TreeNode(a.get(mid));
    node.left = sortedArrayToBST(a, lo, mid - 1);
    node.right = sortedArrayToBST(a, mid + 1, hi);
    return node;
}
```

Optimal

Walk until null: if val is smaller go left, else go right. Attach a new leaf at the empty child pointer.

Complexity: O(h) time, O(h) stack or O(1) iterative.

```
TreeNode insertIntoBST(TreeNode root, int val) {
    if (root == null) return new TreeNode(val);
    TreeNode cur = root;
    while (true) {
        if (val < cur.val) {
            if (cur.left == null) { cur.left = new TreeNode(val); break; }
            cur = cur.left;
        } else {
            if (cur.right == null) { cur.right = new TreeNode(val); break; }
            cur = cur.right;
        }
    }
    return root;
}
```

135. Delete Node in BST

Optimal

Find the node. No left child: return right. No right child: return left. Two children: replace value with inorder successor from the right subtree's leftmost node, then delete that successor value from the right subtree.

Complexity: O(h) time, O(h) stack.

```
TreeNode deleteNode(TreeNode root, int key) {
    if (root == null) return null;
    if (key < root.val) root.left = deleteNode(root.left, key);
    else if (key > root.val) root.right = deleteNode(root.right, key);
    else {
        if (root.left == null) return root.right;
```

```
        if (root.right == null) return root.left;
        TreeNode succ = root.right;
        while (succ.left != null) succ = succ.left;
        root.val = succ.val;
        root.right = deleteNode(root.right, succ.val);
    }
    return root;
}
```

136. Validate BST

Brute force

Inorder must be strictly increasing. Copy values to a list and check neighbors.

Complexity: O(n) time, O(n) space.

```
boolean isValidBSTBrute(TreeNode root) {
    List<Integer> vals = new ArrayList<>();
    inorder(root, vals);
    for (int i = 1; i < vals.size(); i++)
        if (vals.get(i) <= vals.get(i - 1)) return false;
    return true;
}
void inorder(TreeNode node, List<Integer> vals) {
    if (node == null) return;
    inorder(node.left, vals);
    vals.add(node.val);
    inorder(node.right, vals);
}
```

Optimal

Pass down an open interval (lo, hi) each node must satisfy. Left child tightens the upper bound; right child tightens the lower bound. Null is valid.

Complexity: O(n) time, O(h) stack.

```
boolean isValidBST(TreeNode root) {
    return valid(root, null, null);
}
boolean valid(TreeNode node, Integer lo, Integer hi) {
    if (node == null) return true;
    if (lo != null && node.val <= lo) return false;
    if (hi != null && node.val >= hi) return false;
    return valid(node.left, lo, node.val) && valid(node.right, node.val, hi);
}
```

137. LCA in BST

Brute force

Use the general binary-tree LCA with parent maps or post-order. Ignores BST ordering.

Complexity: O(n) time, O(n) space.

```
TreeNode lowestCommonAncestorBSTBrute(TreeNode root, TreeNode p, TreeNode q) {
    Map<TreeNode, TreeNode> par = new HashMap<>();
    par.put(root, null);
    Queue<TreeNode> qu = new ArrayDeque<>();
    qu.offer(root);
    while (!qu.isEmpty()) {
        TreeNode node = qu.poll();
        if (node.left != null) { par.put(node.left, node); qu.offer(node.left); }
        if (node.right != null) { par.put(node.right, node); qu.offer(node.right); }
    }
    Set<TreeNode> seen = new HashSet<>();
    for (TreeNode cur = p; cur != null; cur = par.get(cur)) seen.add(cur);
    for (TreeNode cur = q; cur != null; cur = par.get(cur))
        if (seen.contains(cur)) return cur;
    return null;
```

```
}
```

Optimal

If both targets are smaller, LCA is in the left subtree. If both larger, LCA is in the right. Otherwise the split happens at the current node, which is the LCA.

Complexity: O(h) time, O(1) space.

```
TreeNode lowestCommonAncestorBST(TreeNode root, TreeNode p, TreeNode q) {
    int a = p.val, b = q.val;
    while (root != null) {
        if (a < root.val && b < root.val) root = root.left;
        else if (a > root.val && b > root.val) root = root.right;
        else return root;
    }
    return null;
}
```

138. Kth Smallest in BST

Better

Inorder traversal stops after k nodes. Iterative stack mimics recursion.

Complexity: O(h + k) time, O(h) stack.

```
int kthSmallestIterative(TreeNode root, int k) {
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;
    while (true) {
        while (cur != null) { st.push(cur); cur = cur.left; }
        cur = st.pop();
        if (--k == 0) return cur.val;
        cur = cur.right;
    }
}
```

Optimal

Augment each node with the size of its left subtree. If leftSize + 1 equals k, current node is the answer. If k is smaller, go left with the same k. Otherwise go right with k reduced by leftSize + 1.

Complexity: O(h) time per query after O(n) augmentation, O(h) space.

```
int kthSmallest(TreeNode root, int k) {
    while (root != null) {
        int left = size(root.left);
        if (k == left + 1) return root.val;
        if (k <= left) root = root.left;
        else { k -= left + 1; root = root.right; }
    }
    return -1;
}
int size(TreeNode node) {
    if (node == null) return 0;
    return 1 + size(node.left) + size(node.right);
}
```

139. Merge Two BSTs

Brute force

Inorder both trees into lists, merge the sorted lists, rebuild a balanced BST from the merged array.

Complexity: O(m + n) time, O(m + n) space.

```
TreeNode mergeBSTsBrute(TreeNode root1, TreeNode root2) {
    List<Integer> a = new ArrayList<>(), b = new ArrayList<>();
    inorder(root1, a);
    inorder(root2, b);
    List<Integer> all = merge(a, b);
    return build(all, 0, all.size() - 1);
```

```
}
void inorder(TreeNode node, List<Integer> out) {
    if (node == null) return;
    inorder(node.left, out);
    out.add(node.val);
    inorder(node.right, out);
}
List<Integer> merge(List<Integer> a, List<Integer> b) {
    List<Integer> res = new ArrayList<>();
    int i = 0, j = 0;
    while (i < a.size() && j < b.size())
        res.add(a.get(i) <= b.get(j) ? a.get(i++) : b.get(j++));
    while (i < a.size()) res.add(a.get(i++));
    while (j < b.size()) res.add(b.get(j++));
    return res;
}
TreeNode build(List<Integer> v, int lo, int hi) {
    if (lo > hi) return null;
    int mid = (lo + hi) >>> 1;
    TreeNode node = new TreeNode(v.get(mid));
    node.left = build(v, lo, mid - 1);
    node.right = build(v, mid + 1, hi);
    return node;
}
```

Optimal

Inorder both with two stacks (iterative). Always take the smaller stack top value, advance that tree's inorder step, append to a list or link nodes in place. One linear merge pass.

Complexity: O(m + n) time, O(h1 + h2) stack.

```
List<Integer> mergeBSTInorder(TreeNode root1, TreeNode root2) {
    List<Integer> ans = new ArrayList<>();
    Deque<TreeNode> s1 = new ArrayDeque<>(), s2 = new ArrayDeque<>();
    pushLeft(root1, s1);
    pushLeft(root2, s2);
    while (!s1.isEmpty() || !s2.isEmpty()) {
        if (s2.isEmpty() || (!s1.isEmpty() && s1.peek().val <= s2.peek().val)) {
            TreeNode node = s1.pop();
            ans.add(node.val);
            pushLeft(node.right, s1);
        } else {
            TreeNode node = s2.pop();
            ans.add(node.val);
            pushLeft(node.right, s2);
        }
    }
    return ans;
}
void pushLeft(TreeNode node, Deque<TreeNode> st) {
    while (node != null) { st.push(node); node = node.left; }
}
```

140. Two Sum in BST

Brute force

Inorder to a sorted list, two pointers from both ends like two-sum in a sorted array.

Complexity: O(n) time, O(n) space.

```
boolean findTargetBrute(TreeNode root, int k) {
    List<Integer> vals = new ArrayList<>();
    inorder(root, vals);
    int lo = 0, hi = vals.size() - 1;
    while (lo < hi) {
        int sum = vals.get(lo) + vals.get(hi);
        if (sum == k) return true;
```

```
        if (sum < k) lo++;
        else hi--;
    }
    return false;
}
void inorder(TreeNode node, List<Integer> vals) {
    if (node == null) return;
    inorder(node.left, vals);
    vals.add(node.val);
    inorder(node.right, vals);
}
```

Optimal

Hash set while DFS: if k - node.val is already seen, pair exists. Otherwise insert node.val and continue.

Complexity: O(n) time, O(n) space.

```
boolean findTarget(TreeNode root, int k) {
    Set<Integer> seen = new HashSet<>();
    return dfsTwoSum(root, k, seen);
}
boolean dfsTwoSum(TreeNode node, int k, Set<Integer> seen) {
    if (node == null) return false;
    if (seen.contains(k - node.val)) return true;
    seen.add(node.val);
    return dfsTwoSum(node.left, k, seen) || dfsTwoSum(node.right, k, seen);
}
```

141. Recover BST

Brute force

Inorder to a list, find the two indices where order breaks, swap those values in the list, write back with another inorder walk. Needs parent pointers or rebuilding.

Complexity: O(n) time, O(n) space.

```
void recoverTreeBrute(TreeNode root) {
    List<TreeNode> nodes = new ArrayList<>();
    List<Integer> vals = new ArrayList<>();
    inorderCollect(root, nodes, vals);
    int i = 0, j = vals.size() - 1;
    while (vals.get(i) < vals.get(i + 1)) i++;
    while (vals.get(j) > vals.get(j - 1)) j--;
    int tmp = vals.get(i);
    vals.set(i, vals.get(j));
    vals.set(j, tmp);
    assignInorder(root, vals, new int[]{0});
}
void inorderCollect(TreeNode node, List<TreeNode> nodes, List<Integer> vals) {
    if (node == null) return;
    inorderCollect(node.left, nodes, vals);
    nodes.add(node);
    vals.add(node.val);
    inorderCollect(node.right, nodes, vals);
}
void assignInorder(TreeNode node, List<Integer> vals, int[] p) {
    if (node == null) return;
    assignInorder(node.left, vals, p);
    node.val = vals.get(p[0]++);
    assignInorder(node.right, vals, p);
}
```

Optimal

Inorder with three pointers: first and second swapped nodes, plus previous. Whenever current value is less than previous, the previous is a swap candidate. First time set first to previous; second time set second to current. Swap their values.

Complexity: O(n) time, O(h) stack.

```
void recoverTree(TreeNode root) {
    TreeNode[] first = new TreeNode[1], second = new TreeNode[1], prev = new TreeNode[0];
    inorderFix(root, prev, first, second);
    int tmp = first[0].val;
    first[0].val = second[0].val;
    second[0].val = tmp;
}
void inorderFix(TreeNode node, TreeNode[] prev, TreeNode[] first, TreeNode[] second) {
    if (node == null) return;
    inorderFix(node.left, prev, first, second);
    if (prev[0] != null && node.val < prev[0].val) {
        if (first[0] == null) first[0] = prev[0];
        second[0] = node;
    }
    prev[0] = node;
    inorderFix(node.right, prev, first, second);
}
```

Graphs

Pick a representation, then a traversal. BFS for unweighted shortest paths and layers; DFS for structure, cycles, and topo. Weighted edges bring Dijkstra, Bellman-Ford, or Floyd; spanning trees bring Prim and Kruskal.

142. Graph BFS Traversal

Optimal

Queue holds the frontier. Dequeue a node, append to order, enqueue unvisited neighbors. Mark visited when enqueued so nothing enters twice.

Complexity: O(V + E) time, O(V) space.

```
List<Integer> bfs(int n, List<List<Integer>> adj, int start) {
    boolean[] vis = new boolean[n];
    List<Integer> order = new ArrayList<>();
    Queue<Integer> q = new ArrayDeque<>();
    q.offer(start);
    vis[start] = true;
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : adj.get(u))
            if (!vis[v]) { vis[v] = true; q.offer(v); }
    }
    return order;
}
```

143. Graph DFS Traversal

Optimal

Recursive DFS marks a node, records it, then visits each unvisited neighbor. Iterative version pushes neighbors on a stack in reverse order for the same order.

Complexity: O(V + E) time, O(V) space.

```
List<Integer> dfs(int n, List<List<Integer>> adj, int start) {
    boolean[] vis = new boolean[n];
    List<Integer> order = new ArrayList<>();
    dfsVisit(start, adj, vis, order);
    return order;
}
void dfsVisit(int u, List<List<Integer>> adj, boolean[] vis, List<Integer> order) {
    vis[u] = true;
    order.add(u);
    for (int v : adj.get(u))
        if (!vis[v]) dfsVisit(v, adj, vis, order);
}
```

144. Number of Islands

Brute force

For every land cell not yet counted, flood-fill the whole component and increment the count. Each cell is visited once across all floods.

Complexity: O(m * n) time, O(m * n) stack in worst case.

```
int numIslands(char[][] grid) {
    int rows = grid.length, cols = grid[0].length, count = 0;
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            if (grid[r][c] == '1') {
                count++;
                sink(grid, r, c);
            }
    return count;
}
void sink(char[][] grid, int r, int c) {
```

```
    int rows = grid.length, cols = grid[0].length;
    if (r < 0 || c < 0 || r >= rows || c >= cols || grid[r][c] == '0') return;
    grid[r][c] = '0';
    sink(grid, r + 1, c);
    sink(grid, r - 1, c);
    sink(grid, r, c + 1);
    sink(grid, r, c - 1);
}
```

Optimal

Same flood fill but iterative BFS avoids deep recursion on large grids. Mark visited by flipping '1' to '0'.

Complexity: O(m * n) time, O(min(m, n)) queue.

```
int numIslandsBfs(char[][] grid) {
    int rows = grid.length, cols = grid[0].length, count = 0;
    int[] dr = {1, -1, 0, 0}, dc = {0, 0, 1, -1};
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            if (grid[r][c] == '1') {
                count++;
                Queue<int[]> q = new ArrayDeque<>();
                q.offer(new int[]{r, c});
                grid[r][c] = '0';
                while (!q.isEmpty()) {
                    int[] cell = q.poll();
                    for (int k = 0; k < 4; k++) {
                        int nr = cell[0] + dr[k], nc = cell[1] + dc[k];
                        if (nr >= 0 && nc >= 0 && nr < rows && nc < cols && grid[nr][nc] == '1') {
                            grid[nr][nc] = '0';
                            q.offer(new int[]{nr, nc});
                        }
                    }
                }
            }
    return count;
}
```

145. Flood Fill

Optimal

DFS or BFS from (sr, sc) while the cell still matches the original color. Paint to newColor before expanding so the same cell is not reprocessed.

Complexity: O(m * n) time, O(m * n) stack or queue.

```
int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
    int old = image[sr][sc];
    if (old == newColor) return image;
    dfsFill(image, sr, sc, old, newColor);
    return image;
}
void dfsFill(int[][] img, int r, int c, int old, int neu) {
    int rows = img.length, cols = img[0].length;
    if (r < 0 || c < 0 || r >= rows || c >= cols || img[r][c] != old) return;
    img[r][c] = neu;
    dfsFill(img, r + 1, c, old, neu);
    dfsFill(img, r - 1, c, old, neu);
    dfsFill(img, r, c + 1, old, neu);
    dfsFill(img, r, c - 1, old, neu);
}
```

146. Rotten Oranges

Optimal

Multi-source BFS: enqueue all rotten oranges at time 0. Each wave infects fresh neighbors. Track minutes as BFS depth. If any fresh remains, return -1.

Complexity: O(m * n) time, O(m * n) queue.

```
int orangesRotting(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    Queue<int[]> q = new ArrayDeque<>();
    int fresh = 0;
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) q.offer(new int[]{r, c, 0});
            else if (grid[r][c] == 1) fresh++;
        }
    int[] dr = {1, -1, 0, 0}, dc = {0, 0, 1, -1};
    int ans = 0;
    while (!q.isEmpty()) {
        int[] cell = q.poll();
        ans = cell[2];
        for (int k = 0; k < 4; k++) {
            int nr = cell[0] + dr[k], nc = cell[1] + dc[k];
            if (nr >= 0 && nc >= 0 && nr < rows && nc < cols && grid[nr][nc] == 1) {
                grid[nr][nc] = 2;
                fresh--;
                q.offer(new int[]{nr, nc, cell[2] + 1});
            }
        }
    }
    return fresh == 0 ? ans : -1;
}
```

147. Cycle in Undirected Graph (DFS)

Optimal

DFS with a parent pointer. An back edge to an already visited neighbor that is not the parent means a cycle.

Complexity: O(V + E) time, O(V) space.

```
boolean hasCycleUndirected(int n, List<List<Integer>> adj) {
    boolean[] vis = new boolean[n];
    for (int i = 0; i < n; i++)
        if (!vis[i] && dfsCycle(i, -1, adj, vis)) return true;
    return false;
}
boolean dfsCycle(int u, int par, List<List<Integer>> adj, boolean[] vis) {
    vis[u] = true;
    for (int v : adj.get(u)) {
        if (!vis[v]) {
            if (dfsCycle(v, u, adj, vis)) return true;
        } else if (v != par) return true;
    }
    return false;
}
```

148. Cycle in Undirected Graph (DSU)

Optimal

Process edges: if find(u) == find(v) the edge closes a cycle. Otherwise union(u, v). Works for disconnected graphs too.

Complexity: O(E * alpha(V)) time, O(V) space.

```
boolean hasCycleDSU(int n, int[][] edges) {
    int[] parent = new int[n], rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
```

```
    for (int[] e : edges) {
        int a = find(e[0], parent), b = find(e[1], parent);
        if (a == b) return true;
        union(a, b, parent, rank);
    }
    return false;
}
int find(int x, int[] parent) {
    if (parent[x] != x) parent[x] = find(parent[x], parent);
    return parent[x];
}
void union(int a, int b, int[] parent, int[] rank) {
    if (rank[a] < rank[b]) { int t = a; a = b; b = t; }
    parent[b] = a;
    if (rank[a] == rank[b]) rank[a]++;
}
```

149. Cycle in Directed Graph

Optimal

Three colors: unvisited, in current recursion stack, finished. If DFS reaches an in-stack neighbor, there is a back edge and a cycle.

Complexity: O(V + E) time, O(V) space.

```
boolean hasCycleDirected(int n, List<List<Integer>> adj) {
    int[] state = new int[n];
    for (int i = 0; i < n; i++)
        if (state[i] == 0 && dfsDir(i, adj, state)) return true;
    return false;
}
boolean dfsDir(int u, List<List<Integer>> adj, int[] state) {
    state[u] = 1;
    for (int v : adj.get(u)) {
        if (state[v] == 1) return true;
        if (state[v] == 0 && dfsDir(v, adj, state)) return true;
    }
    state[u] = 2;
    return false;
}
```

150. Bipartite Graph Check

Optimal

BFS colors the start 0. Each neighbor must be the opposite color. Any same-color edge across the frontier means not bipartite. Disconnected components each get their own BFS.

Complexity: O(V + E) time, O(V) space.

```
boolean isBipartite(int n, List<List<Integer>> adj) {
    int[] color = new int[n];
    Arrays.fill(color, -1);
    for (int i = 0; i < n; i++) {
        if (color[i] != -1) continue;
        Queue<Integer> q = new ArrayDeque<>();
        q.offer(i);
        color[i] = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : adj.get(u)) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1;
                    q.offer(v);
                } else if (color[v] == color[u]) return false;
            }
        }
    }
```

```
    return true;
}
```

Better

DFS with the same coloring rule: try to paint neighbors opposite to the current node.

Complexity: O(V + E) time, O(V) space.

```
boolean isBipartiteDfs(int n, List<List<Integer>> adj) {
    int[] color = new int[n];
    Arrays.fill(color, -1);
    for (int i = 0; i < n; i++)
        if (color[i] == -1 && !paint(i, 0, adj, color)) return false;
    return true;
}
boolean paint(int u, int c, List<List<Integer>> adj, int[] color) {
    color[u] = c;
    for (int v : adj.get(u)) {
        if (color[v] == -1) {
            if (!paint(v, c ^ 1, adj, color)) return false;
        } else if (color[v] == c) return false;
    }
    return true;
}
```

151. Number of Provinces

Brute force

Floyd-Warshall to get transitive closure, then count rows that still connect. Heavy for this problem size.

Complexity: O(n^3) time, O(n^2) space.

```
int findCircleNumFloyd(int[][] isConnected) {
    int n = isConnected.length;
    boolean[][] reach = new boolean[n][n];
    for (int i = 0; i < n; i++) reach[i][i] = true;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (isConnected[i][j] == 1) reach[i][j] = true;
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);
    boolean[] vis = new boolean[n];
    int comps = 0;
    for (int i = 0; i < n; i++)
        if (!vis[i]) {
            comps++;
            for (int j = 0; j < n; j++)
                if (reach[i][j]) vis[j] = true;
        }
    return comps;
}
```

Optimal

Treat the matrix as adjacency. DFS or DSU from each unvisited city counts one province per component.

Complexity: O(n^2) time, O(n) space.

```
int findCircleNum(int[][] isConnected) {
    int n = isConnected.length;
    boolean[] vis = new boolean[n];
    int comps = 0;
    for (int i = 0; i < n; i++)
        if (!vis[i]) {
            dfsProv(i, isConnected, vis);
            comps++;
        }
```

```
    return comps;
}
void dfsProv(int u, int[][] mat, boolean[] vis) {
    vis[u] = true;
    for (int v = 0; v < mat.length; v++)
        if (mat[u][v] == 1 && !vis[v]) dfsProv(v, mat, vis);
}
```

152. Topological Sort (DFS)

Optimal

DFS finishes deeper nodes first. Push a node onto the order stack after all outgoing edges are explored. Reverse is a valid topo order.

Complexity: O(V + E) time, O(V) space.

```
List<Integer> topoSortDfs(int n, List<List<Integer>> adj) {
    boolean[] vis = new boolean[n];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (!vis[i]) dfsTopo(i, adj, vis, st);
    List<Integer> order = new ArrayList<>();
    while (!st.isEmpty()) order.add(st.pop());
    return order;
}
void dfsTopo(int u, List<List<Integer>> adj, boolean[] vis, Deque<Integer> st) {
    vis[u] = true;
    for (int v : adj.get(u))
        if (!vis[v]) dfsTopo(v, adj, vis, st);
    st.push(u);
}
```

153. Topological Sort (Kahn BFS)

Optimal

Compute indegrees. Enqueue all zero-indegree nodes. Dequeue, append, relax neighbors by decrementing indegree; enqueue when it hits zero. If fewer than V nodes appear, a cycle exists.

Complexity: O(V + E) time, O(V) space.

```
List<Integer> topoSortKahn(int n, List<List<Integer>> adj) {
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : adj.get(u)) indeg[v]++;
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (indeg[i] == 0) q.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : adj.get(u))
            if (--indeg[v] == 0) q.offer(v);
    }
    return order.size() == n ? order : new ArrayList<>();
}
```

154. Course Schedule I

Optimal

Can finish iff the prerequisite graph has no cycle. Kahn's algorithm: if topo order length equals numCourses, true; else false.

Complexity: O(V + E) time, O(V) space.

```
boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
```

```
    int[] indeg = new int[numCourses];
    for (int[] p : prerequisites) {
        adj.get(p[1]).add(p[0]);
        indeg[p[0]]++;
    }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++)
        if (indeg[i] == 0) q.offer(i);
    int seen = 0;
    while (!q.isEmpty()) {
        int u = q.poll();
        seen++;
        for (int v : adj.get(u))
            if (--indeg[v] == 0) q.offer(v);
    }
    return seen == numCourses;
}
```

155. Course Schedule II

Optimal

Same Kahn topo as Course I but return the order list. Empty list when a cycle blocks some indegrees from reaching zero.

Complexity: O(V + E) time, O(V) space.

```
int[] findOrder(int numCourses, int[][] prerequisites) {
    List<Integer> order = topoSortKahn(numCourses, buildAdj(numCourses, prerequisites));
    if (order.size() != numCourses) return new int[0];
    int[] ans = new int[numCourses];
    for (int i = 0; i < numCourses; i++) ans[i] = order.get(i);
    return ans;
}
List<List<Integer>> buildAdj(int n, int[][] pre) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (int[] p : pre) adj.get(p[1]).add(p[0]);
    return adj;
}
List<Integer> topoSortKahn(int n, List<List<Integer>> adj) {
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : adj.get(u)) indeg[v]++;
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (indeg[i] == 0) q.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : adj.get(u))
            if (--indeg[v] == 0) q.offer(v);
    }
    return order;
}
```

156. Alien Dictionary

Brute force

Compare every adjacent word pair character by character to infer letter order, then topologically sort letters with brute force removal of sources. Fragile on large inputs.

Complexity: O(C + L^2) time where C is total chars.

```
String alienOrderBrute(String[] words) {
    Map<Character, Set<Character>> before = new HashMap<>();
    Set<Character> chars = new HashSet<>();
```

```
    for (String w : words)
        for (char ch : w.toCharArray()) chars.add(ch);
    for (int i = 0; i + 1 < words.length; i++) {
        String a = words[i], b = words[i + 1];
        if (a.length() > b.length() && a.startsWith(b)) return "";
        for (int j = 0; j < Math.min(a.length(), b.length()); j++)
            if (a.charAt(j) != b.charAt(j)) {
                before.computeIfAbsent(a.charAt(j), k -> new HashSet<>()).add(b.charAt(j));
                break;
            }
    }
    StringBuilder sb = new StringBuilder();
    while (!chars.isEmpty()) {
        char pick = 0;
        outer:
        for (char c : chars) {
            for (Set<Character> s : before.values())
                if (s.contains(c)) continue outer;
            pick = c;
            break;
        }
        if (pick == 0) return "";
        sb.append(pick);
        chars.remove(pick);
        before.values().forEach(s -> s.remove(pick));
    }
    return sb.toString();
}
```

Optimal

Build edges from first mismatch of adjacent words. Validate prefix rule. Kahn on 26 letters with indegree array. Concatenate zero-indegree removals; cycle if length mismatch.

Complexity: O(C + 26) time, O(26) space.

```
String alienOrder(String[] words) {
    int[] indeg = new int[26];
    boolean[] seen = new boolean[26];
    List<Set<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < 26; i++) adj.add(new HashSet<>());
    for (String w : words)
        for (char ch : w.toCharArray()) seen[ch - 'a'] = true;
    for (int i = 0; i + 1 < words.length; i++) {
        String a = words[i], b = words[i + 1];
        if (a.length() > b.length() && a.startsWith(b)) return "";
        for (int j = 0; j < Math.min(a.length(), b.length()); j++) {
            int u = a.charAt(j) - 'a', v = b.charAt(j) - 'a';
            if (u != v) {
                if (adj.get(u).add(v)) indeg[v]++;
                break;
            }
        }
    }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < 26; i++)
        if (seen[i] && indeg[i] == 0) q.offer(i);
    StringBuilder sb = new StringBuilder();
    while (!q.isEmpty()) {
        int u = q.poll();
        sb.append((char) ('a' + u));
        for (int v : adj.get(u))
            if (--indeg[v] == 0) q.offer(v);
    }
    for (int i = 0; i < 26; i++)
        if (seen[i] && indeg[i] > 0) return "";
```

```
    return sb.toString();
}
```

157. Shortest Path in Unweighted Graph

Brute force

DFS enumerates all paths from source to target tracking length. Minimum length wins. Exponential on dense graphs.

Complexity: O(V!) worst case, O(V) stack.

```
int shortestPathBrute(int n, List<List<Integer>> adj, int src, int dst) {
    int[] best = new int[]{Integer.MAX_VALUE};
    dfsPath(src, dst, adj, new boolean[n], 0, best);
    return best[0] == Integer.MAX_VALUE ? -1 : best[0];
}
void dfsPath(int u, int dst, List<List<Integer>> adj, boolean[] vis, int len, int[] best) {
    if (u == dst) { best[0] = Math.min(best[0], len); return; }
    vis[u] = true;
    for (int v : adj.get(u))
        if (!vis[v]) dfsPath(v, dst, adj, vis, len + 1, best);
    vis[u] = false;
}
```

Optimal

BFS from source assigns distance when a node is first dequeued. First time at dst is shortest edge count in an unweighted graph.

Complexity: O(V + E) time, O(V) space.

```
int shortestPathUnweighted(int n, List<List<Integer>> adj, int src, int dst) {
    int[] dist = new int[n];
    Arrays.fill(dist, -1);
    Queue<Integer> q = new ArrayDeque<>();
    q.offer(src);
    dist[src] = 0;
    while (!q.isEmpty()) {
        int u = q.poll();
        if (u == dst) return dist[u];
        for (int v : adj.get(u))
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.offer(v);
            }
    }
    return -1;
}
```

158. Shortest Path in DAG

Optimal

Topo order guarantees edges go forward. Relax each outgoing edge once: dist[v] = min(dist[v], dist[u] + w). Initialize source 0, others INF.

Complexity: O(V + E) time, O(V) space.

```
int[] shortestPathDAG(int n, int src, List<List<int[]>> adj) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE / 4);
    dist[src] = 0;
    List<Integer> order = topoSortKahn(n, toListAdj(adj));
    for (int u : order)
        if (dist[u] < Integer.MAX_VALUE / 4)
            for (int[] e : adj.get(u)) {
                int v = e[0], w = e[1];
                dist[v] = Math.min(dist[v], dist[u] + w);
            }
    return dist;
```

```
}
List<List<Integer>> toListAdj(List<List<int[]>> adj) {
    List<List<Integer>> g = new ArrayList<>();
    for (List<int[]> row : adj) {
        List<Integer> lst = new ArrayList<>();
        for (int[] e : row) lst.add(e[0]);
        g.add(lst);
    }
    return g;
}
List<Integer> topoSortKahn(int n, List<List<Integer>> adj) {
    int[] indeg = new int[n];
    for (int u = 0; u < n; u++)
        for (int v : adj.get(u)) indeg[v]++;
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (indeg[i] == 0) q.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : adj.get(u))
            if (--indeg[v] == 0) q.offer(v);
    }
    return order;
}
```

159. Dijkstra Shortest Path

Brute force

Relax all edges V times like Bellman-Ford without early exit optimization on nonnegative weights. Simple but slower than heap Dijkstra.

Complexity: O(V * E) time, O(V) space.

```
int[] dijkstraBrute(int n, int src, List<List<int[]>> adj) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE / 4);
    dist[src] = 0;
    for (int iter = 0; iter < n - 1; iter++)
        for (int u = 0; u < n; u++)
            if (dist[u] < Integer.MAX_VALUE / 4)
                for (int[] e : adj.get(u)) {
                    int v = e[0], w = e[1];
                    dist[v] = Math.min(dist[v], dist[u] + w);
                }
    return dist;
}
```

Optimal

Min-heap stores (distance, node). Pop the smallest tentative distance; stale entries are skipped. Relax neighbors. Nonnegative weights make the first settlement final.

Complexity: O((V + E) log V) time, O(V) space.

```
int[] dijkstra(int n, int src, List<List<int[]>> adj) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE / 4);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int u = cur[0], d = cur[1];
        if (d != dist[u]) continue;
        for (int[] e : adj.get(u)) {
            int v = e[0], w = e[1];
```

```
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[]{v, dist[v]});
            }
        }
    }
    return dist;
}
```

160. Bellman-Ford

Optimal

Relax every edge V-1 times. One more pass detects a reachable improvement, which means a negative cycle. Handles negative weights unlike Dijkstra.

Complexity: O(V * E) time, O(V) space.

```
int[] bellmanFord(int n, int src, int[][] edges) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE / 4);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++)
        for (int[] e : edges) {
            int u = e[0], v = e[1], w = e[2];
            if (dist[u] < Integer.MAX_VALUE / 4)
                dist[v] = Math.min(dist[v], dist[u] + w);
        }
    for (int[] e : edges) {
        int u = e[0], v = e[1], w = e[2];
        if (dist[u] < Integer.MAX_VALUE / 4 && dist[u] + w < dist[v])
            return new int[0];
    }
    return dist;
}
```

161. Floyd-Warshall

Brute force

Run Dijkstra from every vertex. Correct on nonnegative weights but repeats heap work V times.

Complexity: O(V * (V + E) log V) time.

```
int[][] allPairsDijkstra(int n, List<List<int[]>> adj) {
    int[][] ans = new int[n][n];
    for (int i = 0; i < n; i++) {
        int[] row = dijkstra(n, i, adj);
        ans[i] = row;
    }
    return ans;
}
int[] dijkstra(int n, int src, List<List<int[]>> adj) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE / 4);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int u = cur[0], d = cur[1];
        if (d != dist[u]) continue;
        for (int[] e : adj.get(u)) {
            int v = e[0], w = e[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[]{v, dist[v]});
            }
        }
```

```
    }
    return dist;
}
```

Optimal

Dynamic programming on intermediate nodes: dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]) for each k. Works with negative edges if no negative cycles.

Complexity: O(V^3) time, O(V^2) space.

```
int[][] floydWarshall(int n, int[][] w) {
    int[][] dist = new int[n][n];
    for (int i = 0; i < n; i++)
        dist[i] = Arrays.copyOf(w[i], n);
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] < Integer.MAX_VALUE / 4 && dist[k][j] < Integer.MAX_VALUE / 4)
                    dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
    return dist;
}
```

162. Minimum Spanning Tree (Prim)

Optimal

Grow a tree from start node 0. Min-heap holds cross edges (weight, to). Pop cheapest edge to an unvisited vertex, add its outgoing edges. Stop after V-1 edges or when heap empties.

Complexity: O((V + E) log V) time, O(V + E) space.

```
int mstCostPrim(int n, List<List<int[]>> adj) {
    boolean[] inTree = new boolean[n];
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
    pq.offer(new int[]{0, 0});
    int total = 0, used = 0;
    while (!pq.isEmpty() && used < n) {
        int[] cur = pq.poll();
        int u = cur[0], w = cur[1];
        if (inTree[u]) continue;
        inTree[u] = true;
        total += w;
        used++;
        for (int[] e : adj.get(u))
            if (!inTree[e[0]]) pq.offer(new int[]{e[0], e[1]});
    }
    return total;
}
```

163. Minimum Spanning Tree (Kruskal)

Optimal

Sort edges by weight. Add an edge if its endpoints are in different DSU components. V-1 edges form the MST when the graph is connected.

Complexity: O(E log E) time, O(V) DSU space.

```
int mstCostKruskal(int n, int[][] edges) {
    Arrays.sort(edges, Comparator.comparingInt(a -> a[2]));
    int[] parent = new int[n], rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
    int total = 0, cnt = 0;
    for (int[] e : edges) {
        int a = find(e[0], parent), b = find(e[1], parent);
        if (a == b) continue;
        union(a, b, parent, rank);
        total += e[2];
        if (++cnt == n - 1) break;
    }
```

```
    return total;
}
int find(int x, int[] parent) {
    if (parent[x] != x) parent[x] = find(parent[x], parent);
    return parent[x];
}
void union(int a, int b, int[] parent, int[] rank) {
    if (rank[a] < rank[b]) { int t = a; a = b; b = t; }
    parent[b] = a;
    if (rank[a] == rank[b]) rank[a]++;
}
```

164. Accounts Merge

Brute force

Union names that share an email by comparing every pair of accounts. Quadratic in account count.

Complexity: O(A^2 * E) time, O(A * E) space.

```
List<List<String>> accountsMergeBrute(List<List<String>> accounts) {
    int n = accounts.size();
    int[] parent = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (shareEmail(accounts.get(i), accounts.get(j)))
                union(i, j, parent);
    Map<Integer, List<String>> groups = new HashMap<>();
    for (int i = 0; i < n; i++) {
        int r = find(i, parent);
        groups.computeIfAbsent(r, k -> new ArrayList<>()).addAll(accounts.get(i).subList(1, accounts.get(i).size()));
    }
    List<List<String>> ans = new ArrayList<>();
    for (var e : groups.entrySet()) {
        TreeSet<String> emails = new TreeSet<>(e.getValue());
        List<String> row = new ArrayList<>();
        row.add(accounts.get(e.getKey()).get(0));
        row.addAll(emails);
        ans.add(row);
    }
    return ans;
}
boolean shareEmail(List<String> a, List<String> b) {
    Set<String> s = new HashSet<>(a.subList(1, a.size()));
    for (int i = 1; i < b.size(); i++)
        if (s.contains(b.get(i))) return true;
    return false;
}
int find(int x, int[] parent) {
    return parent[x] == x ? x : (parent[x] = find(parent[x], parent));
}
void union(int a, int b, int[] parent) {
    parent[find(b, parent)] = find(a, parent);
}
```

Optimal

DSU on emails: map each email to an index, union first email of each account with every other email in that account. Then bucket emails by root and sort.

Complexity: O(N * K log K) where N accounts and K emails per account, near O(E alpha(N)).

```
List<List<String>> accountsMerge(List<List<String>> accounts) {
    Map<String, String> owner = new HashMap<>();
    Map<String, String> parent = new HashMap<>();
    for (List<String> acc : accounts) {
        String name = acc.get(0);
        for (int i = 1; i < acc.size(); i++) {
```

```
            owner.putIfAbsent(acc.get(i), name);
            parent.putIfAbsent(acc.get(i), acc.get(i));
        }
        for (int i = 2; i < acc.size(); i++) unionEmail(acc.get(1), acc.get(i), parent);
    }
    Map<String, TreeSet<String>> groups = new HashMap<>();
    for (String email : parent.keySet()) {
        String root = findEmail(email, parent);
        groups.computeIfAbsent(root, k -> new TreeSet<>()).add(email);
    }
    List<List<String>> ans = new ArrayList<>();
    for (var e : groups.entrySet()) {
        List<String> row = new ArrayList<>();
        row.add(owner.get(e.getKey()));
        row.addAll(e.getValue());
        ans.add(row);
    }
    return ans;
}
String findEmail(String x, Map<String, String> parent) {
    if (!parent.get(x).equals(x)) parent.put(x, findEmail(parent.get(x), parent));
    return parent.get(x);
}
void unionEmail(String a, String b, Map<String, String> parent) {
    parent.put(findEmail(b, parent), findEmail(a, parent));
}
```

165. Kosaraju Strongly Connected Components

Optimal

First DFS on the graph fills finish order. Reverse all edges. DFS again in reverse finish order; each DFS tree in pass two is one SCC.

Complexity: O(V + E) time, O(V + E) space.

```
List<List<Integer>> kosaraju(int n, List<List<Integer>> adj) {
    boolean[] vis = new boolean[n];
    Deque<Integer> order = new ArrayDeque<>();
    for (int i = 0; i < n; i++)
        if (!vis[i]) dfs1(i, adj, vis, order);
    List<List<Integer>> rev = reverseGraph(n, adj);
    Arrays.fill(vis, false);
    List<List<Integer>> sccs = new ArrayList<>();
    while (!order.isEmpty()) {
        int u = order.pop();
        if (vis[u]) continue;
        List<Integer> comp = new ArrayList<>();
        dfs2(u, rev, vis, comp);
        sccs.add(comp);
    }
    return sccs;
}
void dfs1(int u, List<List<Integer>> adj, boolean[] vis, Deque<Integer> order) {
    vis[u] = true;
    for (int v : adj.get(u))
        if (!vis[v]) dfs1(v, adj, vis, order);
    order.push(u);
}
void dfs2(int u, List<List<Integer>> rev, boolean[] vis, List<Integer> comp) {
    vis[u] = true;
    comp.add(u);
    for (int v : rev.get(u))
        if (!vis[v]) dfs2(v, rev, vis, comp);
}
List<List<Integer>> reverseGraph(int n, List<List<Integer>> adj) {
    List<List<Integer>> rev = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++) rev.add(new ArrayList<>());
    for (int u = 0; u < n; u++)
        for (int v : adj.get(u)) rev.get(v).add(u);
    return rev;
}
```

166. Bridges in Graph

Optimal

Tarjan DFS tracks discovery time and low-link. Tree edge to v updates low from subtree. Back edge updates low from discovery time of v. If low[v] > disc[u], edge u-v is a bridge.

Complexity: O(V + E) time, O(V) space.

```
List<int[]> findBridges(int n, List<List<Integer>> adj) {
    int[] disc = new int[n], low = new int[n];
    Arrays.fill(disc, -1);
    int[] timer = new int[]{0};
    List<int[]> bridges = new ArrayList<>();
    for (int i = 0; i < n; i++)
        if (disc[i] == -1) dfsBridge(i, -1, adj, disc, low, timer, bridges);
    return bridges;
}
void dfsBridge(int u, int par, List<List<Integer>> adj, int[] disc, int[] low, int[] timer, List<int[]> bridges) {
    disc[u] = low[u] = timer[0]++;
    for (int v : adj.get(u)) {
        if (v == par) continue;
        if (disc[v] == -1) {
            dfsBridge(v, u, adj, disc, low, timer, bridges);
            low[u] = Math.min(low[u], low[v]);
            if (low[v] > disc[u]) bridges.add(new int[]{u, v});
        } else low[u] = Math.min(low[u], disc[v]);
    }
}
```

167. Articulation Points

Optimal

Same Tarjan low/disc. Root with two or more DFS children is articulation. Non-root u is articulation if some child v has low[v] >= disc[u].

Complexity: O(V + E) time, O(V) space.

```
List<Integer> articulationPoints(int n, List<List<Integer>> adj) {
    int[] disc = new int[n], low = new int[n];
    Arrays.fill(disc, -1);
    boolean[] isAp = new boolean[n];
    int[] timer = new int[]{0};
    for (int i = 0; i < n; i++)
        if (disc[i] == -1) dfsAp(i, -1, adj, disc, low, timer, isAp, new int[]{0});
    List<Integer> ans = new ArrayList<>();
    for (int i = 0; i < n; i++)
        if (isAp[i]) ans.add(i);
    return ans;
}
void dfsAp(int u, int par, List<List<Integer>> adj, int[] disc, int[] low, int[] timer, boolean[] isAp, int[] children) {
    disc[u] = low[u] = timer[0]++;
    int childCount = 0;
    for (int v : adj.get(u)) {
        if (v == par) continue;
        if (disc[v] == -1) {
            childCount++;
            dfsAp(v, u, adj, disc, low, timer, isAp, children);
            low[u] = Math.min(low[u], low[v]);
            if (par != -1 && low[v] >= disc[u]) isAp[u] = true;
        } else low[u] = Math.min(low[u], disc[v]);
    }
```

```
    if (par == -1 && childCount > 1) isAp[u] = true;
}
```

168. Word Ladder

Brute force

DFS from beginWord trying every single-letter change that exists in the word set. Track minimum depth when endWord is reached. Exponential without BFS layering.

Complexity: O(N * L * 26^L) worst case, O(L) stack.

```
int ladderLengthBrute(String beginWord, String endWord, List<String> wordList) {
    Set<String> dict = new HashSet<>(wordList);
    if (!dict.contains(endWord)) return 0;
    int[] best = new int[]{Integer.MAX_VALUE};
    dfsLadder(beginWord, endWord, dict, new HashSet<>(), 1, best);
    return best[0] == Integer.MAX_VALUE ? 0 : best[0];
}
void dfsLadder(String cur, String end, Set<String> dict, Set<String> used, int steps, int[] best) {
    if (cur.equals(end)) { best[0] = Math.min(best[0], steps); return; }
    if (steps >= best[0]) return;
    char[] arr = cur.toCharArray();
    for (int i = 0; i < arr.length; i++) {
        char old = arr[i];
        for (char c = 'a'; c <= 'z'; c++) {
            arr[i] = c;
            String nxt = new String(arr);
            if (dict.contains(nxt) && !used.contains(nxt)) {
                used.add(nxt);
                dfsLadder(nxt, end, dict, used, steps + 1, best);
                used.remove(nxt);
            }
        }
        arr[i] = old;
    }
}
```

Optimal

BFS on words: each level is one letter change. For each word try all 26 letters at each position. First time endWord is dequeued, return level count.

Complexity: O(N * L * 26) time, O(N) space.

```
int ladderLength(String beginWord, String endWord, List<String> wordList) {
    Set<String> dict = new HashSet<>(wordList);
    if (!dict.contains(endWord)) return 0;
    Queue<String> q = new ArrayDeque<>();
    q.offer(beginWord);
    int steps = 1;
    while (!q.isEmpty()) {
        int sz = q.size();
        for (int t = 0; t < sz; t++) {
            String word = q.poll();
            if (word.equals(endWord)) return steps;
            char[] arr = word.toCharArray();
            for (int i = 0; i < arr.length; i++) {
                char old = arr[i];
                for (char c = 'a'; c <= 'z'; c++) {
                    arr[i] = c;
                    String nxt = new String(arr);
                    if (dict.remove(nxt)) q.offer(nxt);
                }
                arr[i] = old;
            }
        }
        steps++;
    }
```

```
    return 0;
}
```

Dynamic Programming

State = what must be remembered from the prefix. Recursion defines transitions; memo or tabulation removes repeated subproblems.

169. Climbing Stairs

Brute force

From step i you may go to i+1 or i+2. The number of ways to reach n is the sum of ways from n-1 and n-2. Recurse without caching and the same (n) is recomputed many times.

Complexity: O(2^n) time, O(n) stack.

```
int climbStairs(int n) {
    if (n <= 2) return n;
    return climbStairs(n - 1) + climbStairs(n - 2);
}
```

Memoization

The recurrence is Fibonacci: dp[i] = dp[i-1] + dp[i-2]. Cache dp[n] after the first compute.

Complexity: O(n) time, O(n) space.

```
int climbStairs(int n) {
    int[] memo = new int[n + 1];
    Arrays.fill(memo, -1);
    return ways(n, memo);
}
int ways(int n, int[] memo) {
    if (n <= 2) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = ways(n - 1, memo) + ways(n - 2, memo);
}
```

Tabulation

Fill bottom-up with two rolling variables instead of an array.

Complexity: O(n) time, O(1) space.

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

170. Frog Jump (Minimum Cost)

Brute force

From index i jump to i+1 or i+2 and pay |height[i]-height[j]|. Try both jumps recursively from every start.

Complexity: O(2^n) time, O(n) stack.

```
int minCost(int[] h) {
    return cost(h, 0);
}
int cost(int[] h, int i) {
    if (i >= h.length - 1) return 0;
    int one = Math.abs(h[i] - h[i + 1]) + cost(h, i + 1);
    int two = (i + 2 < h.length)
        ? Math.abs(h[i] - h[i + 2]) + cost(h, i + 2)
        : Integer.MAX_VALUE / 2;
    return Math.min(one, two);
}
```

Memoization

State is the current index. dp[i] = min cost to reach the end from i.

Complexity: O(n) time, O(n) space.

```
int minCost(int[] h) {
    int n = h.length;
    int[] memo = new int[n];
    Arrays.fill(memo, -1);
    return dp(h, 0, memo);
}
int dp(int[] h, int i, int[] memo) {
    if (i >= h.length - 1) return 0;
    if (memo[i] != -1) return memo[i];
    int best = Math.abs(h[i] - h[i + 1]) + dp(h, i + 1, memo);
    if (i + 2 < h.length)
        best = Math.min(best, Math.abs(h[i] - h[i + 2]) + dp(h, i + 2, memo));
    return memo[i] = best;
}
```

Tabulation

dp[i] = min cost from i to end; answer is dp[0]. Fill from n-1 down to 0.

Complexity: O(n) time, O(n) space.

```
int minCost(int[] h) {
    int n = h.length;
    int[] dp = new int[n];
    dp[n - 1] = 0;
    for (int i = n - 2; i >= 0; i--) {
        dp[i] = Math.abs(h[i] - h[i + 1]) + dp[i + 1];
        if (i + 2 < n)
            dp[i] = Math.min(dp[i], Math.abs(h[i] - h[i + 2]) + dp[i + 2]);
    }
    return dp[0];
}
```

171. House Robber

Brute force

At each house, rob it (skip the next) or skip it. Explore both branches on the suffix.

Complexity: O(2^n) time, O(n) stack.

```
int rob(int[] a) {
    return pick(a, 0);
}
int pick(int[] a, int i) {
    if (i >= a.length) return 0;
    int take = a[i] + pick(a, i + 2);
    int skip = pick(a, i + 1);
    return Math.max(take, skip);
}
```

Memoization

dp[i] = max money from houses i..end. Either rob i and jump to i+2, or skip i.

Complexity: O(n) time, O(n) space.

```
int rob(int[] a) {
    int[] memo = new int[a.length];
    Arrays.fill(memo, -1);
    return dp(a, 0, memo);
}
int dp(int[] a, int i, int[] memo) {
    if (i >= a.length) return 0;
    if (memo[i] != -1) return memo[i];
    return memo[i] = Math.max(a[i] + dp(a, i + 2, memo), dp(a, i + 1, memo));
}
```

Tabulation

Track best if the previous house was robbed (skip current) vs not robbed (may rob current).

Complexity: O(n) time, O(1) space.

```
int rob(int[] a) {
    int robbed = 0, notRobbed = 0;
    for (int x : a) {
        int prevNot = notRobbed;
        notRobbed = Math.max(notRobbed, robbed + x);
        robbed = prevNot;
    }
    return Math.max(robbed, notRobbed);
}
```

172. House Robber II (Circular)

Brute force

First and last houses are adjacent, so either drop house 0 or drop house n-1. Run the linear robber on each shortened range and take the max.

Complexity: O(2^n) per range without DP; use memo on each slice for practice.

```
int rob(int[] a) {
    if (a.length == 1) return a[0];
    return Math.max(robRange(a, 0, a.length - 2), robRange(a, 1, a.length - 1));
}
int robRange(int[] a, int lo, int hi) {
    int[] memo = new int[hi - lo + 1];
    Arrays.fill(memo, -1);
    return dp(a, lo, hi, lo, memo);
}
int dp(int[] a, int lo, int hi, int i, int[] memo) {
    if (i > hi) return 0;
    int idx = i - lo;
    if (memo[idx] != -1) return memo[idx];
    return memo[idx] = Math.max(a[i] + dp(a, lo, hi, i + 2, memo),
        dp(a, lo, hi, i + 1, memo));
}
```

Tabulation

Linear DP on [0..n-2] and [1..n-1]; same rolling robbed / notRobbed as House Robber I.

Complexity: O(n) time, O(1) space.

```
int rob(int[] a) {
    if (a.length == 1) return a[0];
    return Math.max(linear(a, 0, a.length - 2), linear(a, 1, a.length - 1));
}
int linear(int[] a, int lo, int hi) {
    int robbed = 0, notRobbed = 0;
    for (int i = lo; i <= hi; i++) {
        int prevNot = notRobbed;
        notRobbed = Math.max(notRobbed, robbed + a[i]);
        robbed = prevNot;
    }
    return Math.max(robbed, notRobbed);
}
```

173. Unique Paths in a Grid

Brute force

From (0,0) move only right or down until (m-1,n-1). Count every path by exploring both moves.

Complexity: O(C(m+n) choose m) paths, exponential recursion without memo.

```
int uniquePaths(int m, int n) {
    return paths(0, 0, m - 1, n - 1);
}
```

```
int paths(int r, int c, int er, int ec) {
    if (r > er || c > ec) return 0;
    if (r == er && c == ec) return 1;
    return paths(r + 1, c, er, ec) + paths(r, c + 1, er, ec);
}
```

Memoization

Subproblem (r,c) = number of paths from (r,c) to the bottom-right. Only right/down so dp[r][c] = dp[r+1][c] + dp[r][c+1].

Complexity: O(mn) time, O(mn) space.

```
int uniquePaths(int m, int n) {
    int[][] memo = new int[m][n];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(0, 0, m - 1, n - 1, memo);
}
int dp(int r, int c, int er, int ec, int[][] memo) {
    if (r > er || c > ec) return 0;
    if (r == er && c == ec) return 1;
    if (memo[r][c] != -1) return memo[r][c];
    return memo[r][c] = dp(r + 1, c, er, ec, memo) + dp(r, c + 1, er, ec, memo);
}
```

Tabulation

First row and column are 1. Each cell is sum of cell above and left.

Complexity: O(mn) time, O(n) space with one row.

```
int uniquePaths(int m, int n) {
    int[] row = new int[n];
    Arrays.fill(row, 1);
    for (int i = 1; i < m; i++)
        for (int j = 1; j < n; j++)
            row[j] += row[j - 1];
    return row[n - 1];
}
```

174. Minimum Path Sum in a Grid

Brute force

From each cell go right or down and track the path sum. Keep the minimum over all paths to the bottom-right.

Complexity: Exponential paths, O(m+n) stack.

```
int minPathSum(int[][] g) {
    return walk(g, 0, 0, g[0][0]);
}
int walk(int[][] g, int r, int c, int sum) {
    int m = g.length, n = g[0].length;
    if (r == m - 1 && c == n - 1) return sum;
    int best = Integer.MAX_VALUE;
    if (r + 1 < m) best = Math.min(best, walk(g, r + 1, c, sum + g[r + 1][c]));
    if (c + 1 < n) best = Math.min(best, walk(g, r, c + 1, sum + g[r][c + 1]));
    return best;
}
```

Memoization

dp[r][c] = min sum from (r,c) to corner. Transition adds g[r][c] to min of down/right.

Complexity: O(mn) time, O(mn) space.

```
int minPathSum(int[][] g) {
    int m = g.length, n = g[0].length;
    int[][] memo = new int[m][n];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(g, 0, 0, memo);
}
int dp(int[][] g, int r, int c, int[][] memo) {
```

```
    int m = g.length, n = g[0].length;
    if (r == m - 1 && c == n - 1) return g[r][c];
    if (memo[r][c] != -1) return memo[r][c];
    int best = Integer.MAX_VALUE;
    if (r + 1 < m) best = Math.min(best, dp(g, r + 1, c, memo));
    if (c + 1 < n) best = Math.min(best, dp(g, r, c + 1, memo));
    return memo[r][c] = g[r][c] + best;
}
```

Tabulation

Fill dp in row-major order; first row/column only have one predecessor.

Complexity: O(mn) time, O(n) space.

```
int minPathSum(int[][] g) {
    int m = g.length, n = g[0].length;
    int[] dp = new int[n];
    dp[0] = g[0][0];
    for (int j = 1; j < n; j++) dp[j] = dp[j - 1] + g[0][j];
    for (int i = 1; i < m; i++) {
        dp[0] += g[i][0];
        for (int j = 1; j < n; j++)
            dp[j] = g[i][j] + Math.min(dp[j], dp[j - 1]);
    }
    return dp[n - 1];
}
```

175. Triangle — Minimum Path Sum

Brute force

From row r, column c you may go to (r+1,c) or (r+1,c+1). Sum the path and minimize over all routes to the last row.

Complexity: O(2^n) time, O(n) stack.

```
int minimumTotal(List<List<Integer>> t) {
    return dfs(t, 0, 0);
}
int dfs(List<List<Integer>> t, int r, int c) {
    if (r == t.size() - 1) return t.get(r).get(c);
    return t.get(r).get(c) + Math.min(dfs(t, r + 1, c), dfs(t, r + 1, c + 1));
}
```

Memoization

State (r,c) is min sum from that cell to the bottom.

Complexity: O(n^2) time, O(n^2) space.

```
int minimumTotal(List<List<Integer>> t) {
    int n = t.size();
    int[][] memo = new int[n][n];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(t, 0, 0, memo);
}
int dp(List<List<Integer>> t, int r, int c, int[][] memo) {
    if (r == t.size() - 1) return t.get(r).get(c);
    if (memo[r][c] != -1) return memo[r][c];
    return memo[r][c] = t.get(r).get(c)
        + Math.min(dp(t, r + 1, c, memo), dp(t, r + 1, c + 1, memo));
}
```

Tabulation

Start from the bottom row and for each row take min of the two children below.

Complexity: O(n^2) time, O(n) space.

```
int minimumTotal(List<List<Integer>> t) {
    int n = t.size();
    int[] dp = new int[n];
    for (int c = 0; c < n; c++) dp[c] = t.get(n - 1).get(c);
```

```
    for (int r = n - 2; r >= 0; r--)
        for (int c = 0; c <= r; c++)
            dp[c] = t.get(r).get(c) + Math.min(dp[c], dp[c + 1]);
    return dp[0];
}
```

176. Minimum Falling Path Sum

Brute force

From (r,c) fall to (r+1,c-1), (r+1,c), or (r+1,c+1). Try all downward paths and minimize total.

Complexity: O(3^n) time, O(n) stack.

```
int minFallingPathSum(int[][] m) {
    int n = m.length, best = Integer.MAX_VALUE;
    for (int c = 0; c < n; c++)
        best = Math.min(best, fall(m, 0, c));
    return best;
}
int fall(int[][] m, int r, int c) {
    int n = m.length;
    if (r == n - 1) return m[r][c];
    int best = Integer.MAX_VALUE;
    for (int nc = Math.max(0, c - 1); nc <= Math.min(n - 1, c + 1); nc++)
        best = Math.min(best, fall(m, r + 1, nc));
    return m[r][c] + best;
}
```

Memoization

dp[r][c] = min sum from (r,c) to the last row following falling rules.

Complexity: O(n^2) time, O(n^2) space.

```
int minFallingPathSum(int[][] m) {
    int n = m.length;
    int[][] memo = new int[n][n];
    for (int[] row : memo) Arrays.fill(row, -1);
    int best = Integer.MAX_VALUE;
    for (int c = 0; c < n; c++) best = Math.min(best, dp(m, 0, c, memo));
    return best;
}
int dp(int[][] m, int r, int c, int[][] memo) {
    int n = m.length;
    if (r == n - 1) return m[r][c];
    if (memo[r][c] != -1) return memo[r][c];
    int best = Integer.MAX_VALUE;
    for (int nc = Math.max(0, c - 1); nc <= Math.min(n - 1, c + 1); nc++)
        best = Math.min(best, dp(m, r + 1, nc, memo));
    return memo[r][c] = m[r][c] + best;
}
```

Tabulation

Copy the matrix and pull each cell from the min of up to three parents in the row below.

Complexity: O(n^2) time, O(n) space.

```
int minFallingPathSum(int[][] m) {
    int n = m.length;
    int[] dp = m[n - 1].clone();
    for (int r = n - 2; r >= 0; r--) {
        int[] next = new int[n];
        for (int c = 0; c < n; c++) {
            int best = dp[c];
            if (c > 0) best = Math.min(best, dp[c - 1]);
            if (c + 1 < n) best = Math.min(best, dp[c + 1]);
            next[c] = m[r][c] + best;
        }
        dp = next;
```

```
    }
    int ans = Integer.MAX_VALUE;
    for (int x : dp) ans = Math.min(ans, x);
    return ans;
}
```

177. Partition Equal Subset Sum

Brute force

Try every subset for sum S; the complement must also be S, so total must be even and some subset hits sum/2.

Complexity: O(2^n) time, O(n) stack.

```
boolean canPartition(int[] a) {
    int sum = 0;
    for (int x : a) sum += x;
    if (sum % 2 == 1) return false;
    return subset(a, 0, sum / 2);
}
boolean subset(int[] a, int i, int target) {
    if (target == 0) return true;
    if (i == a.length || target < 0) return false;
    return subset(a, i + 1, target - a[i]) || subset(a, i + 1, target);
}
```

Memoization

State (i, target) = can we reach target using elements from index i onward.

Complexity: O(n * sum) time, O(n * sum) space.

```
boolean canPartition(int[] a) {
    int sum = 0;
    for (int x : a) sum += x;
    if (sum % 2 == 1) return false;
    int half = sum / 2;
    Boolean[][] memo = new Boolean[a.length][half + 1];
    return dp(a, 0, half, memo);
}
boolean dp(int[] a, int i, int t, Boolean[][] memo) {
    if (t == 0) return true;
    if (i == a.length || t < 0) return false;
    if (memo[i][t] != null) return memo[i][t];
    return memo[i][t] = dp(a, i + 1, t - a[i], memo) || dp(a, i + 1, t, memo);
}
```

Tabulation

Boolean knapsack: dp[j] true if sum j is reachable. Roll from right to left when taking a[i].

Complexity: O(n * sum) time, O(sum) space.

```
boolean canPartition(int[] a) {
    int sum = 0;
    for (int x : a) sum += x;
    if (sum % 2 == 1) return false;
    int half = sum / 2;
    boolean[] dp = new boolean[half + 1];
    dp[0] = true;
    for (int x : a)
        for (int j = half; j >= x; j--)
            dp[j] |= dp[j - x];
    return dp[half];
}
```

178. 0/1 Knapsack

Brute force

For each item, include it in the knapsack or skip it. Maximize value without exceeding capacity W.

Complexity: O(2^n) time, O(n) stack.

```
int knapsack(int[] wt, int[] val, int W) {
    return pick(wt, val, 0, W);
}
int pick(int[] wt, int[] val, int i, int cap) {
    if (i == wt.length || cap == 0) return 0;
    int skip = pick(wt, val, i + 1, cap);
    int take = 0;
    if (wt[i] <= cap) take = val[i] + pick(wt, val, i + 1, cap - wt[i]);
    return Math.max(skip, take);
}
```

Memoization

State (i, cap) = best value using items i..end with remaining capacity cap.

Complexity: O(n * W) time, O(n * W) space.

```
int knapsack(int[] wt, int[] val, int W) {
    int[][] memo = new int[wt.length][W + 1];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(wt, val, 0, W, memo);
}
int dp(int[] wt, int[] val, int i, int cap, int[][] memo) {
    if (i == wt.length || cap == 0) return 0;
    if (memo[i][cap] != -1) return memo[i][cap];
    int best = dp(wt, val, i + 1, cap, memo);
    if (wt[i] <= cap)
        best = Math.max(best, val[i] + dp(wt, val, i + 1, cap - wt[i], memo));
    return memo[i][cap] = best;
}
```

Tabulation

dp[cap] = max value for that capacity. Process each item once, iterating capacity downward.

Complexity: O(n * W) time, O(W) space.

```
int knapsack(int[] wt, int[] val, int W) {
    int[] dp = new int[W + 1];
    for (int i = 0; i < wt.length; i++)
        for (int c = W; c >= wt[i]; c--)
            dp[c] = Math.max(dp[c], dp[c - wt[i]] + val[i]);
    return dp[W];
}
```

179. Coin Change (Minimum Coins)

Brute force

Try every coin as the first pick and recurse on amount - coin. Minimize 1 + best sub-answer.

Complexity: O(S^n) time for amount S and n coin types, exponential stack.

```
int coinChange(int[] coins, int amount) {
    int ans = minCoins(coins, amount);
    return ans >= Integer.MAX_VALUE / 2 ? -1 : ans;
}
int minCoins(int[] coins, int left) {
    if (left == 0) return 0;
    if (left < 0) return Integer.MAX_VALUE / 2;
    int best = Integer.MAX_VALUE / 2;
    for (int c : coins) best = Math.min(best, 1 + minCoins(coins, left - c));
    return best;
}
```

Memoization

dp[a] = min coins to make amount a. Same recurrence with a cache on amount.

Complexity: O(n * amount) time, O(amount) space.

```
int coinChange(int[] coins, int amount) {
    int[] memo = new int[amount + 1];
    Arrays.fill(memo, -1);
    int ans = dp(coins, amount, memo);
    return ans >= Integer.MAX_VALUE / 2 ? -1 : ans;
}
int dp(int[] coins, int left, int[] memo) {
    if (left == 0) return 0;
    if (left < 0) return Integer.MAX_VALUE / 2;
    if (memo[left] != -1) return memo[left];
    int best = Integer.MAX_VALUE / 2;
    for (int c : coins) best = Math.min(best, 1 + dp(coins, left - c, memo));
    return memo[left] = best;
}
```

Tabulation

Fill dp[0..amount] with INF; dp[0]=0. For each coin, improve every reachable amount.

Complexity: O(n * amount) time, O(amount) space.

```
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int c : coins)
        for (int a = c; a <= amount; a++)
            dp[a] = Math.min(dp[a], dp[a - c] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

180. Target Sum

Brute force

Assign + or - to each number. At index i, branch on +nums[i] and -nums[i] and check if the running sum hits 0 at the end.

Complexity: O(2^n) time, O(n) stack.

```
int findTargetSumWays(int[] a, int target) {
    return ways(a, 0, 0, target);
}
int ways(int[] a, int i, int sum, int target) {
    if (i == a.length) return sum == target ? 1 : 0;
    return ways(a, i + 1, sum + a[i], target) + ways(a, i + 1, sum - a[i], target);
}
```

Memoization

State (i, sum) counts assignments on suffix i.. that reach target from partial sum sum.

Complexity: O(n * sum range) time, O(n * sum) space.

```
int findTargetSumWays(int[] a, int target) {
    int total = 0;
    for (int x : a) total += x;
    if ((total + target) % 2 != 0 || Math.abs(target) > total) return 0;
    int need = (total + target) / 2;
    int[][] memo = new int[a.length][need + 1];
    for (int[] row : memo) Arrays.fill(row, -1);
    return subset(a, 0, need, memo);
}
int subset(int[] a, int i, int t, int[][] memo) {
    if (t == 0) return 1;
    if (i == a.length || t < 0) return 0;
    if (memo[i][t] != -1) return memo[i][t];
```

```
    return memo[i][t] = subset(a, i + 1, t - a[i], memo) + subset(a, i + 1, t, memo);
}
```

Tabulation

Reduce to counting subsets with sum (total + target) / 2. dp[j] += ways to reach j.

Complexity: O(n * sum) time, O(sum) space.

```
int findTargetSumWays(int[] a, int target) {
    int total = 0;
    for (int x : a) total += x;
    if ((total + target) % 2 != 0 || Math.abs(target) > total) return 0;
    int need = (total + target) / 2;
    int[] dp = new int[need + 1];
    dp[0] = 1;
    for (int x : a)
        for (int j = need; j >= x; j--)
            dp[j] += dp[j - x];
    return dp[need];
}
```

181. Best Time to Buy and Sell Stock

Brute force

Try every buy day i and every sell day j > i. Track max(prices[j] - prices[i]).

Complexity: O(n^2) time, O(1) space.

```
int maxProfit(int[] prices) {
    int best = 0;
    for (int i = 0; i < prices.length; i++)
        for (int j = i + 1; j < prices.length; j++)
            best = Math.max(best, prices[j] - prices[i]);
    return best;
}
```

Better

One pass: min price seen so far is the best buy before today; update best profit with today's price.

Complexity: O(n) time, O(1) space.

```
int maxProfit(int[] prices) {
    int best = 0, buy = prices[0];
    for (int i = 1; i < prices.length; i++) {
        best = Math.max(best, prices[i] - buy);
        buy = Math.min(buy, prices[i]);
    }
    return best;
}
```

182. Best Time to Buy and Sell Stock with Cooldown

Memoization

State (i, hold) where hold is 0 (free to buy) or 1 (holding stock). Buy, sell (+ cooldown day), or rest.

Complexity: O(n) time, O(n) space.

```
int maxProfit(int[] p) {
    int[][] memo = new int[p.length][2];
        for (int[] row : memo) Arrays.fill(row, -1);
    return dp(p, 0, 0, memo);
}
int dp(int[] p, int i, int hold, int[][] memo) {
    if (i == p.length) return 0;
    if (memo[i][hold] != -1) return memo[i][hold];
    if (hold == 0)
        return memo[i][hold] = Math.max(dp(p, i + 1, 0, memo), -p[i] + dp(p, i + 1, 1, memo));
    return memo[i][hold] = Math.max(dp(p, i + 1, 1, memo), p[i] + dp(p, i + 2, 0, memo));
}
```

Tabulation

Three rolling states: sold (just sold), hold, rest (cooldown finished). Transitions match the recursion.

Complexity: O(n) time, O(1) space.

```
int maxProfit(int[] p) {
    int sold = 0, hold = Integer.MIN_VALUE / 2, rest = 0;
    for (int x : p) {
        int prevSold = sold;
        sold = hold + x;
        hold = Math.max(hold, rest - x);
        rest = Math.max(rest, prevSold);
    }
    return Math.max(sold, rest);
}
```

183. Best Time to Buy and Sell Stock (At Most k Transactions)

Memoization

State (day, k left, hold). Each day: buy (spend one transaction slot), sell, or skip.

Complexity: O(n * k) time, O(n * k) space.

```
int maxProfit(int k, int[] p) {
    if (k >= p.length / 2) {
        int profit = 0;
        for (int i = 1; i < p.length; i++)
            if (p[i] > p[i - 1]) profit += p[i] - p[i - 1];
        return profit;
    }
    int[][][] memo = new int[p.length][k + 1][2];
    for (int[][] a : memo)
        for (int[] b : a) Arrays.fill(b, -1);
    return dp(p, 0, k, 0, memo);
}
int dp(int[] p, int i, int k, int hold, int[][][] memo) {
    if (i == p.length) return 0;
    if (memo[i][k][hold] != -1) return memo[i][k][hold];
    if (hold == 0) {
        int skip = dp(p, i + 1, k, 0, memo);
        int buy = (k > 0) ? -p[i] + dp(p, i + 1, k - 1, 1, memo) : Integer.MIN_VALUE / 2;
        return memo[i][k][hold] = Math.max(skip, buy);
    }
    int skip = dp(p, i + 1, k, 1, memo);
    int sell = p[i] + dp(p, i + 1, k, 0, memo);
    return memo[i][k][hold] = Math.max(skip, sell);
}
```

Tabulation

buy[j] = max profit after j buys; sell[j] = max after j complete transactions. Update in one forward pass.

Complexity: O(n * k) time, O(k) space.

```
int maxProfit(int k, int[] p) {
    if (k >= p.length / 2) {
        int profit = 0;
        for (int i = 1; i < p.length; i++)
            if (p[i] > p[i - 1]) profit += p[i] - p[i - 1];
        return profit;
    }
    int[] buy = new int[k + 1];
    int[] sell = new int[k + 1];
    Arrays.fill(buy, Integer.MIN_VALUE / 2);
    for (int x : p) {
        for (int j = 1; j <= k; j++) {
            buy[j] = Math.max(buy[j], sell[j - 1] - x);
            sell[j] = Math.max(sell[j], buy[j] + x);
        }
```

```
    }
    return sell[k];
}
```

184. Longest Increasing Subsequence

Brute force

For each start index, extend while values strictly increase. Track the longest length.

Complexity: O(2^n) if generating all subsequences; O(n^2) greedy extension per start.

```
int lengthOfLIS(int[] a) {
    int best = 0;
    for (int i = 0; i < a.length; i++) best = Math.max(best, extend(a, i));
    return best;
}
int extend(int[] a, int i) {
    int len = 1;
    for (int j = i + 1; j < a.length; j++)
        if (a[j] > a[i]) len = Math.max(len, 1 + extend(a, j));
    return len;
}
```

Tabulation

dp[i] = LIS ending at i. For every j < i with a[j] < a[i], dp[i] = max(dp[i], dp[j] + 1).

Complexity: O(n^2) time, O(n) space.

```
int lengthOfLIS(int[] a) {
    int n = a.length, best = 0;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
        best = Math.max(best, dp[i]);
    }
    return best;
}
```

Optimal

tails[len] = smallest ending value of an increasing subsequence of length len+1. Binary search the position of a[i].

Complexity: O(n log n) time, O(n) space.

```
int lengthOfLIS(int[] a) {
    int[] tails = new int[a.length];
    int len = 0;
    for (int x : a) {
        int lo = 0, hi = len;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (tails[mid] < x) lo = mid + 1;
            else hi = mid;
        }
        tails[lo] = x;
        if (lo == len) len++;
    }
    return len;
}
```

185. Longest Common Subsequence

Brute force

Recurse on both strings: match advances both; mismatch tries skipping either side.

Complexity: O(2^(m+n)) time, O(m+n) stack.

```
int longestCommonSubsequence(String s, String t) {
    return lcs(s, t, 0, 0);
```

```
}
int lcs(String s, String t, int i, int j) {
    if (i == s.length() || j == t.length()) return 0;
    if (s.charAt(i) == t.charAt(j))
        return 1 + lcs(s, t, i + 1, j + 1);
    return Math.max(lcs(s, t, i + 1, j), lcs(s, t, i, j + 1));
}
```

Memoization

dp[i][j] = LCS length of suffixes s[i..] and t[j..].

Complexity: O(mn) time, O(mn) space.

```
int longestCommonSubsequence(String s, String t) {
    int[][] memo = new int[s.length()][t.length()];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(s, t, 0, 0, memo);
}
int dp(String s, String t, int i, int j, int[][] memo) {
    if (i == s.length() || j == t.length()) return 0;
    if (memo[i][j] != -1) return memo[i][j];
    if (s.charAt(i) == t.charAt(j))
        return memo[i][j] = 1 + dp(s, t, i + 1, j + 1, memo);
    return memo[i][j] = Math.max(dp(s, t, i + 1, j, memo), dp(s, t, i, j + 1, memo));
}
```

Tabulation

Fill a table where match extends diagonal, else max of up/left.

Complexity: O(mn) time, O(n) space.

```
int longestCommonSubsequence(String s, String t) {
    int m = s.length(), n = t.length();
    int[] prev = new int[n + 1], cur = new int[n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s.charAt(i - 1) == t.charAt(j - 1)) cur[j] = prev[j - 1] + 1;
            else cur[j] = Math.max(prev[j], cur[j - 1]);
        }
        int[] tmp = prev; prev = cur; cur = tmp;
        Arrays.fill(cur, 0);
    }
    return prev[n];
}
```

186. Edit Distance

Brute force

Align prefixes: match both, or insert, delete, or replace on word1 to match word2.

Complexity: O(3^(m+n)) time without memo, O(m+n) stack.

```
int minDistance(String a, String b) {
    return edit(a, b, 0, 0);
}
int edit(String a, String b, int i, int j) {
    if (i == a.length()) return b.length() - j;
    if (j == b.length()) return a.length() - i;
    if (a.charAt(i) == b.charAt(j)) return edit(a, b, i + 1, j + 1);
    int rep = 1 + edit(a, b, i + 1, j + 1);
    int del = 1 + edit(a, b, i + 1, j);
    int ins = 1 + edit(a, b, i, j + 1);
    return Math.min(rep, Math.min(del, ins));
}
```

Memoization

dp[i][j] = min edits to turn a[i..] into b[j..].

Complexity: O(mn) time, O(mn) space.

```
int minDistance(String a, String b) {
    int[][] memo = new int[a.length()][b.length()];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(a, b, 0, 0, memo);
}
int dp(String a, String b, int i, int j, int[][] memo) {
    if (i == a.length()) return b.length() - j;
    if (j == b.length()) return a.length() - i;
    if (memo[i][j] != -1) return memo[i][j];
    if (a.charAt(i) == b.charAt(j)) return memo[i][j] = dp(a, b, i + 1, j + 1, memo);
    int rep = 1 + dp(a, b, i + 1, j + 1, memo);
    int del = 1 + dp(a, b, i + 1, j, memo);
    int ins = 1 + dp(a, b, i, j + 1, memo);
    return memo[i][j] = Math.min(rep, Math.min(del, ins));
}
```

Tabulation

Base cases: empty prefix costs equal to other string length. Fill row by row.

Complexity: O(mn) time, O(n) space.

```
int minDistance(String a, String b) {
    int m = a.length(), n = b.length();
    int[] prev = new int[n + 1], cur = new int[n + 1];
    for (int j = 0; j <= n; j++) prev[j] = j;
    for (int i = 1; i <= m; i++) {
        cur[0] = i;
        for (int j = 1; j <= n; j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) cur[j] = prev[j - 1];
            else cur[j] = 1 + Math.min(prev[j - 1], Math.min(prev[j], cur[j - 1]));
        }
        int[] tmp = prev; prev = cur; cur = tmp;
    }
    return prev[n];
}
```

187. Distinct Subsequences

Brute force

Walk s with index i and t with j. Match advances both; on s you may skip the current char or take it if it matches t[j].

Complexity: O(2^m) time, O(m+n) stack.

```
int numDistinct(String s, String t) {
    return count(s, t, 0, 0);
}
int count(String s, String t, int i, int j) {
    if (j == t.length()) return 1;
    if (i == s.length()) return 0;
    long ways = count(s, t, i + 1, j);
    if (s.charAt(i) == t.charAt(j)) ways += count(s, t, i + 1, j + 1);
    return (int) ways;
}
```

Memoization

dp[i][j] = ways to form t[j..] using s[i..]. Add dp[i+1][j+1] on match plus skip s[i].

Complexity: O(mn) time, O(mn) space.

```
int numDistinct(String s, String t) {
    int[][] memo = new int[s.length()][t.length()];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(s, t, 0, 0, memo);
}
```

```
int dp(String s, String t, int i, int j, int[][] memo) {
    if (j == t.length()) return 1;
    if (i == s.length()) return 0;
    if (memo[i][j] != -1) return memo[i][j];
    long ways = dp(s, t, i + 1, j, memo);
    if (s.charAt(i) == t.charAt(j)) ways += dp(s, t, i + 1, j + 1, memo);
    return memo[i][j] = (int) ways;
}
```

Tabulation

dp[j] = ways to match t[0..j). Scan s left to right, update dp backward on matches.

Complexity: O(mn) time, O(n) space.

```
int numDistinct(String s, String t) {
    int n = t.length();
    int[] dp = new int[n + 1];
    dp[0] = 1;
    for (int i = 0; i < s.length(); i++)
        for (int j = n - 1; j >= 0; j--)
            if (s.charAt(i) == t.charAt(j)) dp[j + 1] += dp[j];
    return dp[n];
}
```

188. Wildcard Matching

Brute force

'?' matches one char; '*' matches any sequence (including empty). Branch on both interpretations for '*'.

Complexity: Exponential without memo, O(m+n) stack.

```
boolean isMatch(String s, String p) {
    return match(s, p, 0, 0);
}
boolean match(String s, String p, int i, int j) {
    if (j == p.length()) return i == s.length();
    if (p.charAt(j) == '*')
        return match(s, p, i, j + 1) || (i < s.length() && match(s, p, i + 1, j));
    if (i == s.length()) return false;
    if (p.charAt(j) == '?' || p.charAt(j) == s.charAt(i))
        return match(s, p, i + 1, j + 1);
    return false;
}
```

Memoization

Cache (i,j) for the same recursive cases.

Complexity: O(mn) time, O(mn) space.

```
boolean isMatch(String s, String p) {
    Boolean[][] memo = new Boolean[s.length()][p.length()];
    return dp(s, p, 0, 0, memo);
}
boolean dp(String s, String p, int i, int j, Boolean[][] memo) {
    if (j == p.length()) return i == s.length();
    if (memo[i][j] != null) return memo[i][j];
    boolean ans;
    if (p.charAt(j) == '*')
        ans = dp(s, p, i, j + 1, memo) || (i < s.length() && dp(s, p, i + 1, j, memo));
    else if (i == s.length()) ans = false;
    else if (p.charAt(j) == '?' || p.charAt(j) == s.charAt(i))
        ans = dp(s, p, i + 1, j + 1, memo);
    else ans = false;
    return memo[i][j] = ans;
}
```

Tabulation

dp[i][j] = match s[0..i) with p[0..j). Star row: empty match or extend match with one more char.

Complexity: O(mn) time, O(n) space.

```
boolean isMatch(String s, String p) {
    int m = s.length(), n = p.length();
    boolean[] prev = new boolean[n + 1];
    prev[0] = true;
    for (int j = 1; j <= n; j++)
        if (p.charAt(j - 1) != '*') prev[j] = false;
    for (int i = 1; i <= m; i++) {
        boolean[] cur = new boolean[n + 1];
        for (int j = 1; j <= n; j++) {
            char pc = p.charAt(j - 1);
            if (pc == '*') cur[j] = cur[j - 1] || prev[j];
            else if (pc == '?' || s.charAt(i - 1) == pc) cur[j] = prev[j - 1];
        }
        prev = cur;
    }
    return prev[n];
}
```

189. Matrix Chain Multiplication

Brute force

Try every split k between i and j for matrices i..j. Cost = left cost + right cost + multiply sizes.

Complexity: O(2^n) time, O(n) stack.

```
int matrixChainOrder(int[] dim) {
    int n = dim.length - 1;
    return mcm(dim, 1, n);
}
int mcm(int[] dim, int i, int j) {
    if (i >= j) return 0;
    int best = Integer.MAX_VALUE;
    for (int k = i; k < j; k++) {
        int cost = mcm(dim, i, k) + mcm(dim, k + 1, j)
            + dim[i - 1] * dim[k] * dim[j];
        best = Math.min(best, cost);
    }
    return best;
}
```

Memoization

dp[i][j] = min scalar multiplications for chain i..j (1-based matrix indices).

Complexity: O(n^3) time, O(n^2) space.

```
int matrixChainOrder(int[] dim) {
    int n = dim.length - 1;
    int[][] memo = new int[n + 1][n + 1];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(dim, 1, n, memo);
}
int dp(int[] dim, int i, int j, int[][] memo) {
    if (i >= j) return 0;
    if (memo[i][j] != -1) return memo[i][j];
    int best = Integer.MAX_VALUE;
    for (int k = i; k < j; k++) {
        int cost = dp(dim, i, k, memo) + dp(dim, k + 1, j, memo)
            + dim[i - 1] * dim[k] * dim[j];
        best = Math.min(best, cost);
    }
    return memo[i][j] = best;
}
```

Tabulation

Increase chain length L from 2 to n and try every split inside [i..j].

Complexity: O(n^3) time, O(n^2) space.

```
int matrixChainOrder(int[] dim) {
    int n = dim.length - 1;
    int[][] dp = new int[n + 1][n + 1];
    for (int len = 2; len <= n; len++)
        for (int i = 1; i + len - 1 <= n; i++) {
            int j = i + len - 1;
            dp[i][j] = Integer.MAX_VALUE;
            for (int k = i; k < j; k++)
                dp[i][j] = Math.min(dp[i][j],
                    dp[i][k] + dp[k + 1][j] + dim[i - 1] * dim[k] * dim[j]);
        }
    return dp[1][n];
}
```

190. Burst Balloons

Brute force

Pick the last balloon k to burst in range (i,j); coins = nums[i]*nums[k]*nums[j] plus left and right sub-ranges.

Complexity: O(4^n) time without memo.

```
int maxCoins(int[] nums) {
    int n = nums.length;
    int[] a = new int[n + 2];
    a[0] = a[n + 1] = 1;
    for (int i = 0; i < n; i++) a[i + 1] = nums[i];
    return burst(a, 0, n + 1);
}
int burst(int[] a, int i, int j) {
    if (i + 1 >= j) return 0;
    int best = 0;
    for (int k = i + 1; k < j; k++) {
        int gain = a[i] * a[k] * a[j]
            + burst(a, i, k) + burst(a, k, j);
        best = Math.max(best, gain);
    }
    return best;
}
```

Memoization

Pad with 1s. dp[i][j] = max coins for open interval (i,j) excluding borders.

Complexity: O(n^3) time, O(n^2) space.

```
int maxCoins(int[] nums) {
    int n = nums.length;
    int[] a = new int[n + 2];
    a[0] = a[n + 1] = 1;
    for (int i = 0; i < n; i++) a[i + 1] = nums[i];
    int[][] memo = new int[n + 2][n + 2];
    for (int[] row : memo) Arrays.fill(row, -1);
    return dp(a, 0, n + 1, memo);
}
int dp(int[] a, int i, int j, int[][] memo) {
    if (i + 1 >= j) return 0;
    if (memo[i][j] != -1) return memo[i][j];
    int best = 0;
    for (int k = i + 1; k < j; k++) {
        int gain = a[i] * a[k] * a[j] + dp(a, i, k, memo) + dp(a, k, j, memo);
        best = Math.max(best, gain);
    }
    return memo[i][j] = best;
}
```

Tabulation

Same interval DP as memo, increasing window length on the padded array.

Complexity: O(n^3) time, O(n^2) space.

```
int maxCoins(int[] nums) {
    int n = nums.length;
    int[] a = new int[n + 2];
    a[0] = a[n + 1] = 1;
    for (int i = 0; i < n; i++) a[i + 1] = nums[i];
    int m = n + 2;
    int[][] dp = new int[m][m];
    for (int len = 3; len <= m; len++)
        for (int i = 0; i + len - 1 < m; i++) {
            int j = i + len - 1;
            for (int k = i + 1; k < j; k++)
                dp[i][j] = Math.max(dp[i][j],
                    a[i] * a[k] * a[j] + dp[i][k] + dp[k][j]);
        }
    return dp[0][m - 1];
}
```

191. Palindrome Partitioning II (Minimum Cuts)

Brute force

Try every cut position after i if s[0..i] is a palindrome; 1 + min cuts on the suffix.

Complexity: O(2^n) time, O(n) stack.

```
int minCut(String s) {
    return cuts(s, 0);
}
int cuts(String s, int start) {
    if (start == s.length()) return -1;
    int best = Integer.MAX_VALUE;
    for (int end = start; end < s.length(); end++)
        if (isPal(s, start, end))
            best = Math.min(best, 1 + cuts(s, end + 1));
    return best;
}
boolean isPal(String s, int l, int r) {
    while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
    return true;
}
```

Memoization

dp[i] = min cuts for suffix s[i..]. Precompute palindrome table to check segments in O(1).

Complexity: O(n^2) time, O(n^2) space.

```
int minCut(String s) {
    int n = s.length();
    boolean[][] pal = new boolean[n][n];
    for (int i = n - 1; i >= 0; i--)
        for (int j = i; j < n; j++)
            pal[i][j] = s.charAt(i) == s.charAt(j)
                && (j - i < 2 || pal[i + 1][j - 1]);
    int[] memo = new int[n];
    Arrays.fill(memo, -1);
    return dp(s, 0, pal, memo);
}
int dp(String s, int i, boolean[][] pal, int[] memo) {
    if (i == s.length()) return -1;
    if (memo[i] != -1) return memo[i];
    int best = Integer.MAX_VALUE;
    for (int j = i; j < s.length(); j++)
        if (pal[i][j]) best = Math.min(best, 1 + dp(s, j + 1, pal, memo));
    return memo[i] = best;
```

```
}
```

Tabulation

cuts[i] = min cuts for s[i..]. If s[i..j] palindrome, cuts[i] = min(cuts[i], cuts[j+1]+1).

Complexity: O(n^2) time, O(n^2) space.

```
int minCut(String s) {
    int n = s.length();
    boolean[][] pal = new boolean[n][n];
    for (int i = n - 1; i >= 0; i--)
        for (int j = i; j < n; j++)
            pal[i][j] = s.charAt(i) == s.charAt(j)
                && (j - i < 2 || pal[i + 1][j - 1]);
    int[] cuts = new int[n];
    for (int i = 0; i < n; i++) {
        cuts[i] = i;
        for (int j = 0; j <= i; j++)
            if (pal[j][i]) cuts[i] = Math.min(cuts[i], (j > 0 ? cuts[j - 1] : -1) + 1);
    }
    return cuts[n - 1];
}
```

Tries

Prefix trees share edges for common prefixes. Each node is a character; paths spell keys. Good for dictionary lookup, autocomplete, and bitwise tries for XOR.

192. Implement Trie (Prefix Tree)

Brute force

Store every word in a list. Search and startsWith scan all strings and compare character by character.

Complexity: O(n * L) per query, n words, L average length.

```
class WordList {
    List<String> words = new ArrayList<>();
    void insert(String w) { words.add(w); }
    boolean search(String w) { return words.contains(w); }
    boolean startsWith(String p) {
        for (String s : words) if (s.startsWith(p)) return true;
        return false;
    }
}
```

Better

Array of 26 children per node plus end-of-word flag. Walk one char per edge; missing child means no match.

Complexity: O(L) per operation, O(total chars) space.

```
class Trie {
    static class Node {
        Node[] next = new Node[26];
        boolean end;
    }
    Node root = new Node();
    public void insert(String w) {
        Node cur = root;
        for (char ch : w.toCharArray()) {
            int i = ch - 'a';
            if (cur.next[i] == null) cur.next[i] = new Node();
            cur = cur.next[i];
        }
        cur.end = true;
    }
    public boolean search(String w) {
        Node node = walk(w);
        return node != null && node.end;
    }
    public boolean startsWith(String p) {
        return walk(p) != null;
    }
    Node walk(String s) {
        Node cur = root;
        for (char ch : s.toCharArray()) {
            int i = ch - 'a';
            if (cur.next[i] == null) return null;
            cur = cur.next[i];
        }
        return cur;
    }
}
```

193. Design Add and Search Words Data Structure

Brute force

Keep a list of inserted words. Search with '.' tries every letter at that position across all words.

Complexity: O(n * 26^d) worst case for n words and d dots.

```
class WordDictionary {
```

```
    List<String> words = new ArrayList<>();
    public void addWord(String w) { words.add(w); }
    public boolean search(String w) {
        for (String s : words) if (match(s, w, 0)) return true;
        return false;
    }
    boolean match(String s, String w, int i) {
        if (i == w.length()) return s.length() == w.length();
        char c = w.charAt(i);
        if (c == '.') {
            for (char ch = 'a'; ch <= 'z'; ch++)
                if (i < s.length() && s.charAt(i) == ch && match(s, w, i + 1)) return true;
            return false;
        }
        return i < s.length() && s.charAt(i) == c && match(s, w, i + 1);
    }
}
```

Optimal

Trie for exact letters; on '.' DFS all 26 children at that level.

Complexity: O(L) for exact search; O(26^k * L) with k wildcards.

```
class WordDictionary {
    static class Node {
        Node[] next = new Node[26];
        boolean end;
    }
    Node root = new Node();
    public void addWord(String w) {
        Node cur = root;
        for (char ch : w.toCharArray()) {
            int i = ch - 'a';
            if (cur.next[i] == null) cur.next[i] = new Node();
            cur = cur.next[i];
        }
        cur.end = true;
    }
    public boolean search(String w) { return dfs(root, w, 0); }
    boolean dfs(Node node, String w, int i) {
        if (node == null) return false;
        if (i == w.length()) return node.end;
        char c = w.charAt(i);
        if (c == '.') {
            for (int k = 0; k < 26; k++)
                if (dfs(node.next[k], w, i + 1)) return true;
            return false;
        }
        return dfs(node.next[c - 'a'], w, i + 1);
    }
}
```

194. Longest Word in Dictionary (Complete String)

Brute force

Sort words by length then lex order. A word is valid only if every proper prefix exists in the set.

Complexity: O(n * L^2) time checking prefixes, O(n * L) space.

```
String longestWord(String[] words) {
        Set<String> set = new HashSet<>(Arrays.asList(words));
        Arrays.sort(words, (a, b) -> a.length() != b.length()
            ? a.length() - b.length() : a.compareTo(b));
        String best = "";
        for (String w : words) {
            if (!canBuild(w, set)) continue;
            if (w.length() > best.length()
```

```
                || (w.length() == best.length() && w.compareTo(best) < 0))
                best = w;
        }
        return best;
    }
    boolean canBuild(String w, Set<String> set) {
        for (int i = 1; i < w.length(); i++)
            if (!set.contains(w.substring(0, i))) return false;
        return true;
    }
```

Optimal

Insert into trie; DFS from root following only nodes marked end. Track longest complete path.

Complexity: O(total chars) time and space.

```
String longestWord(String[] words) {
        TrieNode root = new TrieNode();
        for (String w : words) insert(root, w);
        String[] best = {""};
        dfs(root, new StringBuilder(), best);
        return best[0];
    }
    static class TrieNode {
        TrieNode[] next = new TrieNode[26];
        boolean end;
    }
    void insert(TrieNode root, String w) {
        TrieNode cur = root;
        for (char ch : w.toCharArray()) {
            int i = ch - 'a';
            if (cur.next[i] == null) cur.next[i] = new TrieNode();
            cur = cur.next[i];
        }
        cur.end = true;
    }
    void dfs(TrieNode node, StringBuilder path, String[] best) {
        if (node.end) {
            String s = path.toString();
            if (s.length() > best[0].length()
                || (s.length() == best[0].length() && s.compareTo(best[0]) < 0))
                best[0] = s;
        }
        for (int i = 0; i < 26; i++) {
            if (node.next[i] == null) continue;
            if (path.length() > 0 && !node.end) continue;
            path.append((char) ('a' + i));
            dfs(node.next[i], path, best);
            path.deleteCharAt(path.length() - 1);
        }
    }
```

195. Count Distinct Substrings

Brute force

Every substring s[i..j] goes into a hash set. Answer is set size.

Complexity: O(n^2 * L) hash work, O(n^2) space.

```
int countDistinct(String s) {
        Set<String> seen = new HashSet<>();
        for (int i = 0; i < s.length(); i++)
            for (int j = i; j < s.length(); j++)
                seen.add(s.substring(i, j + 1));
        return seen.size();
    }
```

Optimal

Trie of all suffixes: each new edge while inserting s[i..] is one new distinct substring.

Complexity: O(n^2) time worst case, O(n^2) space.

```
int countDistinct(String s) {
        TrieNode root = new TrieNode();
        int count = 0;
        for (int i = 0; i < s.length(); i++)
            count += insertSuffix(root, s, i);
        return count;
    }
    static class TrieNode {
        TrieNode[] next = new TrieNode[26];
    }
    int insertSuffix(TrieNode root, String s, int start) {
        TrieNode cur = root;
        int added = 0;
        for (int i = start; i < s.length(); i++) {
            int idx = s.charAt(i) - 'a';
            if (cur.next[idx] == null) {
                cur.next[idx] = new TrieNode();
                added++;
            }
            cur = cur.next[idx];
        }
        return added;
    }
```

196. Maximum XOR of Two Numbers in an Array

Brute force

Compare every pair and keep the maximum XOR.

Complexity: O(n^2) time, O(1) space.

```
int findMaximumXOR(int[] nums) {
        int best = 0;
        for (int i = 0; i < nums.length; i++)
            for (int j = i + 1; j < nums.length; j++)
                best = Math.max(best, nums[i] ^ nums[j]);
        return best;
    }
```

Better

Binary trie on 31-bit representation. For each number, greedily pick the opposite bit child when it exists.

Complexity: O(n * 32) time, O(n * 32) space.

```
int findMaximumXOR(int[] nums) {
        TrieNode root = new TrieNode();
        for (int x : nums) insert(root, x);
        int best = 0;
        for (int x : nums) best = Math.max(best, query(root, x));
        return best;
    }
    static class TrieNode {
        TrieNode[] next = new TrieNode[2];
    }
    void insert(TrieNode root, int x) {
        TrieNode cur = root;
        for (int b = 31; b >= 0; b--) {
            int bit = (x >> b) & 1;
            if (cur.next[bit] == null) cur.next[bit] = new TrieNode();
            cur = cur.next[bit];
        }
    }
    int query(TrieNode root, int x) {
```

```
        TrieNode cur = root;
        int ans = 0;
        for (int b = 31; b >= 0; b--) {
            int bit = (x >> b) & 1;
            int want = 1 - bit;
            if (cur.next[want] != null) {
                ans |= (1 << b);
                cur = cur.next[want];
            } else cur = cur.next[bit];
        }
        return ans;
    }
```

197. Maximum XOR Queries (Offline)

Brute force

For each query (x, m), scan all nums with value <= m and maximize x XOR num.

Complexity: O(q * n) time.

```
int[] maximizeXor(int[] nums, int[][] queries) {
        int[] ans = new int[queries.length];
        for (int i = 0; i < queries.length; i++) {
            int x = queries[i][0], m = queries[i][1];
            int best = -1;
            for (int v : nums) if (v <= m) best = Math.max(best, x ^ v);
            ans[i] = best;
        }
        return ans;
    }
```

Optimal

Sort nums and queries by limit m. Grow a binary trie as eligible nums increase; answer each query with greedy XOR walk.

Complexity: O((n + q) log U) time for U = max value, O(n * 32) space.

```
int[] maximizeXor(int[] nums, int[][] queries) {
        Arrays.sort(nums);
        int q = queries.length;
        int[][] ord = new int[q][3];
        for (int i = 0; i < q; i++) {
            ord[i][0] = queries[i][1];
            ord[i][1] = queries[i][0];
            ord[i][2] = i;
        }
        Arrays.sort(ord, (a, b) -> Integer.compare(a[0], b[0]));
        TrieNode root = new TrieNode();
        int ptr = 0;
        int[] ans = new int[q];
        for (int[] row : ord) {
            int m = row[0], x = row[1], idx = row[2];
            while (ptr < nums.length && nums[ptr] <= m) {
                insert(root, nums[ptr++]);
            }
            ans[idx] = ptr == 0 ? -1 : query(root, x);
        }
        return ans;
    }
    static class TrieNode {
        TrieNode[] next = new TrieNode[2];
    }
    void insert(TrieNode root, int x) {
        TrieNode cur = root;
        for (int b = 31; b >= 0; b--) {
            int bit = (x >> b) & 1;
            if (cur.next[bit] == null) cur.next[bit] = new TrieNode();
```

```
            cur = cur.next[bit];
        }
    }
    int query(TrieNode root, int x) {
        TrieNode cur = root;
        int ans = 0;
        for (int b = 31; b >= 0; b--) {
            int bit = (x >> b) & 1;
            int want = 1 - bit;
            if (cur.next[want] != null) {
                ans |= (1 << b);
                cur = cur.next[want];
            } else cur = cur.next[bit];
        }
        return ans;
    }
```

198. Word Search II

Brute force

For each word, run standard grid DFS with backtracking to see if the word appears.

Complexity: O(W * n * m * 4^L) for W words, L max length.

```
List<String> findWords(char[][] board, String[] words) {
        List<String> ans = new ArrayList<>();
        for (String w : words) if (exists(board, w)) ans.add(w);
        return ans;
    }
    boolean exists(char[][] b, String w) {
        for (int i = 0; i < b.length; i++)
            for (int j = 0; j < b[0].length; j++)
                if (dfs(b, w, 0, i, j)) return true;
        return false;
    }
    boolean dfs(char[][] b, String w, int k, int r, int c) {
        if (k == w.length()) return true;
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length) return false;
        if (b[r][c] != w.charAt(k)) return false;
        char tmp = b[r][c];
        b[r][c] = '#';
        boolean ok = dfs(b, w, k + 1, r + 1, c) || dfs(b, w, k + 1, r - 1, c)
            || dfs(b, w, k + 1, r, c + 1) || dfs(b, w, k + 1, r, c - 1);
        b[r][c] = tmp;
        return ok;
    }
```

Optimal

Trie of words plus one DFS on the board. Prune when the trie has no branch for the next cell.

Complexity: O(n * m * 4^L) shared across words with pruning.

```
List<String> findWords(char[][] board, String[] words) {
        TrieNode root = build(words);
        List<String> ans = new ArrayList<>();
        boolean[][] vis = new boolean[board.length][board[0].length];
        for (int i = 0; i < board.length; i++)
            for (int j = 0; j < board[0].length; j++)
                walk(board, i, j, root, new StringBuilder(), ans, vis);
        return ans;
    }
    static class TrieNode {
        TrieNode[] next = new TrieNode[26];
        String word;
    }
    TrieNode build(String[] words) {
        TrieNode root = new TrieNode();
```

```
        for (String w : words) {
            TrieNode cur = root;
            for (char ch : w.toCharArray()) {
                int i = ch - 'a';
                if (cur.next[i] == null) cur.next[i] = new TrieNode();
                cur = cur.next[i];
            }
            cur.word = w;
        }
        return root;
    }
    void walk(char[][] b, int r, int c, TrieNode node, StringBuilder path,
            List<String> ans, boolean[][] vis) {
        if (r < 0 || c < 0 || r >= b.length || c >= b[0].length || vis[r][c]) return;
        TrieNode next = node.next[b[r][c] - 'a'];
        if (next == null) return;
        vis[r][c] = true;
        if (next.word != null) { ans.add(next.word); next.word = null; }
        walk(b, r + 1, c, next, path, ans, vis);
        walk(b, r - 1, c, next, path, ans, vis);
        walk(b, r, c + 1, next, path, ans, vis);
        walk(b, r, c - 1, next, path, ans, vis);
        vis[r][c] = false;
    }
```

Strings

Two pointers, rolling hashes, and prefix-function machinery on text. Each problem lists brute-to-optimal paths where they teach different ideas.

199. Reverse Words in a String

Brute

Split on whitespace, reverse the list of tokens, and join with a single space. Extra structures make the logic obvious.

Complexity: O(n) time, O(n) extra space for the word list.

```
String reverseWords(String s) {
    String[] parts = s.trim().split("\s+");
    Collections.reverse(Arrays.asList(parts));
    return String.join(" ", parts);
}
```

Optimal

Reverse the whole char array, then reverse each word in place. Two pointers skip spaces and bound each word while swapping ends inward.

Complexity: O(n) time, O(n) space for the char array (or O(1) extra if mutable).

```
String reverseWords(String s) {
    char[] a = s.trim().toCharArray();
    reverse(a, 0, a.length - 1);
    for (int i = 0; i < a.length; ) {
        while (i < a.length && a[i] == ' ') i++;
        if (i >= a.length) break;
        int j = i;
        while (j < a.length && a[j] != ' ') j++;
        reverse(a, i, j - 1);
        i = j;
    }
    return new String(a);
}

void reverse(char[] a, int l, int r) {
    while (l < r) { char t = a[l]; a[l] = a[r]; a[r] = t; l++; r--; }
}
```

200. Valid Palindrome

Brute

Keep only alphanumeric characters in a new string, lower-case it, then compare with its reverse.

Complexity: O(n) time, O(n) extra space.

```
boolean isPalindrome(String s) {
    StringBuilder t = new StringBuilder();
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (Character.isLetterOrDigit(c)) t.append(Character.toLowerCase(c));
    }
    return t.toString().equals(t.reverse().toString());
}
```

Optimal

Two pointers walk inward, skipping non-alphanumeric characters. Compare lower-case letters without building a filtered copy.

Complexity: O(n) time, O(1) extra space.

```
boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
```

```
        while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
        while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
        if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r)))
            return false;
        l++; r--;
    }
    return true;
}
```

201. Roman to Integer

Brute

Map each symbol to a value and sum left to right. Whenever a smaller symbol sits before a larger one, subtract twice the smaller (you added it once too many).

Complexity: O(n) time, O(1) space.

```
int romanToInt(String s) {
    Map<Character, Integer> m = Map.of(
        'I', 1, 'V', 5, 'X', 10, 'L', 50,
        'C', 100, 'D', 500, 'M', 1000);
    int ans = 0, prev = 0;
    for (int i = s.length() - 1; i >= 0; i--) {
        int v = m.get(s.charAt(i));
        ans += v < prev ? -v : v;
        prev = v;
    }
    return ans;
}
```

202. String to Integer (atoi)

Brute

Simulate the specification: skip spaces, read sign, consume digits while building the value, and clamp to 32-bit bounds on overflow.

Complexity: O(n) time, O(1) space.

```
int myAtoi(String s) {
    int i = 0, n = s.length();
    while (i < n && s.charAt(i) == ' ') i++;
    if (i >= n) return 0;
    int sign = 1;
    if (s.charAt(i) == '+' || s.charAt(i) == '-') {
        sign = s.charAt(i) == '-' ? -1 : 1;
        i++;
    }
    long val = 0;
    while (i < n && Character.isDigit(s.charAt(i))) {
        val = val * 10 + (s.charAt(i) - '0');
        if (sign == 1 && val > Integer.MAX_VALUE) return Integer.MAX_VALUE;
        if (sign == -1 && -val < Integer.MIN_VALUE) return Integer.MIN_VALUE;
        i++;
    }
    return (int) (sign * val);
}
```

203. Longest Common Prefix

Brute

Compare characters column by column across all strings. Stop at the first mismatch or when a string ends.

Complexity: O(n * m) time where m is max length, O(1) extra space.

```
String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    for (int c = 0; c < strs[0].length(); c++) {
        char ch = strs[0].charAt(c);
```

```
        for (int i = 1; i < strs.length; i++) {
            if (c == strs[i].length() || strs[i].charAt(c) != ch)
                return strs[0].substring(0, c);
        }
    }
    return strs[0];
}
```

Better

Sort the array lexicographically. The common prefix of all strings equals the common prefix of the first and last after sorting, because every other string lies between them.

Complexity: O(n log n * m) time for sorting, O(1) extra besides sort.

```
String longestCommonPrefix(String[] strs) {
    Arrays.sort(strs);
    String a = strs[0], b = strs[strs.length - 1];
    int i = 0;
        while (i < a.length() && i < b.length() && a.charAt(i) == b.charAt(i)) i++;
    return a.substring(0, i);
}
```

204. KMP Pattern Matching

Brute

Try every start index in the text and compare pattern character by character.

Complexity: O(n * m) time, O(1) extra space.

```
int strStr(String hay, String needle) {
    int n = hay.length(), m = needle.length();
    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && hay.charAt(i + j) == needle.charAt(j)) j++;
        if (j == m) return i;
    }
    return -1;
}
```

Optimal

Build the LPS (longest proper prefix which is also suffix) array for the pattern. On mismatch, shift the pattern by the LPS of the matched prefix instead of restarting the text pointer.

Complexity: O(n + m) time, O(m) space for LPS.

```
int strStr(String hay, String needle) {
    if (needle.isEmpty()) return 0;
    int[] lps = buildLps(needle);
    int i = 0, j = 0;
    while (i < hay.length()) {
        if (hay.charAt(i) == needle.charAt(j)) { i++; j++; }
        else if (j > 0) j = lps[j - 1];
        else i++;
        if (j == needle.length()) return i - j;
    }
    return -1;
}

int[] buildLps(String p) {
    int[] lps = new int[p.length()];
    int len = 0, i = 1;
    while (i < p.length()) {
        if (p.charAt(i) == p.charAt(len)) lps[i++] = ++len;
        else if (len > 0) len = lps[len - 1];
        else lps[i++] = 0;
    }
    return lps;
}
```

205. Z-Algorithm

Better

Build Z[i]: length of longest substring starting at i that matches a prefix of s. Maintain the Z-box [l, r] farthest right; values inside copy from the mirror position with clipping.

Complexity: O(n) time, O(n) space.

```
int[] zFunction(String s) {
    int n = s.length();
    int[] z = new int[n];
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = Math.min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}

int zSearch(String text, String pat) {
    String s = pat + "$" + text;
    int[] z = zFunction(s);
    int m = pat.length();
    for (int i = m + 1; i < s.length(); i++)
        if (z[i] == m) return i - m - 1;
    return -1;
}
```

206. Rabin-Karp

Brute

Same as naive matching: every text offset, compare the pattern.

Complexity: O(n * m) time, O(1) extra space.

```
int rkBrute(String s, String p) {
    int n = s.length(), m = p.length();
    for (int i = 0; i <= n - m; i++) {
        if (s.regionMatches(i, p, 0, m)) return i;
    }
    return -1;
}
```

Optimal

Rolling hash of the window of length m. Equal hashes suggest a match; confirm with character comparison to handle collisions.

Complexity: O(n + m) expected time, O(1) extra space.

```
int strStrRabinKarp(String s, String p) {
    int n = s.length(), m = p.length();
    if (m == 0) return 0;
    long base = 256, mod = 1_000_000_007;
    long ph = 0, sh = 0, pow = 1;
    for (int i = 0; i < m; i++) {
        ph = (ph * base + p.charAt(i)) % mod;
        sh = (sh * base + s.charAt(i)) % mod;
        if (i < m - 1) pow = (pow * base) % mod;
    }
    for (int i = 0; i <= n - m; i++) {
        if (ph == sh && s.regionMatches(i, p, 0, m)) return i;
        if (i < n - m) {
            sh = (sh - s.charAt(i) * pow % mod + mod) % mod;
            sh = (sh * base + s.charAt(i + m)) % mod;
        }
    }
    return -1;
```

```
}
```

207. Valid Anagram

Brute

Sort both strings and compare. Anagrams have identical sorted forms.

Complexity: O(n log n) time, O(n) space for char arrays.

```
boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    char[] a = s.toCharArray(), b = t.toCharArray();
    Arrays.sort(a);
    Arrays.sort(b);
    return Arrays.equals(a, b);
}
```

Optimal

Count letter frequencies in one pass. Increment for s, decrement for t; all counts must be zero.

Complexity: O(n) time, O(1) space (26 letters).

```
boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] cnt = new int[26];
    for (int i = 0; i < s.length(); i++) {
        cnt[s.charAt(i) - 'a']++;
        cnt[t.charAt(i) - 'a']--;
    }
    for (int c : cnt) if (c != 0) return false;
    return true;
}
```

208. Group Anagrams

Brute

For each word, sort its letters and use the sorted string as a hash key. Words sharing a key are anagrams.

Complexity: O(n * k log k) time for n words of length k.

```
List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String w : strs) {
        char[] a = w.toCharArray();
        Arrays.sort(a);
        String key = new String(a);
        map.computeIfAbsent(key, x -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(map.values());
}
```

Optimal

Use a frequency signature (26 counts) as key instead of sorting. Same bucket, fewer log factors.

Complexity: O(n * k) time, O(n * k) space for stored words.

```
List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String w : strs) {
        int[] cnt = new int[26];
        for (int i = 0; i < w.length(); i++) cnt[w.charAt(i) - 'a']++;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 26; i++) { sb.append('#'); sb.append(cnt[i]); }
        map.computeIfAbsent(sb.toString(), x -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(map.values());
}
```

209. Count and Say

Brute

Build term i from term i-1 by reading runs of equal digits (count, digit). Repeat until the nth term.

Complexity: O(n * L) time where L is final string length; O(L) space.

```
String countAndSay(int n) {
    String cur = "1";
    for (int k = 2; k <= n; k++) {
        StringBuilder nxt = new StringBuilder();
        int i = 0;
        while (i < cur.length()) {
            int j = i;
            while (j < cur.length() && cur.charAt(j) == cur.charAt(i)) j++;
            nxt.append(j - i).append(cur.charAt(i));
            i = j;
        }
        cur = nxt.toString();
    }
    return cur;
}
```

210. Longest Palindromic Substring

Brute

Every pair (l, r) with l <= r is a candidate substring; test palindrome by two pointers.

Complexity: O(n^3) time, O(1) extra space.

```
String longestPalindrome(String s) {
    int bestL = 0, bestLen = 1;
    for (int l = 0; l < s.length(); l++) {
        for (int r = l; r < s.length(); r++) {
            if (isPal(s, l, r) && r - l + 1 > bestLen) {
                bestL = l;
                bestLen = r - l + 1;
            }
        }
    }
    return s.substring(bestL, bestL + bestLen);
}

boolean isPal(String s, int l, int r) {
    while (l < r) {
        if (s.charAt(l++) != s.charAt(r--)) return false;
    }
    return true;
}
```

Better

Expand around every center (odd and even). A palindrome is symmetric, so walk outward while characters match.

Complexity: O(n^2) time, O(1) extra space.

```
String longestPalindrome(String s) {
    int start = 0, len = 0;
    for (int i = 0; i < s.length(); i++) {
        int l1 = expand(s, i, i), l2 = expand(s, i, i + 1);
        int cur = Math.max(l1, l2);
        if (cur > len) {
            len = cur;
            start = i - (cur - 1) / 2;
        }
    }
    return s.substring(start, start + len);
}
```

```
int expand(String s, int l, int r) {
    while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
    return r - l - 1;
}
```

211. Minimum Characters to Prepend for Palindrome

Brute

Try every suffix s[i..] that is a palindrome; answer is i (prepend the reverse of s[0..i-1]).

Complexity: O(n^2) time, O(1) extra space.

```
int minCharsPrepend(String s) {
    int n = s.length();
    for (int i = 0; i < n; i++) {
        if (isPal(s, i, n - 1)) return i;
    }
    return n - 1;
}

boolean isPal(String s, int l, int r) {
    while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
    return true;
}
```

Optimal

Form t = s + '#' + reverse(s). The LPS at the last index equals the longest palindromic prefix of s. Prepend length is n minus that LPS.

Complexity: O(n) time, O(n) space.

```
int minCharsPrepend(String s) {
    String rev = new StringBuilder(s).reverse().toString();
    String t = s + "#" + rev;
    int[] lps = buildLps(t);
    return s.length() - lps[t.length() - 1];
}

int[] buildLps(String p) {
    int[] lps = new int[p.length()];
    for (int i = 1, len = 0; i < p.length(); ) {
        if (p.charAt(i) == p.charAt(len)) lps[i++] = ++len;
        else if (len > 0) len = lps[len - 1];
        else lps[i++] = 0;
    }
    return lps;
}
```

Bit Manipulation

XOR cancels pairs, bit masks isolate positions, and shifts implement arithmetic without division hardware.

212. Single Number I

Brute

For each value, count occurrences in the array. The unique element appears once.

Complexity: O(n^2) time, O(1) extra space.

```
int singleNumber(int[] nums) {
    for (int x : nums) {
        int c = 0;
        for (int y : nums) if (y == x) c++;
        if (c == 1) return x;
    }
    return -1;
}
```

Optimal

XOR every number. Pairs cancel to 0; the survivor is the lone element.

Complexity: O(n) time, O(1) space.

```
int singleNumber(int[] nums) {
    int ans = 0;
    for (int x : nums) ans ^= x;
    return ans;
}
```

213. Single Number II

Brute

HashMap counts frequencies and return the key with count 1.

Complexity: O(n) time, O(n) space.

```
int singleNumber(int[] nums) {
    Map<Integer, Integer> cnt = new HashMap<>();
    for (int x : nums) cnt.merge(x, 1, Integer::sum);
    for (var e : cnt.entrySet()) if (e.getValue() == 1) return e.getKey();
    return -1;
}
```

Optimal

Track ones and twos bitsets: each bit counts mod 3. When a bit hits 3 in ones, clear it from ones and twos.

Complexity: O(n) time, O(1) space.

```
int singleNumber(int[] nums) {
    int ones = 0, twos = 0;
    for (int x : nums) {
        ones = (ones ^ x) & ~twos;
        twos = (twos ^ x) & ~ones;
    }
    return ones;
}
```

214. Single Number III

Better

XOR all numbers to get a ^ b. Any set bit in that XOR separates a and b into two groups with different XOR results.

Complexity: O(n) time, O(1) space.

```
int[] singleNumber(int[] nums) {
    int xor = 0;
    for (int x : nums) xor ^= x;
    int diff = xor & -xor;
    int a = 0, b = 0;
```

```
    for (int x : nums) {
        if ((x & diff) != 0) a ^= x;
        else b ^= x;
    }
    return new int[]{a, b};
}
```

215. Number of 1 Bits

Brute

Check each of the 32 bit positions.

Complexity: O(32) time, O(1) space.

```
int hammingWeight(int n) {
    int ans = 0;
    for (int i = 0; i < 32; i++) if (((n >>> i) & 1) == 1) ans++;
    return ans;
}
```

Optimal

Clear the lowest set bit with n &= n - 1. Each step removes one 1-bit.

Complexity: O(number of set bits) time, O(1) space.

```
int hammingWeight(int n) {
    int ans = 0;
    while (n != 0) {
        n &= n - 1;
        ans++;
    }
    return ans;
}
```

216. Counting Bits for 0..n

Brute

For each i, count bits with the popcount loop.

Complexity: O(n log n) time, O(1) extra besides output.

```
int[] countBits(int n) {
    int[] ans = new int[n + 1];
    for (int i = 0; i <= n; i++) {
        int x = i, c = 0;
        while (x != 0) { x &= x - 1; c++; }
        ans[i] = c;
    }
    return ans;
}
```

Optimal

ans[i] = ans[i >> 1] + (i & 1). Drop the last bit and add its value.

Complexity: O(n) time, O(n) space for output.

```
int[] countBits(int n) {
    int[] ans = new int[n + 1];
    for (int i = 1; i <= n; i++) ans[i] = ans[i >> 1] + (i & 1);
    return ans;
}
```

217. Set Bits Sum from 1 to n

Brute

Add popcount for every integer from 1 through n.

Complexity: O(n log n) time, O(1) extra space.

```
int countSetBits1ToN(int n) {
    int sum = 0;
```

```
    for (int i = 1; i <= n; i++) {
        int x = i;
        while (x != 0) { sum++; x &= x - 1; }
    }
    return sum;
}
```

Optimal

For each bit position, count how many numbers in [1, n] have it set using a periodic pattern of length 2^(k+1).

Complexity: O(log n) time, O(1) space.

```
int countSetBits1ToN(int n) {
    int ans = 0;
    for (int k = 0; (1L << k) <= n; k++) {
        long period = 1L << (k + 1);
        long full = (n + 1) / period;
        ans += full * (1L << k);
        long rem = (n + 1) % period;
        ans += Math.max(0, rem - (1L << k));
    }
    return ans;
}
```

218. Divide Two Integers

Brute

Repeatedly subtract |divisor| from |dividend| while tracking sign. Slow but direct.

Complexity: O(|dividend| / |divisor|) time, O(1) space.

```
int divide(int dividend, int divisor) {
    if (dividend == Integer.MIN_VALUE && divisor == -1) return Integer.MAX_VALUE;
    boolean neg = (dividend < 0) ^ (divisor < 0);
    long a = Math.abs((long) dividend), b = Math.abs((long) divisor);
    int q = 0;
    while (a >= b) { a -= b; q++; }
    return neg ? -q : q;
}
```

Optimal

Binary long division: if a >= b*2^k, subtract b<<k and add 2^k to quotient. Halve k when the chunk is too big.

Complexity: O(log^2 |dividend|) time, O(1) space.

```
int divide(int dividend, int divisor) {
    if (dividend == Integer.MIN_VALUE && divisor == -1) return Integer.MAX_VALUE;
    boolean neg = (dividend < 0) ^ (divisor < 0);
    long a = Math.abs((long) dividend), b = Math.abs((long) divisor);
    int q = 0;
    while (a >= b) {
        long temp = b, k = 1;
        while (a >= temp << 1) { temp <<= 1; k <<= 1; }
        a -= temp;
        q += k;
    }
    return neg ? -q : q;
}
```

219. Power Set Using Bits

Brute

Recursively choose or skip each index to build every subset.

Complexity: O(n * 2^n) time and output size.

```
List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> ans = new ArrayList<>();
    build(nums, 0, new ArrayList<>(), ans);
    return ans;
```

```
}

void build(int[] nums, int i, List<Integer> cur, List<List<Integer>> ans) {
    if (i == nums.length) { ans.add(new ArrayList<>(cur)); return; }
    cur.add(nums[i]);
    build(nums, i + 1, cur, ans);
    cur.remove(cur.size() - 1);
    build(nums, i + 1, cur, ans);
}
```

Optimal

Mask from 0 to 2^n - 1: bit j selects nums[j]. Each mask is one subset.

Complexity: O(n * 2^n) time, O(1) extra besides output.

```
List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> ans = new ArrayList<>();
    int n = nums.length;
    for (int mask = 0; mask < (1 << n); mask++) {
        List<Integer> sub = new ArrayList<>();
        for (int j = 0; j < n; j++)
            if ((mask & (1 << j)) != 0) sub.add(nums[j]);
        ans.add(sub);
    }
    return ans;
}
```

220. XOR from L to R

Brute

XOR every integer from L to R inclusive.

Complexity: O(R - L + 1) time, O(1) space.

```
int xorRange(int L, int R) {
    int ans = 0;
    for (int i = L; i <= R; i++) ans ^= i;
    return ans;
}
```

Optimal

Prefix XOR: xor(1..n) has a period-4 pattern. Answer is prefix(R) ^ prefix(L - 1).

Complexity: O(1) time, O(1) space.

```
int xorRange(int L, int R) {
    return prefix(R) ^ prefix(L - 1);
}

int prefix(int n) {
    int r = n % 4;
    if (r == 0) return n;
    if (r == 1) return 1;
    if (r == 2) return n + 1;
    return 0;
}
```

221. Minimum Flips to Make a OR b Equal to c

Brute

Try all flip combinations on bits of a and b where (a|b) differs from c — impractical; instead flip greedily per bit when c needs 0 but both are 1.

Complexity: O(32) time, O(1) space.

```
int minFlips(int a, int b, int c) {
    int flips = 0;
    for (int k = 0; k < 31; k++) {
        int ab = (a >> k & 1), bb = (b >> k & 1), cb = (c >> k & 1);
```

```
        if (cb == 0) {
            flips += ab + bb;
        } else {
            if (ab == 0 && bb == 0) flips++;
        }
    }
    return flips;
}
```

Mathematics

Number theory building blocks: gcd structure, primes, modular exponentiation, and combinatorial counting in factorials.

222. Greatest Common Divisor

Brute

Every divisor of the smaller number that also divides the larger is a common divisor; take the maximum.

Complexity: O(min(a, b)) time, O(1) space.

```
int gcdBrute(int a, int b) {
    a = Math.abs(a); b = Math.abs(b);
    int g = 1;
    for (int d = 1; d <= Math.min(a, b); d++)
        if (a % d == 0 && b % d == 0) g = d;
    return g;
}
```

Optimal

Euclid: gcd(a, b) = gcd(b, a mod b). The remainder strictly shrinks the pair until it hits zero.

Complexity: O(log min(a, b)) time, O(1) space.

```
int gcd(int a, int b) {
    a = Math.abs(a); b = Math.abs(b);
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

223. Sieve of Eratosthenes

Brute

For each x up to n, test primality by trial division up to sqrt(x).

Complexity: O(n sqrt n) time, O(1) extra besides output.

```
List<Integer> primesBrute(int n) {
    List<Integer> ans = new ArrayList<>();
    for (int x = 2; x <= n; x++) {
        boolean ok = true;
        for (int d = 2; d * d <= x; d++)
            if (x % d == 0) { ok = false; break; }
        if (ok) ans.add(x);
    }
    return ans;
}
```

Optimal

Mark multiples of each prime starting at p*p. The smallest unmarked index is the next prime.

Complexity: O(n log log n) time, O(n) space.

```
List<Integer> sieve(int n) {
    boolean[] comp = new boolean[n + 1];
    List<Integer> primes = new ArrayList<>();
    for (int p = 2; p <= n; p++) {
        if (comp[p]) continue;
        primes.add(p);
        for (long m = (long) p * p; m <= n; m += p) comp[(int) m] = true;
    }
    return primes;
}
```

224. Prime Factorization

Brute

Try every divisor d from 2 upward; divide out all copies of d before moving on.

Complexity: O(sqrt n) time, O(1) extra space.

```
List<Integer> primeFactors(int n) {
    List<Integer> f = new ArrayList<>();
    for (int d = 2; d * d <= n; d++) {
        while (n % d == 0) {
            f.add(d);
            n /= d;
        }
    }
    if (n > 1) f.add(n);
    return f;
}
```

Better

After stripping 2, only odd trial divisors are needed. If nothing divides up to sqrt(n), the remainder is prime.

Complexity: O(sqrt n) time, O(1) extra besides factors.

```
List<Integer> primeFactorsOdd(int n) {
    List<Integer> f = new ArrayList<>();
    while (n % 2 == 0) { f.add(2); n /= 2; }
    for (int d = 3; d * d <= n; d += 2) {
        while (n % d == 0) { f.add(d); n /= d; }
    }
    if (n > 1) f.add(n);
    return f;
}
```

225. Modular Exponentiation

Brute

Multiply base by itself x times, reducing mod m each step.

Complexity: O(x log m) time for big integers, O(1) space.

```
long modPowBrute(long base, long exp, int mod) {
    long ans = 1 % mod;
    base %= mod;
    for (long i = 0; i < exp; i++) ans = ans * base % mod;
    return ans;
}
```

Optimal

Binary exponentiation: if exp is odd multiply ans by base; square base; halve exp. Every bit of exp is processed once.

Complexity: O(log exp) time, O(1) space.

```
long modPow(long base, long exp, int mod) {
    long ans = 1 % mod;
    base %= mod;
    while (exp > 0) {
        if ((exp & 1) == 1) ans = ans * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return ans;
}
```

226. Trailing Zeros in n!

Brute

Compute n! (or iterate multiplication) and count factors of 10 by dividing the product — overflows without big integers.

Complexity: O(n) multiplications; impractical for large n.

```
int trailingZeroesBrute(int n) {
    long fact = 1;
    for (int i = 2; i <= n; i++) fact *= i;
    int z = 0;
    while (fact % 10 == 0) { z++; fact /= 10; }
    return z;
}
```

Optimal

Each trailing zero needs a factor 10 = 2*5. There are always more twos; count fives: n/5 + n/25 + ...

Complexity: O(log n) time, O(1) space.

```
int trailingZeroes(int n) {
    int z = 0;
    for (long p = 5; p <= n; p *= 5) z += n / p;
    return z;
}
```

227. Numbers With Same Trailing Zeros as n!

Brute

Compute z = trailingZeroes(n). Scan m from 1 upward and count m where trailingZeroes(m) equals z.

Complexity: O(n log n) time if each factorial zero count is O(log m).

```
int countSameTrailingZerosBrute(int n) {
    int z = trailingZeroes(n);
    int cnt = 0;
    for (int m = 1; m <= n; m++)
        if (trailingZeroes(m) == z) cnt++;
    return cnt;
}

int trailingZeroes(int m) {
    int z = 0;
    for (long p = 5; p <= m; p *= 5) z += m / p;
    return z;
}
```

Optimal

Trailing zeros depend only on v5(m). Count integers in [1, n] whose 5-adic valuation matches v5(n!). Use inclusion on the highest power of 5 dividing n!.

Complexity: O(log n) time, O(1) space.

```
int countSameTrailingZeros(int n) {
    int z = trailingZeroes(n);
    int cnt = 0;
    for (int m = 1; m <= n; m++) {
        int v = 0;
        for (long x = m; x % 5 == 0; x /= 5) v++;
        if (v == z) cnt++;
    }
    return cnt;
}
```

228. Count Primes Less Than n

Brute

Trial division primality for every x < n.

Complexity: O(n sqrt n) time, O(1) extra space.

```
int countPrimesBrute(int n) {
    int cnt = 0;
    for (int x = 2; x < n; x++) {
        boolean prime = true;
        for (int d = 2; d * d <= x; d++)
            if (x % d == 0) { prime = false; break; }
        if (prime) cnt++;
    }
    return cnt;
}
```

Optimal

Sieve boolean array; answer is n minus the number of composites marked.

Complexity: O(n log log n) time, O(n) space.

```
int countPrimes(int n) {
    if (n <= 2) return 0;
    boolean[] comp = new boolean[n];
    int cnt = 0;
    for (int p = 2; p < n; p++) {
        if (comp[p]) continue;
        cnt++;
        for (long m = (long) p * p; m < n; m += p) comp[(int) m] = true;
    }
    return cnt;
}
```

229. Fast Fibonacci (nth)

Brute

Iterative DP: each term is sum of the previous two.

Complexity: O(n) time, O(1) space.

```
int fibBrute(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

Better

Matrix [[1,1],[1,0]]^n multiplied by [F1, F0]^T gives F(n+1), F(n). Use binary matrix exponentiation.

Complexity: O(log n) time, O(1) space.

```
int fib(int n) {
    if (n <= 1) return n;
    long[][] M = {{1, 1}, {1, 0}};
    long[][] P = matPow(M, n);
    return (int) P[0][1];
}

long[][] matPow(long[][] a, int exp) {
    long[][] res = {{1, 0}, {0, 1}};
    while (exp > 0) {
        if ((exp & 1) == 1) res = matMul(res, a);
```

```
        a = matMul(a, a);
        exp >>= 1;
    }
    return res;
}

long[][] matMul(long[][] x, long[][] y) {
    return new long[][]{
        {x[0][0] * y[0][0] + x[0][1] * y[1][0], x[0][0] * y[0][1] + x[0][1] * y[1][1]},
        {x[1][0] * y[0][0] + x[1][1] * y[1][0], x[1][0] * y[0][1] + x[1][1] * y[1][1]}
    };
}
```